

CONTENTS

Part 1: System Architecture

(a) System Context Diagram

(b) Module Topology Diagram

(c) Per-Module Responsibility Matrix

(d) Layering Conventions: ORM-Free Service Layer

Service Layer Structure

Rationale

Example: OrderSyncer

(e) Deployment View

Development Environment (Docker)

Staging Environment (VPS)

Deployment Directory Structure

(f) Key Architectural Decision Records (ADR Digest)

Summary

title: SDS 02a - Data Model (Domain Clusters 1-3) date: 2026-07-03 status: Draft-for-owner-review source\_of\_truth: Custom models verified from custom

System Design Specification 02a: Entity-Relationship Data Model (Clusters 1-3)

1. Domain Cluster 1: Etsy Channel Integration

1.1 ER Diagram (Mermaid)

1.2 Model Reference Table

1.3 Key Fields by Model

2. Domain Cluster 2: Listings (Multichannel Intent & Attributes)

2.1 ER Diagram (Mermaid)

2.2 Model Reference Table

2.3 Key Fields by Model

3. Domain Cluster 3: Orders + Fulfillment (Pipeline, Purchase, Picking, Tracking)

3.1 ER Diagram (Mermaid)

3.2 Model Reference Table

3.3 Key Fields by Model

4. Summary of Cluster 1-3 Relationships

## Part 1: System Architecture

### Document Metadata

title: SDS 02b - Data Model (Domain Clusters 4-5, Inheritance, State Machines) date: 2026-07-03 status: Draft-for-owner-review source\_of\_truth: Custom

### System Design Specification 02b: Entity-Relationship Data Model (Clusters 4-5, Inheritance, State Machines)

#### 5. Domain Cluster 4: Design Orders & Files (Document Control)

##### 5.1 ER Diagram (Mermaid)

##### 5.2 Model Reference Table

##### 5.3 Key Fields by Model

#### 6. Domain Cluster 5: Catalog & Product Hub (SKU, Import, Channel Status)

##### 6.1 ER Diagram (Mermaid)

##### 6.2 Model Reference Table

##### 6.3 Key Fields by Model

#### 7. Inheritance Hierarchy (Flowchart)

#### 8. State Machines (Mermaid)

##### 8.1 Sale Order Pipeline States

##### 8.2 Design Order States

##### 8.3 Gearment POD Order States (x\_gearment\_outbound\_state)

##### 8.4 Multichannel Listing States

##### 8.5 Stock Picking States (extended)

##### 8.6 Purchase Order States (extended, Gearment/Dropship)

#### 9. Computed Fields & Related Fields Summary

##### Computed (Read-Only) Fields

##### Related Fields (Stored, Linked)

#### 10. SQL Constraints (Raw init() Pattern)

##### Deployed Constraints

#### 11. Module Dependency Graph

#### 12. Summary: Key ER Relationship Patterns

##### One-to-Many (Parent → Child)

##### Many-to-One (Child → Parent)

##### Self-Join (Hierarchy)

##### Many-to-Many (via Transient/Join)

##### Soft Links (External Reference)

Document Metadata

System Design Spec — 03. External Integrations

Table of Contents

1. Etsy v3 API

1.1 Authentication

1.2 API Endpoints Used

1.3 Rate Limiting

1.4 Error Handling

1.5 Sandbox vs. Production

2. Gearment v3 API

2.1 Authentication

2.2 API Endpoints Used

2.3 Rate Limiting

2.4 Webhook: HMAC-SHA256 Verification + Replay Defense

2.5 Webhook Topics Handled

2.6 Error Handling

3. Gmail Ingestion

3.1 Purpose

3.2 Gmail API Credentials

3.3 Email Polling

3.4 Email Parser Patterns

3.5 Error Handling

4. Google Drive Integration

4.1 Purpose

4.2 Service Account Authentication

4.3 Design File Upload Cron

4.4 Shop Folder Management

4.5 Error Handling

5. GKE Logistics

5.1 Purpose

5.2 Data Import Format

|                                                                                                                                                           |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| 5.3 Carrier Detection                                                                                                                                     |  |
| 5.4 Import Cron                                                                                                                                           |  |
| 6. HTTP Controllers Inventory                                                                                                                             |  |
| 6.1 OAuth Controllers (etsy_integration)                                                                                                                  |  |
| 6.2 Webhook Controller (multichannel_hub_fulfillment)                                                                                                     |  |
| 7. Cron Jobs Inventory                                                                                                                                    |  |
| 7.1 Etsy Integration Crons                                                                                                                                |  |
| 7.2 Multichannel Hub Core Crons                                                                                                                           |  |
| 7.3 Gearment Crons                                                                                                                                        |  |
| 7.4 Orphaned Crons (No ir.cron Record)                                                                                                                    |  |
| 8. Environment Variables & System Parameters                                                                                                              |  |
| 8.1 Environment Variables (Read at Runtime)                                                                                                               |  |
| 8.2 System Parameters (ir.config_parameter)                                                                                                               |  |
| Document Metadata                                                                                                                                         |  |
| Appendix A. Quick Reference — All Routes                                                                                                                  |  |
| Appendix B. Quick Reference — All Crons                                                                                                                   |  |
| Appendix C. Quick Reference — All Env Vars                                                                                                                |  |
| Appendix D. Quick Reference — Key System Parameters                                                                                                       |  |
| title: "SDS 04a: Sequence Flows (Inbound Channels)" date: "2026-07-03" status: "Draft-for-owner-review" source_of_truth: "All flows traced from actual cc |  |
| Sequence Flows — Inbound Channels                                                                                                                         |  |
| 1. Etsy Order Ingest via API (Spec 005, P1-11)                                                                                                            |  |
| Sequence Diagram                                                                                                                                          |  |
| Prose Walkthrough                                                                                                                                         |  |
| Error Paths                                                                                                                                               |  |
| 2. Etsy Order Ingest via Email Fallback (Spec 002, Phase 0)                                                                                               |  |
| Sequence Diagram                                                                                                                                          |  |
| Prose Walkthrough                                                                                                                                         |  |
| Error Paths                                                                                                                                               |  |
| 3. Design Approval Pipeline (ESTY-244, P1-02a/P1-02b)                                                                                                     |  |
| Sequence Diagram                                                                                                                                          |  |
| Design Approval Action Sequence                                                                                                                           |  |

## Part 1: System Architecture

Prose Walkthrough — Approval Flow

Error Paths

4. Dropship Quote Handshake (ESTY-246, ADR-018-gearment-po-quote)

Sequence Diagram

State Machine Diagram

Prose Walkthrough

Error Paths

5. Gearment Webhook Inbound (P0-18b2c, ADR-015)

Sequence Diagram

Prose Walkthrough

Error Paths

6. Tracking Import from GKE Excel (P2-01/P2-02)

Sequence Diagram

Prose Walkthrough

Error Paths

Outbound & Production Completion Workflows

Audit & Observability

Document Metadata

title: "SDS 04b: Sequence Flows (Outbound Publish & Production)" date: "2026-07-03" status: "Draft-for-owner-review" source\_of\_truth: "All flows traced f

Sequence Flows — Outbound Channels & Production Completion

7. Listing Outbound Publish to Etsy (P-PUB-PUBLISH, Spec 011)

Architecture Overview

Sequence Diagram

Payload Schema Notes (Spec 011)

State Machine

Error Paths

Idempotency & Re-publish

8. Production Completion Hook (P2-03, ADR-016-production-completion)

Purpose & Context

Sequence Diagram

State Machine Context (from P1-05)

## Part 1: System Architecture

Production Locations ICP Configuration

Stock Move Semantics

Error Paths

Observability

Cross-Workflow Reference & Sequencing

Audit & Observability (Complete Set)

Audit Tables

State Machines

Observability Dashboards (Per CLAUDE.md & Phase 1)

Implementation Roadmap (Phase 3)

Document Metadata

## Part 5: Security Posture and Access Control

(a) Groups & Role Hierarchy

Group Hierarchy

(b) Roles-to-Menus Matrix

(c) Access Control List (ACL) Summary by Model

Foundation (multichannel\_hub\_core)

Fulfillment (multichannel\_hub\_fulfillment)

Etsy Channel (etsy\_integration)

Design Module (design)

(d) Record Rules (Row-Level Access Control)

multichannel\_hub\_core

etsy\_integration

multichannel\_hub\_fulfillment

(e) Sudo Usage Audit and Justifications

Audit Table: sudo() Usage Across Modules

(f) FR-017: Write-Level Defense Pattern

Pattern Implementation

(g) Webhook HMAC Signature (Gearment v3 API)

Signature Scheme

(h) OAuth2 PKCE (Etsy API)

## Part 1: System Architecture

|                          |       |
|--------------------------|-------|
| Grant Flow               | ..... |
| Key Security Properties  | ..... |
| (i) Secrets Management   | ..... |
| Secrets Inventory        | ..... |
| Best Practices           | ..... |
| (j) Compliance Checklist | ..... |
| Summary                  | ..... |





# Part 1: System Architecture

**Title:** odoo19\_esty System Context, Module Topology, and Deployment

**Date:** 2026-07-03

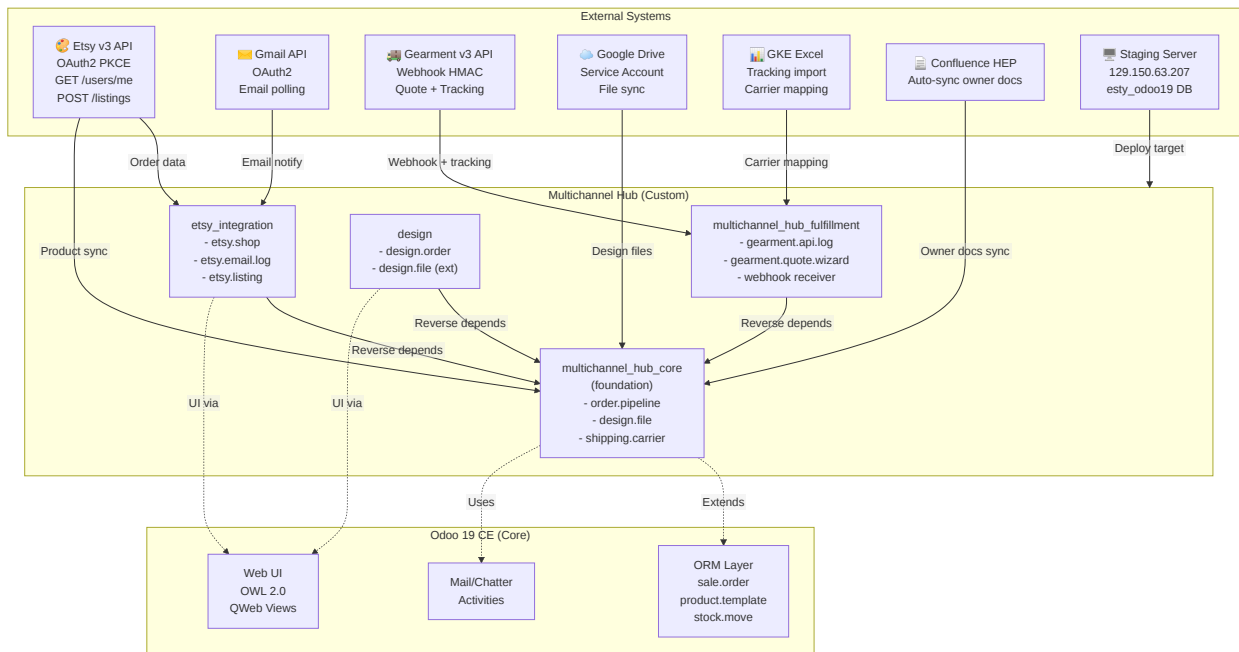
**Status:** Draft-for-owner-review

**Version:** 1.0

**Source of Truth:** Master Plan 006 tracking, .claude/plans/, custom\_addons/ manifest files, deployment/ docker-compose

## (a) System Context Diagram

The system orchestrates order ingest from Etsy, product design workflows, fulfillment coordination across Gearment (POD dropship) and internal VN production, and product catalog management (Phase 3).



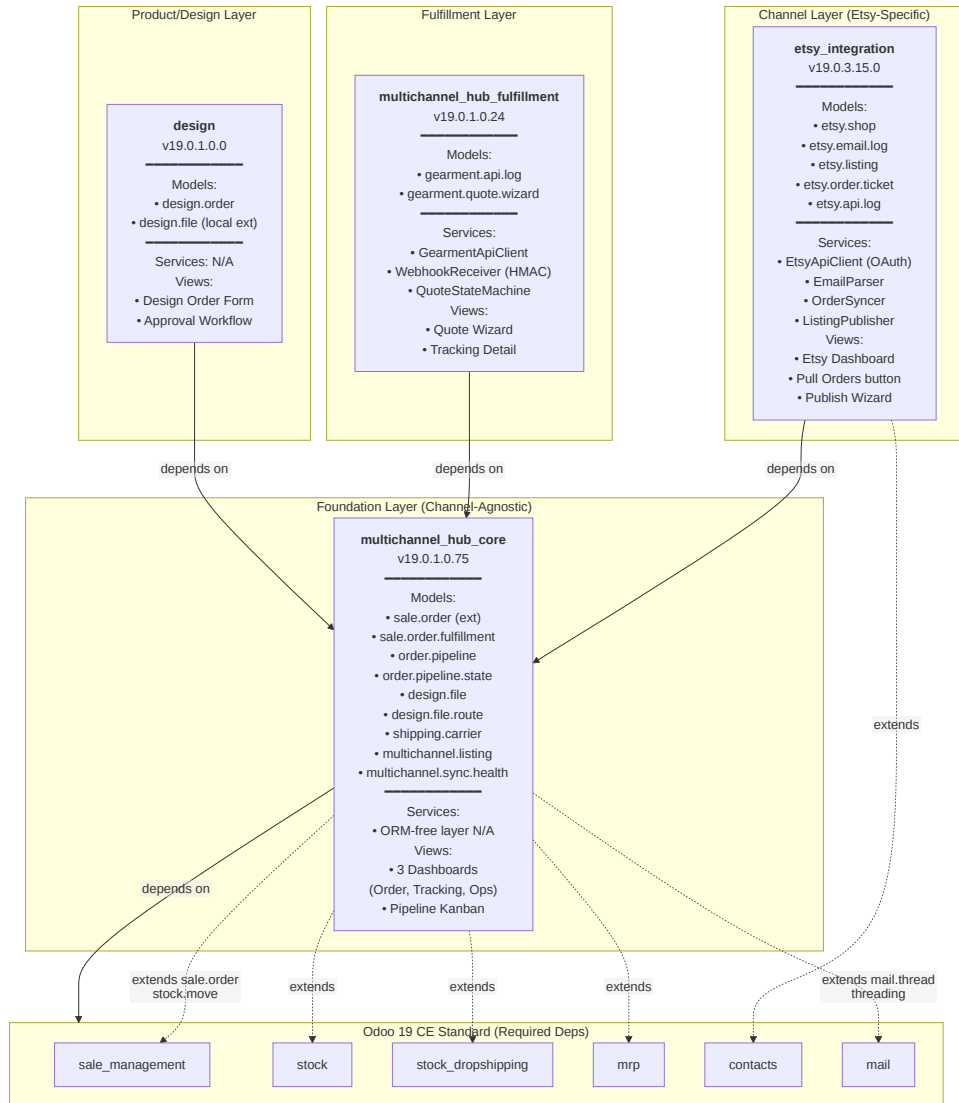
**External Dependencies** (marked by status):

- **Etsy v3 API** — ☒ Approved OAuth scopes (E1): `transactions_r/w`, `listings_r/w`, `shops_r`, `email_r`
- **Gmail API** — ☒ Legacy email fallback (Phase 2 cutover pending P2-07)
- **Gearment v3 API** — ☐ Sandbox keys obtained; production credentials pending
- **Google Drive** — ☒ Service account JSON in `secrets/`
- **GKE Excel** — ☒ Nightly import via `xlsx wizard` (P2-05)
- **Confluence HEP** — ☒ Auto-synced via `.github/hooks/post-commit`
- **Staging Server** — ☒ Live (129.150.63.207, nightly DB restore)

## (b) Module Topology Diagram

Four custom modules arranged in layers: foundation (`multichannel_hub_core`) → channel-specific (`etsy_integration`, `design`) + fulfillment (`multichannel_hub_fulfillment`) → consumers.

## Part 1: System Architecture



**Dependency Flow** (one-directional, acyclic):

- multichannel\_hub\_core → Standard Odoo modules (foundation depends on stdlib)
- etsy\_integration → multichannel\_hub\_core (channel-specific depends on foundation)
- multichannel\_hub\_fulfillment → multichannel\_hub\_core (fulfillment depends on foundation)
- design → multichannel\_hub\_core (product/order design depends on foundation)

**Why this structure?** (ADR-003)

- Isolate channel-agnostic models (pipeline, fulfillment, carrier) in \_core
- Keep Etsy-specific logic (API, email parsing, OAuth) in etsy\_integration
- Reserve future channels (amazon\_integration, website\_channel) to inherit from \_core + MHF
- Design module optional; can be disabled if order design workflows not needed

**(c) Per-Module Responsibility Matrix**

| Module                              | Ownership                         | Key Responsibility                                                                                | Critical Models                                                                                                                                                                                           | Test Coverage                           | Status             |
|-------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|--------------------|
| <b>multichannel_hub_core</b>        | Architect + BA                    | Order pipeline routing, design file management, unified dashboards, product catalog hub (Phase 3) | <code>sale.order.fulfillment</code> , <code>order.pipeline*</code> , <code>design.file</code> , <code>shipping.carrier</code> , <code>multichannel.listing</code> , <code>multichannel.sync.health</code> | 80%+ (Phase 1 complete)                 | ✓ Phase 1 live     |
| <b>etsy_integration</b>             | Etsy specialist + integrator      | Etsy shop OAuth, email ingest (legacy), listing fetch, order creation, publish flow (Phase 3)     | <code>etsy.shop</code> , <code>etsy.email.log</code> , <code>etsy.listing</code> , <code>etsy.api.log</code>                                                                                              | 80%+ (Phase 0/1 regression green)       | ✓ P1 cutover ready |
| <b>multichannel_hub_fulfillment</b> | Fulfillment + Gearment specialist | Gearment API wrapper, webhook receiver, quote state machine, tracking sync                        | <code>gearment.api.log</code> , <code>gearment.quote.wizard</code> , webhook models (transient)                                                                                                           | 80%+ (P4-01 complete)                   | ✓ P4-01 E2E pass   |
| <b>design</b>                       | Product/production team           | Design order document, approval workflow, file attachment, production routing                     | <code>design.order</code> , <code>design.file</code> (extends <code>_core</code> )                                                                                                                        | 80%+ (P0 spec, ESTY-244 implementation) | ✓ ESTY-244 live    |

**(d) Layering Conventions: ORM-Free Service Layer**

**Pattern:** Business logic lives in `_core` as ORM-free service classes; models in `multichannel_hub_core` provide ORM glue.

**SERVICE LAYER STRUCTURE**

```

custom_addons/
├── multichannel_hub_core/
│   ├── models/           # ORM models (sale_order.py, design_file.py, order_pipeline.py)
│   └── services/         # ORM-free business logic (order_pipeline_service.py, design_file_service.py)
├── etsy_integration/
│   ├── models/           # etsy.shop, etsy.email.log, etsy.listing
│   └── services/         # EtsyApiClient, EmailParser, OrderSyncer (ORM-free)
└── multichannel_hub_fulfillment/
    ├── models/           # gearment.api.log, gearment.quote.wizard
    └── services/         # GearmentApiClient, WebhookReceiver, QuoteStateMachine (ORM-free)

```

**RATIONALE**

- Testability:** Service methods can be unit-tested without Odoo ORM; no `TransactionCase` needed for parsing/crypto/API logic
- Reusability:** Services callable from wizards, crons, webhooks, browser-side JS (if APIs exposed)
- Encapsulation:** Models call services; services never call back to models (except constructor injection)
- Clarity:** ORM side effects (create, write, unlink) happen in model methods; logic-only methods in services

**EXAMPLE: ORDERSyncER**

```
# services/order_syncer.py (ORM-free)
class OrderSyncer:
    def __init__(self, partner_repo, order_repo, dedup_service):
        self.partner_repo = partner_repo
        self.order_repo = order_repo
        self.dedup_service = dedup_service

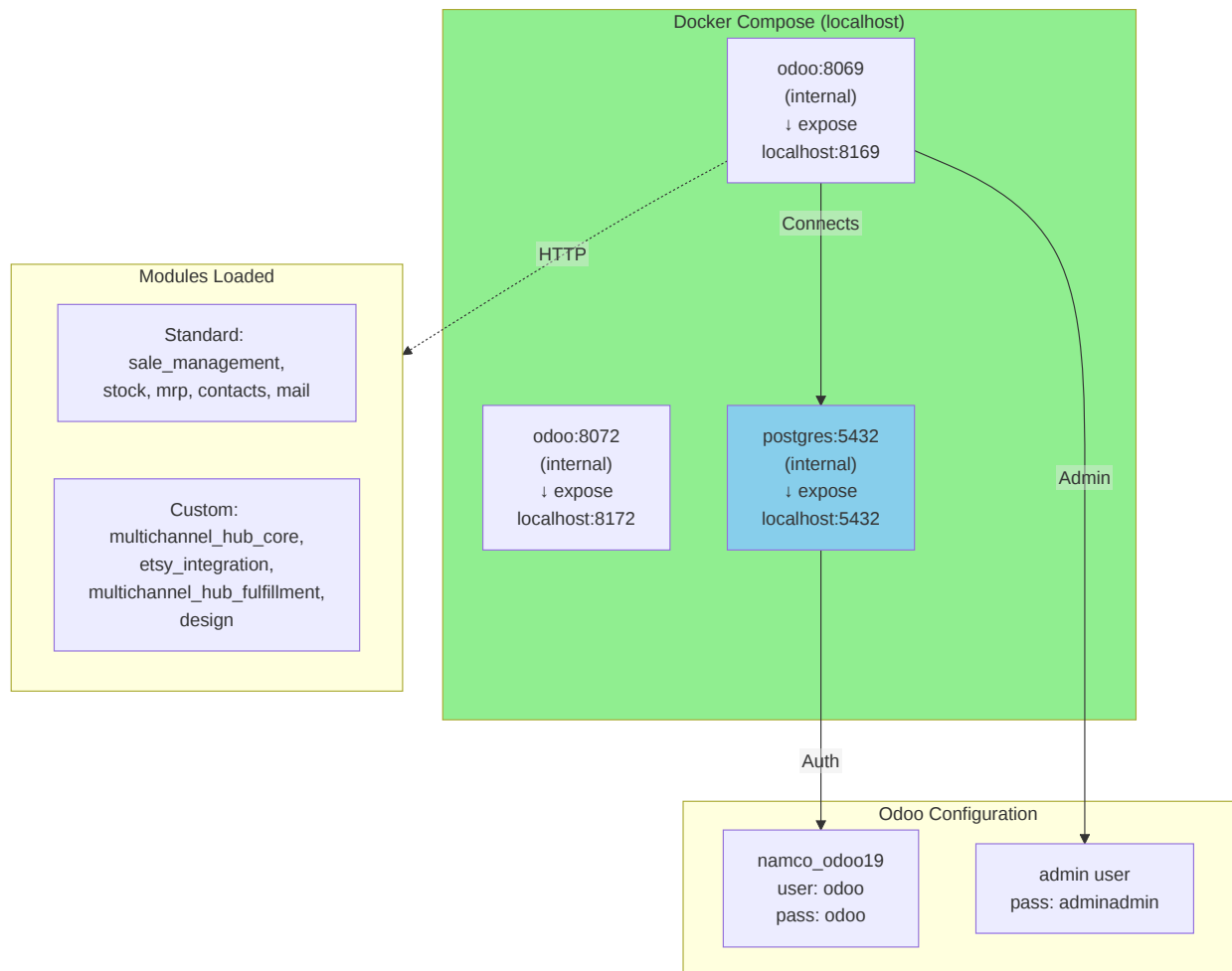
    def sync_order(self, etsy_receipt_dict) -> dict:
        """Returns {order_dict, partner_dict, conflicts} – NO ORM calls."""
        partner = self.dedup_service.resolve_or_new(etsy_receipt_dict)
        order = self._build_order_dict(etsy_receipt_dict, partner)
        return {'order': order, 'partner': partner, 'conflicts': []}

# models/order_syncer_adapter.py (ORM model wrapper)
class OrderSyncerAdapter(models.TransientModel):
    _name = 'order.syncer.adapter'

    def sync_etsy_receipt(self, etsy_receipt_dict):
        """ORM wrapper calling ORM-free service."""
        service = OrderSyncer(
            partner_repo=PartnerRepository(self.env),
            order_repo=OrderRepository(self.env),
            dedup_service=PartnerDeduplicationService()
        )
        result = service.sync_order(etsy_receipt_dict)
        # ORM-side: create/write/link
        partner = self.env['res.partner'].create(result['partner'])
        order = self.env['sale.order'].create(**result['order'], 'partner_id': partner.id)
        return order
```

**Applies to:**

- Etsy API client (API calls, data mapping)
- Email parser (regex parsing, deduplication)
- Gearment webhook receiver (HMAC validation, state machine)
- Catalog import (Excel parsing, SKU derivation)

**(e) Deployment View****DEVELOPMENT ENVIRONMENT (DOCKER)****Quick Start:**

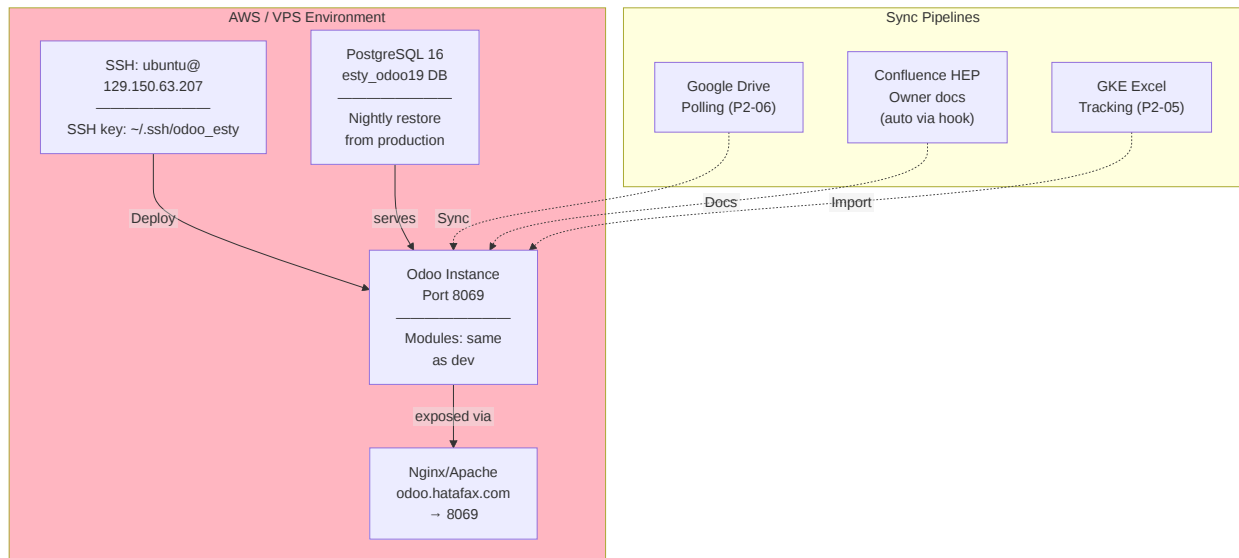
```
cd /home/odoo/odoo_dev/other_projects/odoo19_esty
docker compose build
docker compose up -d
# Wait 10-15s for Odoo startup
curl http://localhost:8169
```

**Key Ports:**

- **8169** — HTTP web interface (Odoo)
- **8172** — WebSocket (real-time dashboards)
- **5432** — PostgreSQL (direct client connections, testing)

**Environment:** See `odoo.conf` for settings (db\_name, db\_user, addons\_path, etc.)

## STAGING ENVIRONMENT (VPS)



**Deployment Process** (per `.claude/plans/006-implementation-playbook.md` Phase 5):

1. **Push to feature branch:** `feature/006-master-plan-coding`
2. **Local test:** Docker green (all tests pass, ruff clean)
3. **SSH to staging:** `ssh ubuntu@129.150.63.207`
4. **rsync modules:** `rsync -av --exclude=__pycache__ custom_addons/ ubuntu@129.150.63.207:/opt/odoo/custom_addons/` (per-module, never `--delete`)
5. **Upgrade module:** `docker exec namco_odoo19 odoo -d esty_odoo19 -u multichannel_hub_core --stop-after-init` (or per-module)
6. **Verify:** Browser test at `odoo.hatafax.com`, run E2E script `scripts/e2e_demo_drop_ship_ordertest2.py`
7. **Merge to main** (after E2E green and owner sign-off)

**Database:**

- Dev: `namco_odoo19` (docker-local, ephemeral if `docker compose down -v`)
- Staging: `esty_odoo19` (VPS, nightly restore from prod snapshot 2026-07-03)

**Secrets** (never in git):

- `.env` file on staging (GEARMENT\_API\_KEY, GEARMENT\_WEBHOOK\_HMAC\_SECRET, ETSY\_OAUTH\_CLIENT\_\*) — mount via Docker secret or chown root:101
- `secrets/` directory (Google Drive service account JSON) — git-ignored

**Logs:**

- Dev: `docker logs -f namco_odoo19`
- Staging: SSH to VPS, `tail -f /var/log/odoo/namco_odoo19.log` (or docker logs if containerized)

**DEPLOYMENT DIRECTORY STRUCTURE**

```

deployment/
├── docker-compose.yml    # Dev orchestration (3 services: odoo, postgres, redis if async)
├── Dockerfile            # Odoo container layer (built on odoo19-ce-base)
├── Dockerfile.base       # Base image (Odoo 19 CE + system deps)
├── odoo.conf             # Odoo server config (db_name, addons_path, log_level)
├── entrypoint.sh         # Container init script (db init, module install)
├── requirements.txt      # Python deps (openpyxl, requests, etc. — bundled in Odoo)
├── secrets/
│   ├── .env.example      # Template (never commit actual .env)
│   └── gearment_webhook_secret # HMAC secret (git-ignored)

```

**(f) Key Architectural Decision Records (ADR Digest)**

Detailed ADRs live in `specs/006-master-plan/adrs/`. Quick reference table below.

| ID       | Title                         | Decision One-Liner                                                                                                                                              | Consequences                                                                                                                                                | Status                          |
|----------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| ADR-001  | Odoo 19 CE Base               | Use Odoo 19 CE (no Enterprise modules)                                                                                                                          | All code must run on CE; no <code>mrp_workorder</code> Enterprise dep; use standard Dropship/MTO                                                            | ✅ Decided<br>2026-04-01         |
| ADR-003  | Four-Module Decomposition     | Split <code>etsy_integration</code> → <code>multichannel_hub_core</code> + <code>etsy_integration</code> + <code>multichannel_hub_fulfillment</code> + design   | Channel-agnostic <code>_core</code> allows future <code>amazon_channel</code> , <code>website_channel</code> without code duplication; clear deps (one-way) | ✅ Decided<br>2026-04-15         |
| ADR-004  | No Enterprise Modules         | No <code>enterprise_*</code> , no features gated to Enterprise tier                                                                                             | Rely on Dropship/MTO/quality/web_studio from CE; no timeline-gating on Enterprise features                                                                  | ✅ Decided<br>2026-04-15         |
| ADR-005  | Unified Carrier Model         | Single <code>shipping.carrier</code> master; per-channel carriers inherit/extend                                                                                | No redundant carrier seed per channel; tracking unified in <code>sale.order.fulfillment</code>                                                              | ✅ Decided<br>2026-05-02         |
| ADR-006  | GDrive Primary                | Design files + mockups + product images primary → GDrive API; <code>ir.attachment</code> (filestore) ≤2MB fallback                                              | Large filestore writes (>2MB) routed to GDrive; images ≤2MB cached locally for speed                                                                        | ✅ Decided<br>2026-05-06         |
| ADR-007  | Fulfillment Delegation        | <code>sale.order.fulfillment</code> (reverse-delegated Many2one from <code>sale.order</code> ) unifies tracking/label/production state                          | Single dashboard source; <code>label_status</code> (Selection → Many2one); <code>tracking_number</code> indexed; <code>bus.bus</code> live updates          | ✅ Decided<br>2026-05-07         |
| ADR-010  | Hybrid Dropship+MTO           | Product category field switches pipeline between Gearment dropship and Internal MTO                                                                             | Shared <code>order.pipeline</code> ; distinct states per route; no per-order toggle (category-driven)                                                       | ✅ Decided<br>2026-05-12         |
| ADR-014  | Central Product Hub (Phase 3) | Odoo as catalog source of truth; Excel bidirectional sync + Etsy API publish chain                                                                              | Phase 3 new scope; revises Phase 2 mind-set (Etsy → Odoo ingest only) to (Odoo → Etsy bidirectional)                                                        | ✅ Decided<br>2026-05-23         |
| ADR-015  | Standard-Odoo-First           | Before custom field/model, grep standard Odoo addons; if standard exists and fits, use it (owner approval required for custom)                                  | Reduces custom code; leverages CE features; owner veto = work stalls until approval                                                                         | ✅ Decided<br>2026-05-27         |
| ADR-016  | Shop Brand Voice 3-Tier       | Listing title/description/image fallback: (1) <code>etsy.listing</code> override, (2) <code>product.template</code> , (3) <code>etsy.shop</code> brand defaults | Reduces per-listing manual overrides; consistent brand across Etsy shop                                                                                     | ✅ Implemented<br>P-ENH-ESTY-190 |
| ADR-017  | Listing Currency Display      | Per-listing currency widget + nightly exchange-rate cron ( <code>OpenExchangeRates</code> API)                                                                  | Displays Etsy-reported currency independent of Odoo company; audit-friendly                                                                                 | ✅ Implemented<br>P-ENH-ESTY-195 |
| ADR-018a | Webhook HMAC v2 (Gearment)    | Webhooks signed HMAC-SHA256( <code>secret, url_path+nonce+timestamp+base64url(body)</code> ); verify before processing                                          | Prevents replay attacks; header format: <code>X-Connect-Signature: algo=sha256 sig=&lt;hex&gt;</code>                                                       | ✅ Implemented<br>P4-01          |
| ADR-018b | OAuth2 PKCE (Etsy)            | Etsy OAuth grant via <code>/users/me</code> + PKCE state code; stateless client refresh; per-user shop scope ACL                                                | No server-side session store; shop-level ACL enforced at model level ( <code>res.users.etsy_shop_ids</code> M2M)                                            | ✅ Implemented<br>P1-10/P1-11    |

**Note: ADR-018 collision** — both HMAC (Gearment) and OAuth (Etsy) numbered 018. Both documented here; Specs/015 consolidation to rename one variant (e.g., ADR-018-gearment-hmac, ADR-018-etsy-oauth).

**Summary**

The `odoo19_esty` system is built on Odoo 19 CE with four-module architecture (foundation + channel + fulfillment + design). Order ingest from Etsy flows through email parser (legacy, Phase 2 deprecates) and API polling (OAuth2 PKCE). Fulfillment routes orders to Gearment dropship (state machine + HMAC webhooks) or Internal MTO (standard Dropship route). Phase 3 adds central product hub (Odoo → Etsy publish via

API). All deployments (dev docker, staging VPS) share codebase; secrets in env vars or secrets/ (git-ignored). Minimum test coverage 80%; two-phase contract (DB schema + ORM unit). No hardcoded secrets, no Enterprise deps, no debug statements.

**Next:** See [Part 2 \(Data Model\)](#) for entity-relationship details and field inventory.

---

*Part 1 authored 2026-07-03. Mermaid diagrams render in most markdown viewers (GitHub, Confluence, Obsidian). For PDF export, render via [https://mermaid.live/SMARTPANTS\\_PRESERVED\\_1587](https://mermaid.live/SMARTPANTS_PRESERVED_1587) and save as PNG/SVG.*

---

**title: SDS 02a - Data Model (Domain Clusters 1-3) date: 2026-07-03 status: Draft-for-owner-review**  
**source of truth: Custom models verified from custom addons/\*.py code**  
**System Design Specification 02a: Entity-Relationship Data Model**  
**(Clusters 1-3)**

This document provides ER (Entity-Relationship) diagrams and field inventories for all custom Odoo 19 CE models across the four custom modules. Split into two parts: **02a covers Clusters 1-3** (Etsy channel, Listings, Orders+fulfillment); **02b covers Clusters 4-5** (Design, Catalog), inheritance hierarchy, and state machines.

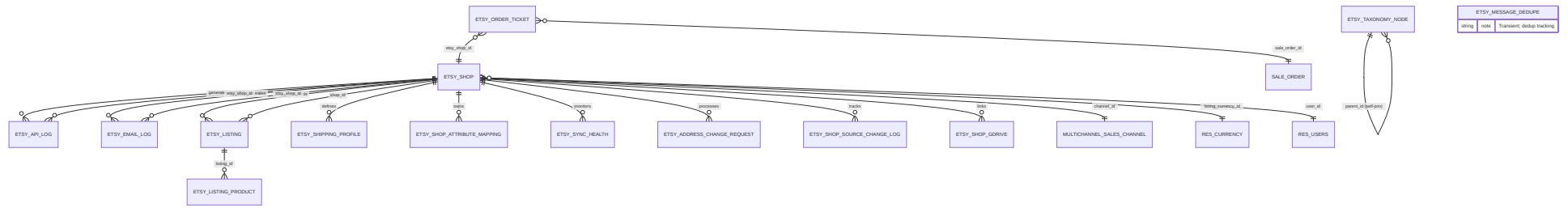
## **1. Domain Cluster 1: Etsy Channel Integration**

The Etsy channel layer manages shop registration, OAuth token storage, email/API ingestion logs, and Etsy-side taxonomies and shipping profiles.

### **1.1 ER DIAGRAM (MERMAID)**



## Part 1: System Architecture



**1.2 MODEL REFERENCE TABLE**

| <u>Model</u>                                | <u>_name</u>                                | <u>Inherits</u>                                                      | <u>Purpose</u>                                                            |
|---------------------------------------------|---------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------|
| <a href="#">Etsy Shop</a>                   | <a href="#">etsy.shop</a>                   | None                                                                 | <a href="#">Registration + OAuth + API config per shop</a>                |
| <a href="#">Etsy API Log</a>                | <a href="#">etsy.api.log</a>                | None                                                                 | <a href="#">Request/response audit for API ingestion</a>                  |
| <a href="#">Etsy Email Log</a>              | <a href="#">etsy.email.log</a>              | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Email ingestion audit trail + chatter</a>                     |
| <a href="#">Etsy Listing</a>                | <a href="#">etsy.listing</a>                | None                                                                 | <a href="#">Read-only mirror of published listings on Etsy</a>            |
| <a href="#">Etsy Listing Product</a>        | <a href="#">etsy.listing.product</a>        | None                                                                 | <a href="#">Variant/product link for each listing</a>                     |
| <a href="#">Etsy Taxonomy Node</a>          | <a href="#">etsy.taxonomy.node</a>          | None                                                                 | <a href="#">Etsy category tree (self-join hierarchy)</a>                  |
| <a href="#">Etsy Shipping Profile</a>       | <a href="#">etsy.shipping.profile</a>       | None                                                                 | <a href="#">Cached Etsy shipping profiles per shop</a>                    |
| <a href="#">Etsy Shop Attribute Mapping</a> | <a href="#">etsy.shop.attribute.mapping</a> | None                                                                 | <a href="#">Shop-level product.attribute → Etsy property_id overrides</a> |
| <a href="#">Etsy Message Dedupe</a>         | <a href="#">etsy.message.dedupe</a>         | None                                                                 | <a href="#">Transient; order-message dedup tracking</a>                   |
| <a href="#">Etsy Sync Health</a>            | <a href="#">etsy.sync.health</a>            | None                                                                 | <a href="#">Sync status + error log per shop</a>                          |
| <a href="#">Etsy Address Change Request</a> | <a href="#">etsy.address.change.request</a> | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Customer address override requests</a>                        |
| <a href="#">Etsy Shop Source Change Log</a> | <a href="#">etsy.shop.source.change.log</a> | None                                                                 | <a href="#">Audit trail of sync_mode / active_source transitions</a>      |
| <a href="#">Etsy Shop GDrive Extension</a>  | <a href="#">etsy.shop (extended)</a>        | <a href="#">etsy.shop</a>                                            | <a href="#">File upload token + GDrive folder ID per shop</a>             |

## 1.3 KEY FIELDS BY MODEL

[etsy.shop](#)

| <a href="#">Field</a>                                | <a href="#">Type</a>                    | <a href="#">Purpose</a>                           | <a href="#">Notes</a>                                                  |
|------------------------------------------------------|-----------------------------------------|---------------------------------------------------|------------------------------------------------------------------------|
| <a href="#">name</a>                                 | <a href="#">Char</a>                    | <a href="#">Shop name</a>                         | <a href="#">Required, unique, indexed</a>                              |
| <a href="#">active</a>                               | <a href="#">Boolean</a>                 | <a href="#">Active flag</a>                       | <a href="#">Default: True</a>                                          |
| <a href="#">user_id</a>                              | <a href="#">Many2one → res.users</a>    | <a href="#">Manager</a>                           | <a href="#">Order visibility scope</a>                                 |
| <a href="#">etsy_oauth_access_token</a>              | <a href="#">Char</a>                    | <a href="#">OAuth token</a>                       | <a href="#">system-group only (P0-14)</a>                              |
| <a href="#">etsy_oauth_refresh_token</a>             | <a href="#">Char</a>                    | <a href="#">Refresh token</a>                     | <a href="#">system-group only</a>                                      |
| <a href="#">etsy_oauth_token_expires_at</a>          | <a href="#">Datetime</a>                | <a href="#">Token expiry</a>                      | <a href="#">Tracked during sync</a>                                    |
| <a href="#">etsy_api_shop_id</a>                     | <a href="#">Char</a>                    | <a href="#">Etsy numeric ID</a>                   | <a href="#">Required before API use; indexed; system-group only</a>    |
| <a href="#">etsy_last_receipt_sync_at</a>            | <a href="#">Datetime</a>                | <a href="#">Sync watermark</a>                    | <a href="#">Incremental cutoff; system-group only</a>                  |
| <a href="#">sync_mode</a>                            | <a href="#">Selection</a>               | <a href="#">email only / api only</a>             | <a href="#">Adapter selector (legacy; superseded by active_source)</a> |
| <a href="#">active_source</a>                        | <a href="#">Selection</a>               | <a href="#">api / email</a>                       | <a href="#">Current upstream source; required</a>                      |
| <a href="#">active_source_changed_at</a>             | <a href="#">Datetime</a>                | <a href="#">Last switch time</a>                  | <a href="#">Auto-stamped on toggle</a>                                 |
| <a href="#">sync_audit_mode</a>                      | <a href="#">Boolean</a>                 | <a href="#">Read-only sync</a>                    | <a href="#">Pilot validation mode</a>                                  |
| <a href="#">auto_recovery</a>                        | <a href="#">Boolean</a>                 | <a href="#">Auto-failover</a>                     | <a href="#">When false, no recovery probe auto-switch</a>              |
| <a href="#">health_check_consecutive_failures</a>    | <a href="#">Integer</a>                 | <a href="#">Failover counter</a>                  | <a href="#">Reset to 0 on success</a>                                  |
| <a href="#">recovery_probe_consecutive_successes</a> | <a href="#">Integer</a>                 | <a href="#">Recovery counter</a>                  | <a href="#">Reset to 0 on failure</a>                                  |
| <a href="#">default_taxonomy_id</a>                  | <a href="#">Char</a>                    | <a href="#">Taxonomy default</a>                  | <a href="#">Etsy category; system-group only</a>                       |
| <a href="#">default_shipping_profile_id</a>          | <a href="#">Char</a>                    | <a href="#">Shipping default</a>                  | <a href="#">Etsy profile ID (int64 as Char); system-group only</a>     |
| <a href="#">default_return_policy_id</a>             | <a href="#">Char</a>                    | <a href="#">Return policy default</a>             | <a href="#">Etsy policy ID; system-group only</a>                      |
| <a href="#">default_readiness_state_id</a>           | <a href="#">Char</a>                    | <a href="#">Readiness state</a>                   | <a href="#">Etsy processing-time profile; system-group only</a>        |
| <a href="#">listing_currency_id</a>                  | <a href="#">Many2one → res.currency</a> | <a href="#">Currency override</a>                 | <a href="#">Shop listing currency (e.g., VND)</a>                      |
| <a href="#">default_who_made</a>                     | <a href="#">Selection</a>               | <a href="#">i did / someone else / collective</a> | <a href="#">Etsy listing attribute</a>                                 |
| <a href="#">default_when_made</a>                    | <a href="#">Char</a>                    | <a href="#">Production timing</a>                 | <a href="#">Etsy enum (e.g., made_to_order)</a>                        |
| <a href="#">default_is_supply</a>                    | <a href="#">Boolean</a>                 | <a href="#">Supply flag</a>                       | <a href="#">Etsy listing attribute</a>                                 |
| <a href="#">default_title</a>                        | <a href="#">Char</a>                    | <a href="#">Listing title</a>                     | <a href="#">Brand voice tier-1 fallback</a>                            |
| <a href="#">default_description</a>                  | <a href="#">Text</a>                    | <a href="#">Listing description</a>               | <a href="#">Brand voice tier-2 fallback</a>                            |
| <a href="#">default_image_1920</a>                   | <a href="#">Image</a>                   | <a href="#">Hero image</a>                        | <a href="#">Brand voice tier-3 fallback</a>                            |
| <a href="#">weight_unit_pref</a>                     | <a href="#">Selection</a>               | <a href="#">oz / g</a>                            | <a href="#">Unit for item weight; default: oz</a>                      |
| <a href="#">dimensions_unit_pref</a>                 | <a href="#">Selection</a>               | <a href="#">cm / in</a>                           | <a href="#">Unit for dimensions; default: cm</a>                       |
| <a href="#">order_ids</a>                            | <a href="#">One2many → sale.order</a>   | <a href="#">Orders linked to this shop</a>        | <a href="#">Computed</a>                                               |

## Part 1: System Architecture

| <u>Field</u>         | <u>Type</u>    | <u>Purpose</u>              | <u>Notes</u>    |
|----------------------|----------------|-----------------------------|-----------------|
| <u>order_count</u>   | <u>Integer</u> | <u>Count of orders</u>      | <u>Computed</u> |
| <u>revenue_total</u> | <u>Float</u>   | <u>Sum of order amounts</u> | <u>Computed</u> |

### Constraints:

- SQL: `UNIQUE (name)` — Shop name unique across system

### etsy.api.log

| <u>Field</u>           | <u>Type</u>                 | <u>Purpose</u>            | <u>Notes</u>                                           |
|------------------------|-----------------------------|---------------------------|--------------------------------------------------------|
| <u>etsy_shop_id</u>    | <u>Many2one → etsy.shop</u> | <u>Shop reference</u>     | <u>Required; foreign key</u>                           |
| <u>method</u>          | <u>Char</u>                 | <u>HTTP method</u>        | <u>GET, POST, PATCH, etc.</u>                          |
| <u>endpoint</u>        | <u>Char</u>                 | <u>API endpoint</u>       | <u>/v3/application/..._path</u>                        |
| <u>request_body</u>    | <u>Text</u>                 | <u>Serialized request</u> | <u>JSON or form-encoded body</u>                       |
| <u>response_status</u> | <u>Integer</u>              | <u>HTTP status</u>        | <u>200, 400, 401, 403, 404, 429, 500, etc.</u>         |
| <u>response_body</u>   | <u>Text</u>                 | <u>Full response</u>      | <u>JSON response (capped); error details preserved</u> |
| <u>duration_ms</u>     | <u>Integer</u>              | <u>Latency</u>            | <u>Request duration in milliseconds</u>                |
| <u>created_at</u>      | <u>Datetime</u>             | <u>Timestamp</u>          | <u>Auto-stamped</u>                                    |

### etsy.email.log

| <u>Field</u>              | <u>Type</u>                 | <u>Purpose</u>                          | <u>Notes</u>                                |
|---------------------------|-----------------------------|-----------------------------------------|---------------------------------------------|
| <u>etsy_shop_id</u>       | <u>Many2one → etsy.shop</u> | <u>Shop reference</u>                   | <u>Required; foreign key</u>                |
| <u>email_subject</u>      | <u>Char</u>                 | <u>Email subject</u>                    | <u>From Etsy email</u>                      |
| <u>email_from</u>         | <u>Char</u>                 | <u>Sender address</u>                   | <u>Etsy's mail address</u>                  |
| <u>email_body</u>         | <u>Text</u>                 | <u>Email body</u>                       | <u>Full raw message</u>                     |
| <u>parsed_order_count</u> | <u>Integer</u>              | <u>Orders extracted</u>                 | <u>Count of SO created/found</u>            |
| <u>parse_status</u>       | <u>Selection</u>            | <u>draft / parsing / linked / error</u> | <u>Ingest lifecycle</u>                     |
| <u>parse_error</u>        | <u>Text</u>                 | <u>Error details</u>                    | <u>Failure reason if parse_status=error</u> |
| <u>received_at</u>        | <u>Datetime</u>             | <u>Ingest time</u>                      | <u>When email arrived</u>                   |
| <u>create_date</u>        | <u>Datetime</u>             | <u>Log timestamp</u>                    | <u>Auto-stamped</u>                         |

**Inherits:** `mail.thread`, `mail.activity.mixin` — supports chatter + activities.

etsy.listing

| Field                  | Type                 | Purpose                     | Notes                        |
|------------------------|----------------------|-----------------------------|------------------------------|
| <u>etsy_shop_id</u>    | Many2one → etsy.shop | Parent shop                 | Required                     |
| <u>etsy_listing_id</u> | Char                 | Etsy's listing ID           | Primary key at Etsy; indexed |
| <u>title</u>           | Char                 | Listing title               | Read-only from Etsy          |
| <u>description</u>     | Text                 | Listing description         | Read-only from Etsy          |
| <u>price</u>           | Float                | Current price               | Last synced                  |
| <u>quantity</u>        | Integer              | Quantity available          | Inventory                    |
| <u>state</u>           | Selection            | active / inactive / expired | Etsy state                   |
| <u>last_synced_at</u>  | Datetime             | Sync timestamp              | Read-only                    |

etsy.taxonomy.node

| Field                   | Type                          | Purpose          | Notes                               |
|-------------------------|-------------------------------|------------------|-------------------------------------|
| <u>etsy_category_id</u> | Char                          | Etsy numeric ID  | Required; indexed                   |
| <u>name</u>             | Char                          | Category name    | e.g., "Clothing"                    |
| <u>parent_id</u>        | Many2one → etsy.taxonomy.node | Parent category  | Self-join hierarchy; NULL for roots |
| <u>level</u>            | Integer                       | Tree depth       | Computed                            |
| <u>path</u>             | Char                          | Full path string | e.g., "Clothing > Women's Clothing" |

etsy.shipping.profile

| Field                     | Type                 | Purpose                  | Notes                          |
|---------------------------|----------------------|--------------------------|--------------------------------|
| <u>etsy_shop_id</u>       | Many2one → etsy.shop | Parent shop              | Required                       |
| <u>etsy_profile_id</u>    | Char                 | Etsy shipping profile ID | Char (int64 overflow); indexed |
| <u>name</u>               | Char                 | Profile name             | e.g., "Domestic + Intl"        |
| <u>origin_country_iso</u> | Char                 | ISO country code         | Shipping origin                |
| <u>last_synced_at</u>     | Datetime             | Sync timestamp           | Read-only                      |

**2. Domain Cluster 2: Listings (Multichannel Intent & Attributes)**

Listings layer sits between `product.template` (the product master) and channel-specific publishing (e.g., `etsy.listing`). It captures per-channel overrides for marketing copy, images, and metadata.

**2.1 ER DIAGRAM (MERMAID)**

## Part 1: System Architecture



## 2.2 MODEL REFERENCE TABLE

| <u>Model</u>                                       | <u>name</u>                                            | <u>Inherits</u>                                                      | <u>Purpose</u>                                                              |
|----------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <a href="#">Multichannel Listing</a>               | <a href="#">multichannel.listing</a>                   | <a href="#">None</a>                                                 | <a href="#">Per-channel/per-shop listing intent (marketing overrides)</a>   |
| <a href="#">Multichannel Listing Attr. Mapping</a> | <a href="#">multichannel.listing.attribute.mapping</a> | <a href="#">None</a>                                                 | <a href="#">Per-listing attribute → Etsy property_id override (Wave 2+)</a> |
| <a href="#">Multichannel Enquiry</a>               | <a href="#">multichannel.enquiry</a>                   | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Customer inquiry (presales question)</a>                        |
| <a href="#">Multichannel Enquiry (Etsy extend)</a> | <a href="#">multichannel.enquiry (ext)</a>             | <a href="#">multichannel.enquiry</a>                                 | <a href="#">Etsy-specific fields (message ID, etc.)</a>                     |
| <a href="#">Multichannel Product Image</a>         | <a href="#">multichannel.product.image</a>             | <a href="#">None</a>                                                 | <a href="#">Hero + gallery images for product</a>                           |
| <a href="#">Multichannel Sales Channel</a>         | <a href="#">multichannel.sales.channel</a>             | <a href="#">None</a>                                                 | <a href="#">Channel master (etsy, website, etc.)</a>                        |

## 2.3 KEY FIELDS BY MODEL

### [multichannel.listing](#)

| <u>Field</u>                        | <u>Type</u>                                           | <u>Purpose</u>                                                                                       | <u>Notes</u>                                                            |
|-------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <a href="#">product_tmpl_id</a>     | <a href="#">Many2one → product.template</a>           | <a href="#">Product</a>                                                                              | <a href="#">Required; cascade delete; indexed</a>                       |
| <a href="#">channel_id</a>          | <a href="#">Many2one → multichannel.sales.channel</a> | <a href="#">Sales channel</a>                                                                        | <a href="#">Required; restrict delete; indexed</a>                      |
| <a href="#">shop_ref</a>            | <a href="#">Char</a>                                  | <a href="#">Shop identifier</a>                                                                      | <a href="#">Etsy: shop slug (e.g., "jahandmadeart"); indexed</a>        |
| <a href="#">sequence</a>            | <a href="#">Integer</a>                               | <a href="#">Display order</a>                                                                        | <a href="#">Per-template ordering; default 10</a>                       |
| <a href="#">title</a>               | <a href="#">Char</a>                                  | <a href="#">Marketing override title</a>                                                             | <a href="#">Empty → falls back to product.template.name</a>             |
| <a href="#">description</a>         | <a href="#">Text</a>                                  | <a href="#">Marketing override description</a>                                                       | <a href="#">Empty → falls back to product.template.description_sale</a> |
| <a href="#">image_1920</a>          | <a href="#">Image</a>                                 | <a href="#">Hero image override</a>                                                                  | <a href="#">Empty → falls back to product.template.image_1920</a>       |
| <a href="#">extra_image_ids</a>     | <a href="#">One2many → multichannel.product.image</a> | <a href="#">Gallery images</a>                                                                       | <a href="#">Related (shared with product)</a>                           |
| <a href="#">video_attachment_id</a> | <a href="#">Many2one → ir.attachment</a>              | <a href="#">Video file</a>                                                                           | <a href="#">Single video per listing; private attachment only</a>       |
| <a href="#">state</a>               | <a href="#">Selection</a>                             | <a href="#">draft</a> / <a href="#">ready</a> / <a href="#">published</a><br><a href="#">/ error</a> | <a href="#">Lifecycle</a>                                               |
| <a href="#">external_ref</a>        | <a href="#">Char</a>                                  | <a href="#">Channel-side ID</a>                                                                      | <a href="#">Etsy: listing_id; indexed; copy=False</a>                   |
| <a href="#">last_synced_at</a>      | <a href="#">Datetime</a>                              | <a href="#">Last publish time</a>                                                                    | <a href="#">Read-only</a>                                               |
| <a href="#">last_sync_error</a>     | <a href="#">Text</a>                                  | <a href="#">Publisher error</a>                                                                      | <a href="#">Computed from product.channel.status (read-only)</a>        |
| <a href="#">display_name</a>        | <a href="#">Char</a>                                  | <a href="#">Computed display</a>                                                                     | <a href="#">Format: "ProductName @ channel/shop"</a>                    |

### Constraints:

- [SQL UNIQUE \(raw init\(\)\)](#): [UNIQUE\(product\\_tmpl\\_id, channel\\_id, COALESCE\(shop\\_ref, ''\)\)](#) — One listing per (product, channel, shop) combination.

**multichannel.listing.attribute.mapping**

| <u>Field</u>            | <u>Type</u>                            | <u>Purpose</u>    | <u>Notes</u>                                  |
|-------------------------|----------------------------------------|-------------------|-----------------------------------------------|
| <u>listing_id</u>       | Many2one → <u>multichannel.listing</u> | Parent listing    | Required                                      |
| <u>attribute_id</u>     | Many2one → <u>product.attribute</u>    | Product attribute | e.g., "Size", "Color"                         |
| <u>etsy_property_id</u> | Char                                   | Etsy property ID  | Etsy numeric property; overrides shop default |

**multichannel.enquiry**

| <u>Field</u>         | <u>Type</u>                   | <u>Purpose</u>                                | <u>Notes</u>                       |
|----------------------|-------------------------------|-----------------------------------------------|------------------------------------|
| <u>name</u>          | Char                          | Reference number                              | Auto-generated; indexed            |
| <u>partner_id</u>    | Many2one → <u>res.partner</u> | Customer                                      | Required                           |
| <u>sale_order_id</u> | Many2one → <u>sale.order</u>  | Related SO                                    | Optional (inquiry may be pre-sale) |
| <u>subject</u>       | Char                          | Inquiry topic                                 | e.g., "Shipping question"          |
| <u>body</u>          | Text                          | Full inquiry text                             | Message body                       |
| <u>state</u>         | Selection                     | <u>draft</u> / <u>replied</u> / <u>closed</u> | Lifecycle                          |
| <u>channel_code</u>  | Char                          | Channel source                                | e.g., "etsy"                       |

**Inherits:** mail.thread, mail.activity.mixin — supports chatter.

**multichannel.product.image**

| <u>Field</u>           | <u>Type</u>                        | <u>Purpose</u>   | <u>Notes</u>                          |
|------------------------|------------------------------------|------------------|---------------------------------------|
| <u>product_tmpl_id</u> | Many2one → <u>product.template</u> | Product          | Required                              |
| <u>image</u>           | Image                              | Image binary     | Hero or gallery image                 |
| <u>sequence</u>        | Integer                            | Order in gallery | Hero first (sequence 0), then gallery |

**multichannel.sales.channel**

| <u>Field</u>  | <u>Type</u> | <u>Purpose</u>       | <u>Notes</u>                           |
|---------------|-------------|----------------------|----------------------------------------|
| <u>name</u>   | Char        | Channel display name | e.g., "Etsy Shop 1", "Company Website" |
| <u>code</u>   | Char        | Channel identifier   | e.g., "etsy", "website"; indexed       |
| <u>active</u> | Boolean     | Active flag          | Default: True                          |

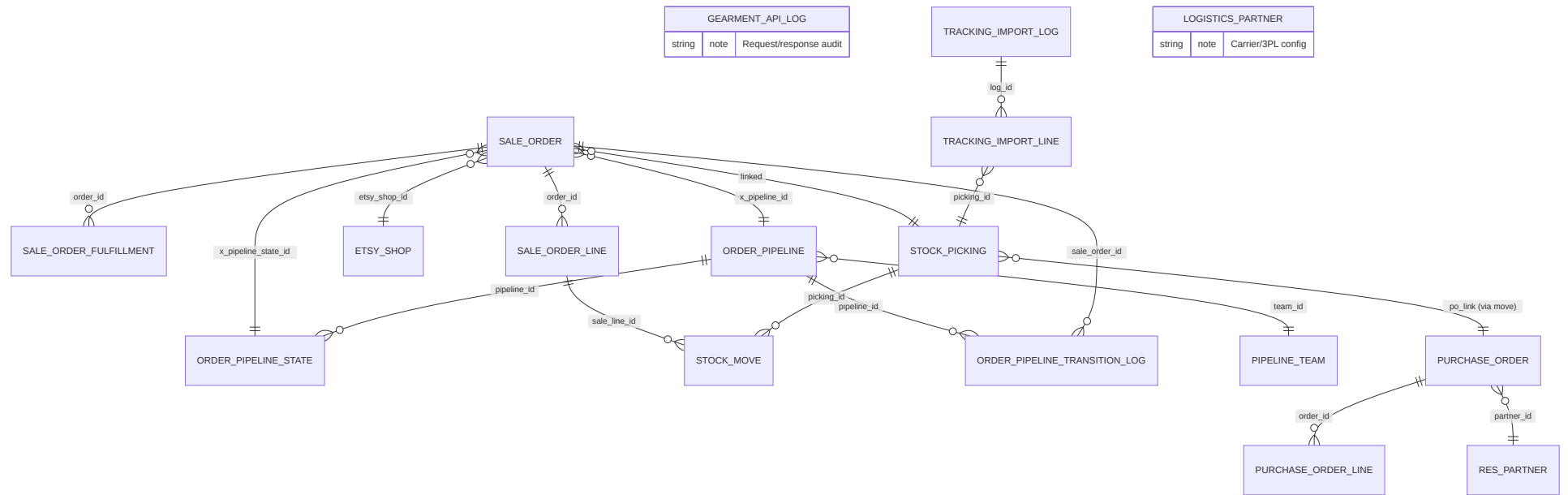
**3. Domain Cluster 3: Orders + Fulfillment (Pipeline, Purchase, Picking, Tracking)**

The order-fulfillment layer connects sale.order across three inbound pathways (Etsy email, Etsy API, manual), stages them through a configurable pipeline (VN internal, Gearment POD, hybrid MTO), and routes them to warehouse (stock.picking) or suppliers (purchase.order, gearment API).

**3.1 ER DIAGRAM (MERMAID)**



## Part 1: System Architecture



### 3.2 MODEL REFERENCE TABLE

| Model                                         | name                                                | Inherits                                                             | Purpose                                                                  |
|-----------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------|
| <a href="#">Sale Order</a>                    | <a href="#">sale.order</a> (ext)                    | <a href="#">sale.order</a>                                           | <a href="#">Etsy order ingestion + pipeline state</a>                    |
| <a href="#">Sale Order Fulfillment</a>        | <a href="#">sale.order.fulfillment</a> (core & ext) | <a href="#">mail.thread</a>                                          | <a href="#">Fulfillment orchestration (phiếu gửi hàng)</a>               |
| <a href="#">Sale Order Line</a>               | <a href="#">sale.order.line</a> (ext)               | <a href="#">sale.order.line</a>                                      | <a href="#">Line-level fulfillment details</a>                           |
| <a href="#">Stock Picking</a>                 | <a href="#">stock.picking</a> (ext)                 | <a href="#">stock.picking</a>                                        | <a href="#">Warehouse picking + ship label generation</a>                |
| <a href="#">Stock Move</a>                    | <a href="#">stock.move</a> (ext)                    | <a href="#">stock.move</a>                                           | <a href="#">Line-item inventory move</a>                                 |
| <a href="#">Purchase Order</a>                | <a href="#">purchase.order</a> (ext)                | <a href="#">purchase.order</a>                                       | <a href="#">Dropship PO to suppliers (Gearment, internal vendors)</a>    |
| <a href="#">Purchase Order Line</a>           | (std Odoo)                                          | —                                                                    | <a href="#">Line-item purchase detail</a>                                |
| <a href="#">Gearment API Log</a>              | <a href="#">gearment.api.log</a>                    | None                                                                 | <a href="#">Gearment quote/order/tracking API audit</a>                  |
| <a href="#">Tracking Import Log</a>           | <a href="#">tracking.import.log</a>                 | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Batch tracking-number ingestion</a>                          |
| <a href="#">Tracking Import Line</a>          | <a href="#">tracking.import.line</a>                | None                                                                 | <a href="#">Individual tracking number row</a>                           |
| <a href="#">Order Pipeline</a>                | <a href="#">order.pipeline</a>                      | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Pipeline master (VN Internal, Gearment, Hybrid)</a>          |
| <a href="#">Order Pipeline State</a>          | <a href="#">order.pipeline.state</a>                | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Pipeline stage (e.g., draft, quoted, confirmed, shipped)</a> |
| <a href="#">Order Pipeline Transition Log</a> | <a href="#">order.pipeline.transition.log</a>       | None                                                                 | <a href="#">Audit trail of state changes</a>                             |
| <a href="#">Pipeline Team</a>                 | <a href="#">pipeline.team</a>                       | <a href="#">mail.thread</a> ,<br><a href="#">mail.activity.mixin</a> | <a href="#">Team responsible for a pipeline stage</a>                    |
| <a href="#">Logistics Partner</a>             | <a href="#">logistics.partner</a>                   | <a href="#">mail.thread</a>                                          | <a href="#">Carrier/3PL configuration (USPS, GKE, etc.)</a>              |

### 3.3 KEY FIELDS BY MODEL

#### [sale.order \(extensions\)](#)

| Field                                     | Type                                                            | Purpose                                                                                                      | Notes                                                              |
|-------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <a href="#">etsy_shop_id</a>              | <a href="#">Many2one</a> → <a href="#">etsy.shop</a>            | <a href="#">Etsy shop source</a>                                                                             | <a href="#">NULL for non-Etsy orders</a>                           |
| <a href="#">etsy_order_id</a>             | <a href="#">Char</a>                                            | <a href="#">Etsy numeric order ID</a>                                                                        | <a href="#">Etsy's unique ID; indexed</a>                          |
| <a href="#">etsy_buyer_user_id</a>        | <a href="#">Char</a>                                            | <a href="#">Etsy buyer ID</a>                                                                                | <a href="#">Buyer's Etsy account ID</a>                            |
| <a href="#">etsy_receipt_id</a>           | <a href="#">Char</a>                                            | <a href="#">Etsy receipt ID</a>                                                                              | <a href="#">Etsy receipt identifier; indexed</a>                   |
| <a href="#">x_pipeline_id</a>             | <a href="#">Many2one</a> → <a href="#">order.pipeline</a>       | <a href="#">Pipeline</a>                                                                                     | <a href="#">e.g., "VN Internal", "Gearment POD"</a>                |
| <a href="#">x_pipeline_state_id</a>       | <a href="#">Many2one</a> → <a href="#">order.pipeline.state</a> | <a href="#">Current stage</a>                                                                                | <a href="#">e.g., "draft", "quoted", "confirmed", "shipped"</a>    |
| <a href="#">x_gearment_outbound_state</a> | <a href="#">Selection</a>                                       | <a href="#">draft</a> / <a href="#">quoted</a> / <a href="#">operator review</a> / <a href="#">confirmed</a> | <a href="#">Gearment POD-specific state (if pipeline=Gearment)</a> |
| <a href="#">x_gearment_po_id</a>          | <a href="#">Char</a>                                            | <a href="#">Gearment quote/order ID</a>                                                                      | <a href="#">External Gearment ID; indexed</a>                      |
| <a href="#">x_production_notes</a>        | <a href="#">Text</a>                                            | <a href="#">Internal notes</a>                                                                               | <a href="#">Production team comments</a>                           |

sale.order.fulfillment

| <u>Field</u>              | <u>Type</u>                      | <u>Purpose</u>                           | <u>Notes</u>                                       |
|---------------------------|----------------------------------|------------------------------------------|----------------------------------------------------|
| <u>name</u>               | <u>Char</u>                      | <u>Document number</u>                   | <u>Auto-assigned; "SOF" prefix</u>                 |
| <u>order_id</u>           | <u>Many2one → sale.order</u>     | <u>Parent SO</u>                         | <u>Required</u>                                    |
| <u>fulfillment_type</u>   | <u>Selection</u>                 | <u>internal_prod / dropship / hybrid</u> | <u>Production route</u>                            |
| <u>state</u>              | <u>Selection</u>                 | <u>Lifecycle states</u>                  | <u>Computed from stock.picking, purchase.order</u> |
| <u>picking_ids</u>        | <u>One2many → stock.picking</u>  | <u>Warehouse picks</u>                   | <u>Outbound pickings for this fulfillment</u>      |
| <u>purchase_order_ids</u> | <u>One2many → purchase.order</u> | <u>Supplier orders</u>                   | <u>Dropship POs linked to this fulfillment</u>     |

Inherits: mail.thread — chatter enabled.

stock.picking (extensions)

| <u>Field</u>                         | <u>Type</u>                         | <u>Purpose</u>            | <u>Notes</u>                                  |
|--------------------------------------|-------------------------------------|---------------------------|-----------------------------------------------|
| <u>x_gearment_shipping_label_url</u> | <u>Char</u>                         | <u>Label download</u>     | <u>URL to Gearment's shipping label PDF</u>   |
| <u>x_logistics_partner_id</u>        | <u>Many2one → logistics.partner</u> | <u>Carrier</u>            | <u>USPS, GKE, etc.</u>                        |
| <u>x_tracking_number</u>             | <u>Char</u>                         | <u>Carrier tracking</u>   | <u>Auto-imported from tracking.import.log</u> |
| <u>x_tracking_imported_at</u>        | <u>Datetime</u>                     | <u>Tracking sync time</u> | <u>When tracking was injected</u>             |

purchase.order (extensions)

| <u>Field</u>                     | <u>Type</u>                  | <u>Purpose</u>           | <u>Notes</u>                      |
|----------------------------------|------------------------------|--------------------------|-----------------------------------|
| <u>x_sale_order_id</u>           | <u>Many2one → sale.order</u> | <u>Linked SO</u>         | <u>Dropship context</u>           |
| <u>x_gearment_quote_id</u>       | <u>Char</u>                  | <u>Gearment quote ID</u> | <u>If vendor is Gearment</u>      |
| <u>x_gearment_order_id</u>       | <u>Char</u>                  | <u>Gearment order ID</u> | <u>After PO confirmation</u>      |
| <u>x_gearment_quote_state</u>    | <u>Selection</u>             | <u>Quote lifecycle</u>   | <u>Mirrored from Gearment API</u> |
| <u>x_requested_delivery_date</u> | <u>Date</u>                  | <u>Target delivery</u>   | <u>Customer-promised date</u>     |

order.pipeline

| <u>Field</u>              | <u>Type</u>                                     | <u>Purpose</u>             | <u>Notes</u>                                        |
|---------------------------|-------------------------------------------------|----------------------------|-----------------------------------------------------|
| <u>name</u>               | <u>Char</u>                                     | <u>Pipeline name</u>       | <u>e.g., "VN Internal Production"</u>               |
| <u>code</u>               | <u>Char</u>                                     | <u>Pipeline identifier</u> | <u>e.g., "vn_internal", "gearment_pod"; indexed</u> |
| <u>active</u>             | <u>Boolean</u>                                  | <u>Active flag</u>         | <u>Default: True</u>                                |
| <u>team_id</u>            | <u>Many2one → pipeline.team</u>                 | <u>Responsible team</u>    | <u>e.g., "Production Team"</u>                      |
| <u>state_ids</u>          | <u>One2many → order.pipeline.state</u>          | <u>Stages</u>              | <u>Ordered list of workflow stages</u>              |
| <u>transition_log_ids</u> | <u>One2many → order.pipeline.transition.log</u> | <u>Audit</u>               | <u>State change history</u>                         |

Inherits: mail.thread, mail.activity.mixin.

order.pipeline.state

| <u>Field</u>               | <u>Type</u>                      | <u>Purpose</u>         | <u>Notes</u>                                      |
|----------------------------|----------------------------------|------------------------|---------------------------------------------------|
| <u>name</u>                | <u>Char</u>                      | <u>Stage name</u>      | <u>e.g., "Quote Pending"</u>                      |
| <u>code</u>                | <u>Char</u>                      | <u>Stage code</u>      | <u>e.g., "quoted", "confirmed"; indexed</u>       |
| <u>pipeline_id</u>         | <u>Many2one → order.pipeline</u> | <u>Parent pipeline</u> | <u>Required</u>                                   |
| <u>sequence</u>            | <u>Integer</u>                   | <u>Stage order</u>     | <u>Determines workflow progression</u>            |
| <u>allow_confirm</u>       | <u>Boolean</u>                   | <u>Confirmable</u>     | <u>If true, SO can be confirmed at this stage</u> |
| <u>auto_create_picking</u> | <u>Boolean</u>                   | <u>Auto-picking</u>    | <u>If true, transition creates stock.picking</u>  |
| <u>auto_create_invoice</u> | <u>Boolean</u>                   | <u>Auto-invoice</u>    | <u>If true, transition triggers invoicing</u>     |

order.pipeline.transition.log

| <u>Field</u>              | <u>Type</u>                            | <u>Purpose</u>            | <u>Notes</u>                      |
|---------------------------|----------------------------------------|---------------------------|-----------------------------------|
| <u>sale_order_id</u>      | <u>Many2one → sale.order</u>           | <u>Order reference</u>    | <u>FK</u>                         |
| <u>pipeline_id</u>        | <u>Many2one → order.pipeline</u>       | <u>Pipeline context</u>   | <u>FK</u>                         |
| <u>old_state_id</u>       | <u>Many2one → order.pipeline.state</u> | <u>Previous stage</u>     | <u>NULL if first transition</u>   |
| <u>new_state_id</u>       | <u>Many2one → order.pipeline.state</u> | <u>New stage</u>          | <u>Required</u>                   |
| <u>changed_by_user_id</u> | <u>Many2one → res.users</u>            | <u>Who changed</u>        | <u>Auto-stamped</u>               |
| <u>changed_at</u>         | <u>Datetime</u>                        | <u>Transition time</u>    | <u>Auto-stamped</u>               |
| <u>change_type</u>        | <u>Selection</u>                       | <u>manual / automatic</u> | <u>User action or system sync</u> |
| <u>notes</u>              | <u>Text</u>                            | <u>Reason/details</u>     | <u>Optional audit note</u>        |

gearment.api.log

| <u>Field</u>             | <u>Type</u>                      | <u>Purpose</u>      | <u>Notes</u>                              |
|--------------------------|----------------------------------|---------------------|-------------------------------------------|
| <u>purchase_order_id</u> | <u>Many2one → purchase.order</u> | <u>Context</u>      | <u>Required FK</u>                        |
| <u>method</u>            | <u>Char</u>                      | <u>HTTP method</u>  | <u>GET, POST, PATCH</u>                   |
| <u>endpoint</u>          | <u>Char</u>                      | <u>API endpoint</u> | <u>Gearment API path</u>                  |
| <u>request_body</u>      | <u>Text</u>                      | <u>Request</u>      | <u>JSON body</u>                          |
| <u>response_status</u>   | <u>Integer</u>                   | <u>HTTP status</u>  | <u>200, 400, 401, 409, 500, etc.</u>      |
| <u>response_body</u>     | <u>Text</u>                      | <u>Response</u>     | <u>JSON response (truncated if large)</u> |
| <u>duration_ms</u>       | <u>Integer</u>                   | <u>Latency</u>      | <u>Milliseconds</u>                       |
| <u>created_at</u>        | <u>Datetime</u>                  | <u>Timestamp</u>    | <u>Auto-stamped</u>                       |

tracking.import.log

| <u>Field</u>           | <u>Type</u>      | <u>Purpose</u>                                | <u>Notes</u>                                 |
|------------------------|------------------|-----------------------------------------------|----------------------------------------------|
| <u>name</u>            | <u>Char</u>      | <u>Import reference</u>                       | <u>e.g., "GKE Tracking Batch 2026-07-03"</u> |
| <u>import_source</u>   | <u>Char</u>      | <u>Source system</u>                          | <u>e.g., "gke", "usps", "manual"</u>         |
| <u>import_file</u>     | <u>Binary</u>    | <u>Uploaded file</u>                          | <u>CSV or JSON</u>                           |
| <u>import_status</u>   | <u>Selection</u> | <u>draft / processing / completed / error</u> | <u>Lifecycle</u>                             |
| <u>processed_count</u> | <u>Integer</u>   | <u>Successfully imported</u>                  | <u>Count of tracking numbers linked</u>      |
| <u>error_count</u>     | <u>Integer</u>   | <u>Failed rows</u>                            | <u>Count of failed imports</u>               |
| <u>import_error</u>    | <u>Text</u>      | <u>Error details</u>                          | <u>First error message</u>                   |
| <u>imported_at</u>     | <u>Datetime</u>  | <u>Completion time</u>                        | <u>Auto-stamped</u>                          |

Inherits: mail.thread, mail.activity.mixin.tracking.import.line

| <u>Field</u>           | <u>Type</u>                           | <u>Purpose</u>                  | <u>Notes</u>                            |
|------------------------|---------------------------------------|---------------------------------|-----------------------------------------|
| <u>log_id</u>          | <u>Many2one → tracking.import.log</u> | <u>Parent import</u>            | <u>Required</u>                         |
| <u>picking_id</u>      | <u>Many2one → stock.picking</u>       | <u>Target picking</u>           | <u>Links tracking to warehouse pick</u> |
| <u>tracking_number</u> | <u>Char</u>                           | <u>Carrier tracking</u>         | <u>e.g., "1Z999AA10123456784"</u>       |
| <u>carrier_code</u>    | <u>Char</u>                           | <u>Carrier identifier</u>       | <u>e.g., "gke", "usps"</u>              |
| <u>import_status</u>   | <u>Selection</u>                      | <u>pending / linked / error</u> | <u>Line status</u>                      |
| <u>import_error</u>    | <u>Text</u>                           | <u>Error reason</u>             | <u>If link failed</u>                   |

logistics.partner

| <u>Field</u>                  | <u>Type</u>    | <u>Purpose</u>          | <u>Notes</u>                              |
|-------------------------------|----------------|-------------------------|-------------------------------------------|
| <u>name</u>                   | <u>Char</u>    | <u>Carrier/3PL name</u> | <u>e.g., "GKE Logistics"</u>              |
| <u>code</u>                   | <u>Char</u>    | <u>Carrier code</u>     | <u>e.g., "gke", "usps"; indexed</u>       |
| <u>active</u>                 | <u>Boolean</u> | <u>Active flag</u>      | <u>Default: True</u>                      |
| <u>api_endpoint</u>           | <u>Char</u>    | <u>API URL</u>          | <u>For automated tracking/label pulls</u> |
| <u>api_key</u>                | <u>Char</u>    | <u>API credential</u>   | <u>system-group only</u>                  |
| <u>tracking_number_format</u> | <u>Char</u>    | <u>Regex pattern</u>    | <u>Validate incoming tracking numbers</u> |

Inherits: mail.thread.pipeline.team

| <u>Field</u>        | <u>Type</u>                      | <u>Purpose</u>      | <u>Notes</u>                        |
|---------------------|----------------------------------|---------------------|-------------------------------------|
| <u>name</u>         | <u>Char</u>                      | <u>Team name</u>    | <u>e.g., "Production Team"</u>      |
| <u>member_ids</u>   | <u>Many2many → res.users</u>     | <u>Team members</u> | <u>Users in this team</u>           |
| <u>pipeline_ids</u> | <u>One2many → order.pipeline</u> | <u>Pipelines</u>    | <u>Pipelines this team operates</u> |

Inherits: mail.thread, mail.activity.mixin.

#### 4. Summary of Cluster 1-3 Relationships

| <u>Relationship</u>                           | <u>Source</u>                                 | <u>Target</u>                          | <u>Multiplicity</u> | <u>Purpose</u>                                            |
|-----------------------------------------------|-----------------------------------------------|----------------------------------------|---------------------|-----------------------------------------------------------|
| <a href="#">Shop → Orders</a>                 | <a href="#">etsy.shop</a>                     | <a href="#">sale.order</a>             | <a href="#">1:N</a> | <a href="#">Each order ingested from a shop</a>           |
| <a href="#">Shop → Listings</a>               | <a href="#">etsy.shop</a>                     | <a href="#">etsy.listing</a>           | <a href="#">1:N</a> | <a href="#">Shop hosts multiple listings</a>              |
| <a href="#">Listing → Product</a>             | <a href="#">etsy.listing</a>                  | <a href="#">etsy.listing.product</a>   | <a href="#">1:N</a> | <a href="#">Listing variants</a>                          |
| <a href="#">Product → Listing Intent</a>      | <a href="#">product.template</a>              | <a href="#">multichannel.listing</a>   | <a href="#">1:N</a> | <a href="#">Multiple channels per product</a>             |
| <a href="#">Order → Fulfillment</a>           | <a href="#">sale.order</a>                    | <a href="#">sale.order.fulfillment</a> | <a href="#">1:N</a> | <a href="#">May split into multiple fulfillments</a>      |
| <a href="#">Fulfillment → Picking</a>         | <a href="#">sale.order.fulfillment</a>        | <a href="#">stock.picking</a>          | <a href="#">1:N</a> | <a href="#">Warehouse pick per fulfillment line</a>       |
| <a href="#">Order → Pipeline</a>              | <a href="#">sale.order</a>                    | <a href="#">order.pipeline</a>         | <a href="#">N:1</a> | <a href="#">Order assigned to one pipeline</a>            |
| <a href="#">Pipeline → States</a>             | <a href="#">order.pipeline</a>                | <a href="#">order.pipeline.state</a>   | <a href="#">1:N</a> | <a href="#">Pipeline defines its workflow</a>             |
| <a href="#">Order State Change → Audit</a>    | <a href="#">order.pipeline.transition.log</a> | <a href="#">sale.order</a>             | <a href="#">N:1</a> | <a href="#">All transitions logged</a>                    |
| <a href="#">Picking → Tracking</a>            | <a href="#">stock.picking</a>                 | <a href="#">tracking.import.line</a>   | <a href="#">1:N</a> | <a href="#">Multiple tracking numbers? (1:1 expected)</a> |
| <a href="#">Purchase Order → Gearment Log</a> | <a href="#">purchase.order</a>                | <a href="#">gearment.api.log</a>       | <a href="#">1:N</a> | <a href="#">Each API call logged</a>                      |

#### Document Metadata

- **Date:** [2026-07-03](#)
- **Status:** [Draft for owner review](#)
- **Odoo Version:** [19.0 CE](#)
- **Module Scope:** [etsy integration](#), [multichannel hub core](#), [multichannel hub fulfillment \(partial\)](#), [design \(partial\)](#)
- **Related SDS:** [02b-data-model.md](#) (Clusters 4-5, inheritance hierarchy, state machines)
- **Code Verification:** All models, fields, and relationships verified against [/home/odoo/odoo\\_dev/other\\_projects/odoo19\\_esty/custom\\_addons/](#) [source code on 2026-07-03](#).

**title:** [SDS 02b - Data Model \(Domain Clusters 4-5, Inheritance, State Machines\)](#) **date:** [2026-07-03](#)  
**status:** [Draft-for-owner-review](#) **source of truth:** [Custom models verified from custom\\_addons/\\*.py code](#)

## **System Design Specification 02b: Entity-Relationship Data Model (Clusters 4-5, Inheritance, State Machines)**

Continuation of 02a. This document covers the Design cluster, Catalog/Product hub cluster, the full Odoo inheritance hierarchy, and state machines across all models.

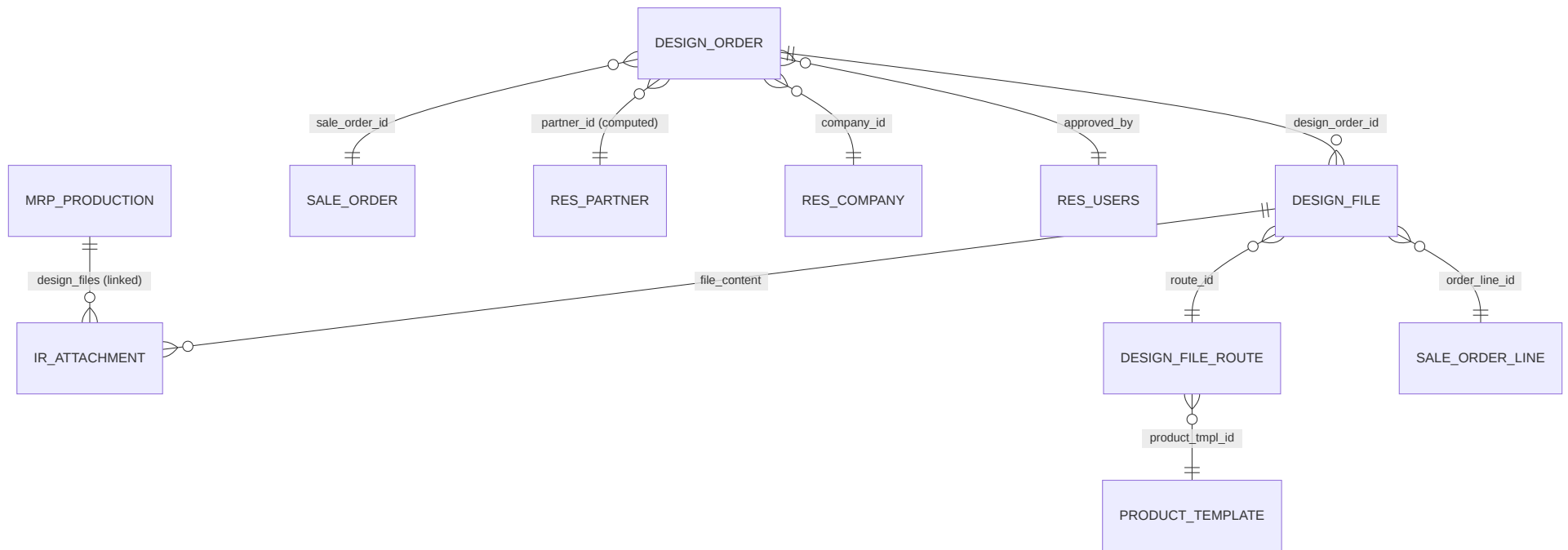
---

### **5. Domain Cluster 4: Design Orders & Files (Document Control)**

Design orders ( `design.order` ) are first-class documents parallel to `sale.order` and `mrp.production` . They track the approval flow for customer-submitted artwork, organize design files, and attach approved files to manufacturing orders.

#### **5.1 ER DIAGRAM (MERMAID)**

## Part 1: System Architecture





## 5.2 MODEL REFERENCE TABLE

| <u>Model</u>                              | <u>_name</u>                           | <u>Inherits</u>                                                | <u>Purpose</u>                                                         |
|-------------------------------------------|----------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------|
| <a href="#">Design Order</a>              | <code>design.order</code>              | <code>mail.thread</code> ,<br><code>mail.activity.mixin</code> | <a href="#">Document for design approval flow (phiếu design)</a>       |
| <a href="#">Design File (core)</a>        | <code>design.file</code>               | <code>mail.thread</code> ,<br><code>mail.activity.mixin</code> | <a href="#">Design file (artwork, mockup, proof)</a>                   |
| <a href="#">Design File (design ext)</a>  | <code>design.file</code>               | <code>design.file</code> ( inherit)                            | <a href="#">Extension with design_order_id link</a>                    |
| <a href="#">Design File Route</a>         | <code>design.file.route</code>         | <code>mail.thread</code> ,<br><code>mail.activity.mixin</code> | <a href="#">Route template for design uploads (deprecated/archive)</a> |
| <a href="#">Design File Upload Wizard</a> | <code>design.file.upload.wizard</code> | —                                                              | <a href="#">Transient; file upload form</a>                            |
| <a href="#">Sale Order (design ext)</a>   | <code>sale.order</code> ( inherit)     | <code>sale.order</code>                                        | <a href="#">Extensions: auto-create design orders</a>                  |

## 5.3 KEY FIELDS BY MODEL

### [design.order](#)

| <u>Field</u>                         | <u>Type</u>                            | <u>Purpose</u>                                                                                 | <u>Notes</u>                                                                  |
|--------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <a href="#">name</a>                 | <a href="#">Char</a>                   | <a href="#">Document reference</a>                                                             | <a href="#">Auto-assigned from sequence; readonly after creation; tracked</a> |
| <a href="#">sale_order_id</a>        | <a href="#">Many2one → sale.order</a>  | <a href="#">Parent SO</a>                                                                      | <a href="#">Required; cascade delete; indexed; tracked</a>                    |
| <a href="#">partner_id</a>           | <a href="#">Many2one → res.partner</a> | <a href="#">Customer</a>                                                                       | <a href="#">Stored, computed from sale_order_id.partner_id; readonly</a>      |
| <a href="#">company_id</a>           | <a href="#">Many2one → res.company</a> | <a href="#">Company</a>                                                                        | <a href="#">Required; defaults to current company; indexed</a>                |
| <a href="#">state</a>                | <a href="#">Selection</a>              | <code>pending</code> / <code>proof sent</code> / <code>approved</code> / <code>rejected</code> | <a href="#">Readonly after pending; tracked; indexed; default: pending</a>    |
| <a href="#">design_file_ids</a>      | <a href="#">One2many → design.file</a> | <a href="#">Child files</a>                                                                    | <a href="#">All design files linked to this order</a>                         |
| <a href="#">design_files_count</a>   | <a href="#">Integer</a>                | <a href="#">Total file count</a>                                                               | <a href="#">Computed; readonly</a>                                            |
| <a href="#">approved_files_count</a> | <a href="#">Integer</a>                | <a href="#">Approved file count</a>                                                            | <a href="#">Computed; readonly</a>                                            |
| <a href="#">approved_by</a>          | <a href="#">Many2one → res.users</a>   | <a href="#">Approver</a>                                                                       | <a href="#">Readonly; null</a>                                                |
| <a href="#">approved_at</a>          | <a href="#">Datetime</a>               | <a href="#">Approval timestamp</a>                                                             | <a href="#">Readonly; auto-stamped</a>                                        |
| <a href="#">rejection_reason</a>     | <a href="#">Text</a>                   | <a href="#">Rejection details</a>                                                              | <a href="#">Shown when state=rejected; tracked</a>                            |
| <a href="#">create_date</a>          | <a href="#">Datetime</a>               | <a href="#">Creation timestamp</a>                                                             | <a href="#">Auto-stamped</a>                                                  |
| <a href="#">write_date</a>           | <a href="#">Datetime</a>               | <a href="#">Last modification</a>                                                              | <a href="#">Auto-stamped</a>                                                  |

### Constraints:

- SQL: `UNIQUE(sale_order_id)` — One design order per SO (enforced via init() raw SQL per project pattern).

**design.file**

| Field                              | Type                                         | Purpose                                                                                                  | Notes                                                                 |
|------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <a href="#">name</a>               | <a href="#">Char</a>                         | <a href="#">File name</a>                                                                                | <a href="#">Display name</a> ; <a href="#">required</a>               |
| <a href="#">order_id</a>           | <a href="#">Many2one → sale.order</a>        | <a href="#">Header-level link</a>                                                                        | <a href="#">Design file at SO level</a> (NULL if line-level)          |
| <a href="#">order_line_id</a>      | <a href="#">Many2one → sale.order.line</a>   | <a href="#">Line-level link</a>                                                                          | <a href="#">Design file at line level</a> (NULL if header-level)      |
| <a href="#">design_order_id</a>    | <a href="#">Many2one → design.order</a>      | <a href="#">Parent order</a>                                                                             | <a href="#">Extension field</a> (added by design module)              |
| <a href="#">storage_mode</a>       | <a href="#">Selection</a>                    | <a href="#">small</a> / <a href="#">url</a> / <a href="#">gdrive</a>                                     | <a href="#">Where the file is stored</a>                              |
| <a href="#">file_url</a>           | <a href="#">Char</a>                         | <a href="#">External URL</a>                                                                             | If <a href="#">storage_mode=url</a> ; <a href="#">clickable link</a>  |
| <a href="#">gdrive_preview_url</a> | <a href="#">Char</a>                         | <a href="#">GDrive preview</a>                                                                           | If <a href="#">storage_mode=gdrive</a> ; <a href="#">preview link</a> |
| <a href="#">design_file</a>        | <a href="#">Binary</a>                       | <a href="#">File binary</a>                                                                              | If <a href="#">storage_mode=small</a> ; <a href="#">binary data</a>   |
| <a href="#">file_size</a>          | <a href="#">Integer</a>                      | <a href="#">File size</a>                                                                                | <a href="#">In bytes</a> ; <a href="#">computed</a>                   |
| <a href="#">route_id</a>           | <a href="#">Many2one → design.file.route</a> | <a href="#">Route (deprecated)</a>                                                                       | <a href="#">Legacy field</a> ; may be NULL                            |
| <a href="#">state</a>              | <a href="#">Selection</a>                    | <a href="#">draft</a> / <a href="#">proof sent</a> / <a href="#">approved</a> / <a href="#">rejected</a> | <a href="#">File approval state</a> ; <a href="#">tracked</a>         |
| <a href="#">created_by_user_id</a> | <a href="#">Many2one → res.users</a>         | <a href="#">Uploader</a>                                                                                 | <a href="#">Who uploaded the file</a>                                 |
| <a href="#">created_at</a>         | <a href="#">Datetime</a>                     | <a href="#">Upload time</a>                                                                              | <a href="#">Auto-stamped</a>                                          |

**Inherits:** [mail.thread](#), [mail.activity.mixin](#) — chatter per file.

**design.file.route**

| Field                           | Type                                        | Purpose                          | Notes                                          |
|---------------------------------|---------------------------------------------|----------------------------------|------------------------------------------------|
| <a href="#">name</a>            | <a href="#">Char</a>                        | <a href="#">Route name</a>       | <a href="#">e.g.</a> , "Custom T-Shirt Design" |
| <a href="#">product_tmpl_id</a> | <a href="#">Many2one → product.template</a> | <a href="#">Product template</a> | <a href="#">Route scoped to a product</a>      |
| <a href="#">active</a>          | <a href="#">Boolean</a>                     | <a href="#">Active flag</a>      | <a href="#">Default: True</a>                  |

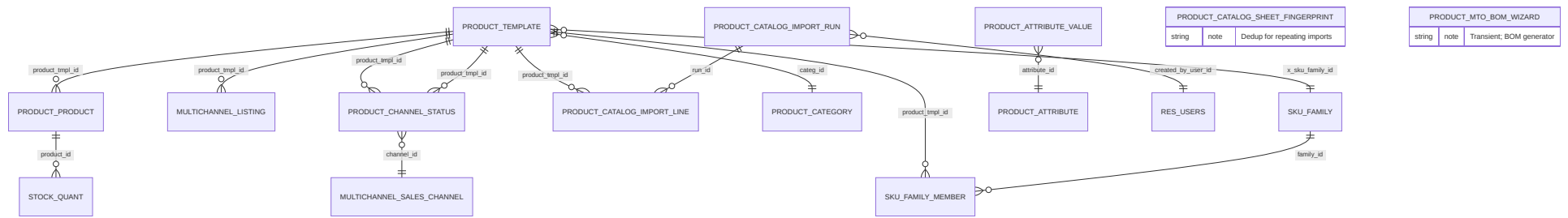
**Note:** This model is largely deprecated (use direct [design.file](#) links instead).

## 6. Domain Cluster 5: Catalog & Product Hub (SKU, Import, Channel Status)

The catalog layer manages product master data, channel-specific status tracking, bulk imports from spreadsheets, and SKU family groupings.

### 6.1 ER DIAGRAM (MERMAID)

## Part 1: System Architecture



**6.2 MODEL REFERENCE TABLE**

| <u>Model</u>                                       | <u>_name</u>                                      | <u>Inherits</u>                                   | <u>Purpose</u>                                                 |
|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------|
| <a href="#">Product Template (core+ext)</a>        | <a href="#">product.template</a>                  | <a href="#">product.template</a> (ext x2)         | <a href="#">Product master + channel SKU + import tracking</a> |
| <a href="#">Product Product (core+ext)</a>         | <a href="#">product.product</a>                   | <a href="#">product.product</a> (inherit)         | <a href="#">Variant + Etsy link</a>                            |
| <a href="#">Product Attribute Value (core+ext)</a> | <a href="#">product.attribute.value</a>           | <a href="#">product.attribute.value</a> (inherit) | <a href="#">Variant attribute value + Etsy mapping</a>         |
| <a href="#">Product Attribute (ext)</a>            | <a href="#">product.attribute</a> (inherit)       | <a href="#">product.attribute</a>                 | <a href="#">Attribute definition + Etsy mapping</a>            |
| <a href="#">Product Channel Status</a>             | <a href="#">product.channel.status</a>            | None                                              | <a href="#">Per-channel publish status + error log</a>         |
| <a href="#">Product Catalog Import Run</a>         | <a href="#">product.catalog.import.run</a>        | None                                              | <a href="#">Batch import execution record</a>                  |
| <a href="#">Product Catalog Import Line</a>        | <a href="#">product.catalog.import.line</a>       | None                                              | <a href="#">Individual row from XLSX import</a>                |
| <a href="#">Product Catalog Sheet Fingerprint</a>  | <a href="#">product.catalog.sheet.fingerprint</a> | None                                              | <a href="#">Dedup tracking for repeating imports</a>           |
| <a href="#">SKU Family</a>                         | <a href="#">mhc.sku.family</a>                    | None                                              | <a href="#">Grouping of related SKUs</a>                       |
| <a href="#">Product MTO BOM Wizard</a>             | <a href="#">product.mto.bom.wizard</a>            | =                                                 | <a href="#">Transient; BOM generation helper</a>               |
| <a href="#">Product Category (ext)</a>             | <a href="#">product.category</a> (inherit)        | <a href="#">product.category</a>                  | <a href="#">Category + pipeline assignment</a>                 |

**6.3 KEY FIELDS BY MODEL****[product.template \(extensions\)](#)**

| <u>Field</u>                               | <u>Type</u>                                           | <u>Purpose</u>                        | <u>Notes</u>                                            |
|--------------------------------------------|-------------------------------------------------------|---------------------------------------|---------------------------------------------------------|
| <a href="#">name</a>                       | <a href="#">Char</a>                                  | <a href="#">Product name</a>          | <a href="#">Canonical product name</a>                  |
| <a href="#">description_sale</a>           | <a href="#">Text</a>                                  | <a href="#">Sales description</a>     | <a href="#">Tier-1 fallback for listing description</a> |
| <a href="#">image_1920</a>                 | <a href="#">Image</a>                                 | <a href="#">Hero image</a>            | <a href="#">Tier-1 fallback for listing image</a>       |
| <a href="#">x_extra_image_ids</a>          | <a href="#">One2many → multichannel.product.image</a> | <a href="#">Gallery images</a>        | <a href="#">Shared across all channels (Wave 2)</a>     |
| <a href="#">x_sku_family_id</a>            | <a href="#">Many2one → mhc.sku.family</a>             | <a href="#">SKU family</a>            | <a href="#">Logical grouping of related variants</a>    |
| <a href="#">categ_id</a>                   | <a href="#">Many2one → product.category</a>           | <a href="#">Category</a>              | <a href="#">Odoo category + pipeline assignment</a>     |
| <a href="#">list_price</a>                 | <a href="#">Float</a>                                 | <a href="#">List price (Monetary)</a> | <a href="#">Standard Odoo selling price</a>             |
| <a href="#">standard_price</a>             | <a href="#">Float</a>                                 | <a href="#">Cost price</a>            | <a href="#">Standard Odoo cost</a>                      |
| <a href="#">weight</a>                     | <a href="#">Float</a>                                 | <a href="#">Weight</a>                | <a href="#">In kg (Odoo standard)</a>                   |
| <a href="#">product_variant_ids</a>        | <a href="#">One2many → product.product</a>            | <a href="#">Variants</a>              | <a href="#">All variants of this product</a>            |
| <a href="#">product_variant_count</a>      | <a href="#">Integer</a>                               | <a href="#">Variant count</a>         | <a href="#">Computed; readonly</a>                      |
| <a href="#">attribute_line_ids</a>         | <a href="#">One2many → product.attribute.line</a>     | <a href="#">Variant attributes</a>    | <a href="#">Attributes that drive variants</a>          |
| <a href="#">x_etsy_category_id</a>         | <a href="#">Char</a>                                  | <a href="#">Etsy category</a>         | <a href="#">Cached Etsy taxonomy node ID (Wave 2)</a>   |
| <a href="#">x_etsy_shipping_profile_id</a> | <a href="#">Char</a>                                  | <a href="#">Shipping profile</a>      | <a href="#">Cached Etsy profile ID (Wave 2)</a>         |

product.product (extensions)

| <u>Field</u>              | <u>Type</u>                                     | <u>Purpose</u>                | <u>Notes</u>                                   |
|---------------------------|-------------------------------------------------|-------------------------------|------------------------------------------------|
| <u>product_tmpl_id</u>    | Many2one → <a href="#">product.template</a>     | <a href="#">Template</a>      | <a href="#">Parent template</a>                |
| <u>default_code</u>       | <a href="#">Char</a>                            | <a href="#">SKU</a>           | <a href="#">Standard Odoo SKU field</a>        |
| <u>x_etsy_listing_ids</u> | One2many → <a href="#">etsy.listing.product</a> | <a href="#">Etsy listings</a> | <a href="#">Listings carrying this variant</a> |

product.channel.status

| <u>Field</u>           | <u>Type</u>                                           | <u>Purpose</u>                                                                                           | <u>Notes</u>                            |
|------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <u>product_tmpl_id</u> | Many2one → <a href="#">product.template</a>           | <a href="#">Product</a>                                                                                  | <a href="#">Required; indexed</a>       |
| <u>channel_id</u>      | Many2one → <a href="#">multichannel.sales.channel</a> | <a href="#">Channel</a>                                                                                  | <a href="#">Required; indexed</a>       |
| <u>publish_state</u>   | <a href="#">Selection</a>                             | <a href="#">draft</a> / <a href="#">published</a> / <a href="#">error</a> / <a href="#">out of stock</a> | <a href="#">Current status</a>          |
| <u>external_ref</u>    | <a href="#">Char</a>                                  | <a href="#">Channel-side ID</a>                                                                          | <a href="#">e.g., Etsy listing_id</a>   |
| <u>last_sync_at</u>    | <a href="#">Datetime</a>                              | <a href="#">Last publish time</a>                                                                        | <a href="#">Read-only</a>               |
| <u>last_sync_error</u> | <a href="#">Text</a>                                  | <a href="#">Publisher error</a>                                                                          | <a href="#">Latest error message</a>    |
| <u>is_active</u>       | <a href="#">Boolean</a>                               | <a href="#">Active on channel</a>                                                                        | <a href="#">Should it be available?</a> |

Constraints:

- [SQL UNIQUE](#): [UNIQUE\(product\\_tmpl\\_id, channel\\_id\)](#) — One status per (product, channel).

product.catalog.import.run

| <u>Field</u>              | <u>Type</u>                                            | <u>Purpose</u>                                                                                         | <u>Notes</u>                                            |
|---------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <u>name</u>               | <a href="#">Char</a>                                   | <a href="#">Import reference</a>                                                                       | <a href="#">e.g., "Catalog 2026-06 Q2"</a>              |
| <u>import_file</u>        | <a href="#">Binary</a>                                 | <a href="#">Uploaded XLSX</a>                                                                          | <a href="#">Raw spreadsheet data</a>                    |
| <u>import_status</u>      | <a href="#">Selection</a>                              | <a href="#">draft</a> / <a href="#">processing</a> / <a href="#">completed</a> / <a href="#">error</a> | <a href="#">Lifecycle</a>                               |
| <u>processed_count</u>    | <a href="#">Integer</a>                                | <a href="#">Processed rows</a>                                                                         | <a href="#">Count of rows parsed</a>                    |
| <u>error_count</u>        | <a href="#">Integer</a>                                | <a href="#">Failed rows</a>                                                                            | <a href="#">Count of validation failures</a>            |
| <u>import_error</u>       | <a href="#">Text</a>                                   | <a href="#">Error details</a>                                                                          | <a href="#">First error message</a>                     |
| <u>created_by_user_id</u> | Many2one → <a href="#">res.users</a>                   | <a href="#">Uploader</a>                                                                               | <a href="#">Who initiated import</a>                    |
| <u>created_at</u>         | <a href="#">Datetime</a>                               | <a href="#">Upload time</a>                                                                            | <a href="#">Auto-stamped</a>                            |
| <u>imported_at</u>        | <a href="#">Datetime</a>                               | <a href="#">Completion time</a>                                                                        | <a href="#">Auto-stamped; NULL if not yet completed</a> |
| <u>import_line_ids</u>    | One2many → <a href="#">product.catalog.import.line</a> | <a href="#">Detail rows</a>                                                                            | <a href="#">Each imported row</a>                       |

product.catalog.import.line

| <u>Field</u>           | <u>Type</u>                                           | <u>Purpose</u>                                                                                      | <u>Notes</u>                            |
|------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------|
| <u>run_id</u>          | Many2one → <a href="#">product.catalog.import.run</a> | Parent run                                                                                          | <a href="#">Required; FK</a>            |
| <u>product_tmpl_id</u> | Many2one → <a href="#">product.template</a>           | Linked product                                                                                      | <a href="#">NULL if not yet matched</a> |
| <u>sku</u>             | Char                                                  | <a href="#">SKU from XLSX</a>                                                                       | <a href="#">Reference for matching</a>  |
| <u>name</u>            | Char                                                  | <a href="#">Product name</a>                                                                        | <a href="#">From XLSX</a>               |
| <u>description</u>     | Text                                                  | <a href="#">Product description</a>                                                                 | <a href="#">From XLSX</a>               |
| <u>cost_price</u>      | Float                                                 | <a href="#">Cost</a>                                                                                | <a href="#">From XLSX</a>               |
| <u>list_price</u>      | Float                                                 | <a href="#">Selling price</a>                                                                       | <a href="#">From XLSX</a>               |
| <u>weight</u>          | Float                                                 | <a href="#">Weight</a>                                                                              | <a href="#">From XLSX (kg)</a>          |
| <u>import_status</u>   | Selection                                             | <a href="#">pending</a> / <a href="#">matched</a> / <a href="#">created</a> / <a href="#">error</a> | <a href="#">Line status</a>             |
| <u>import_error</u>    | Text                                                  | <a href="#">Validation error</a>                                                                    | <a href="#">If import_status=error</a>  |

mhc.sku.family

| <u>Field</u>       | <u>Type</u>                                 | <u>Purpose</u>                     | <u>Notes</u>                               |
|--------------------|---------------------------------------------|------------------------------------|--------------------------------------------|
| <u>name</u>        | Char                                        | <a href="#">Family name</a>        | <a href="#">e.g., "Classic T-Shirt"</a>    |
| <u>code</u>        | Char                                        | <a href="#">Family code</a>        | <a href="#">e.g., "classic-t"; indexed</a> |
| <u>description</u> | Text                                        | <a href="#">Family description</a> | <a href="#">Marketing overview</a>         |
| <u>member_ids</u>  | One2many → <a href="#">product.template</a> | <a href="#">Members</a>            | <a href="#">Products in this family</a>    |
| <u>active</u>      | Boolean                                     | <a href="#">Active flag</a>        | <a href="#">Default: True</a>              |

product.attribute (extensions)

| <u>Field</u>              | <u>Type</u> | <u>Purpose</u>                        | <u>Notes</u>                                      |
|---------------------------|-------------|---------------------------------------|---------------------------------------------------|
| <u>name</u>               | Char        | <a href="#">Attribute name</a>        | <a href="#">e.g., "Size", "Color"</a>             |
| <u>x_etsy_property_id</u> | Char        | <a href="#">Default Etsy property</a> | <a href="#">Etsy property_id mapping (Wave 2)</a> |

product.attribute.value (extensions)

| <u>Field</u>           | <u>Type</u>                                  | <u>Purpose</u>                   | <u>Notes</u>                                     |
|------------------------|----------------------------------------------|----------------------------------|--------------------------------------------------|
| <u>name</u>            | Char                                         | <a href="#">Attribute value</a>  | <a href="#">e.g., "Small", "Red"</a>             |
| <u>attribute_id</u>    | Many2one → <a href="#">product.attribute</a> | <a href="#">Parent attribute</a> | <a href="#">Required</a>                         |
| <u>x_etsy_value_id</u> | Char                                         | <a href="#">Etsy value ID</a>    | <a href="#">Etsy-side value mapping (Wave 2)</a> |

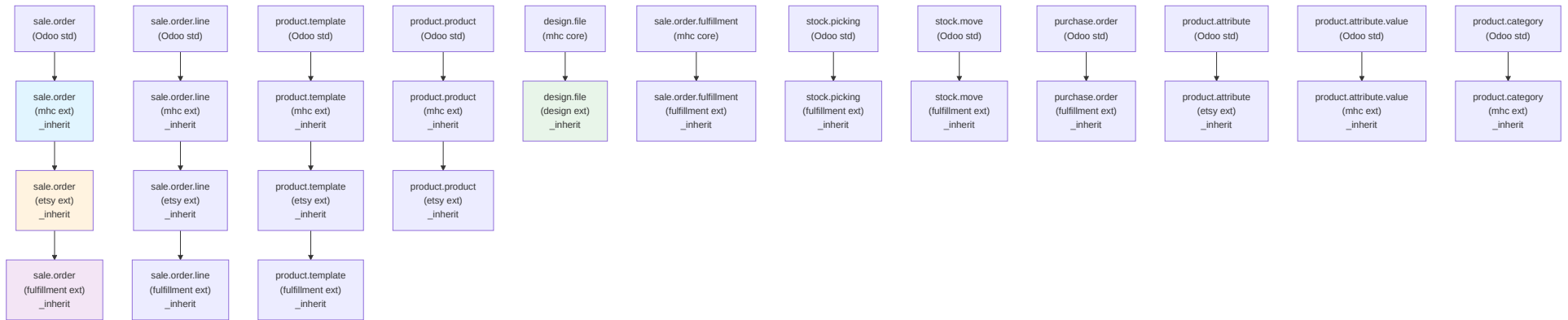
product.category (extensions)

| <u>Field</u>         | <u>Type</u>                                 | <u>Purpose</u>                   | <u>Notes</u>                                                       |
|----------------------|---------------------------------------------|----------------------------------|--------------------------------------------------------------------|
| <u>name</u>          | Char                                        | <a href="#">Category name</a>    | <a href="#">Standard Odoo field</a>                                |
| <u>parent_id</u>     | Many2one → <a href="#">product.category</a> | <a href="#">Parent category</a>  | <a href="#">Standard Odoo hierarchy</a>                            |
| <u>x_pipeline_id</u> | Many2one → <a href="#">order.pipeline</a>   | <a href="#">Default pipeline</a> | <a href="#">Products in this category default to this pipeline</a> |

## **7. Inheritance Hierarchy (Flowchart)**

All custom model extensions and their inheritance chains:

## Part 1: System Architecture



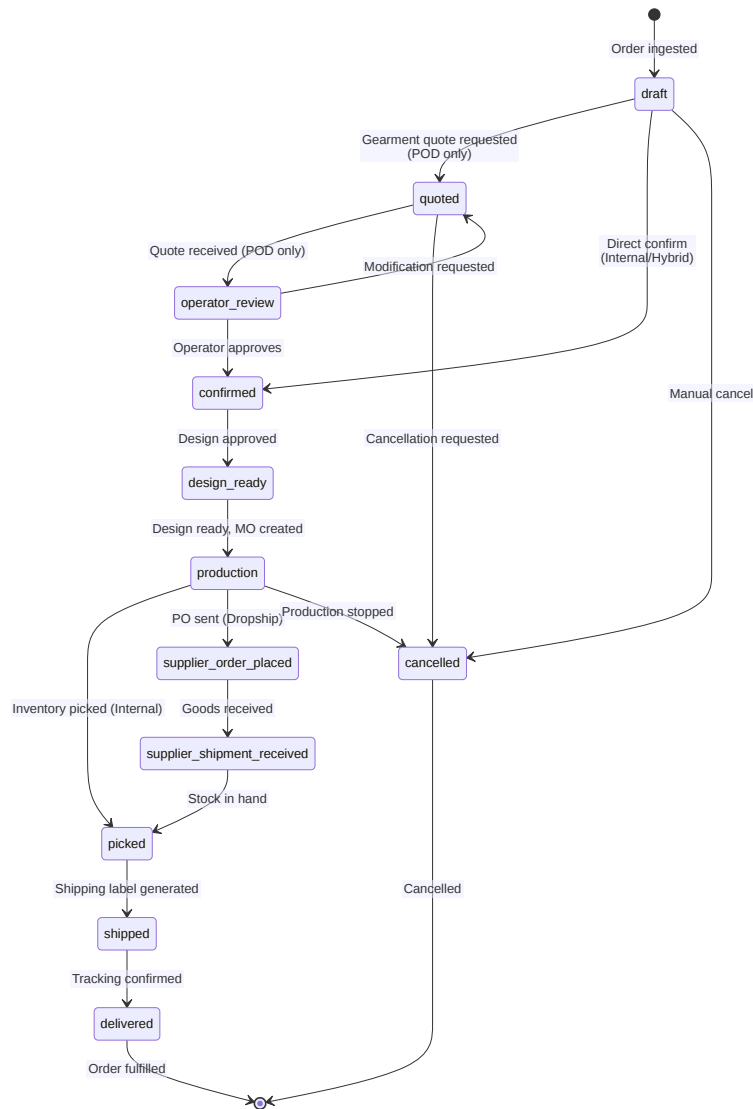


**Color Legend:**

- Blue: `multichannel hub core (mhc)`
- Orange: `etsy integration`
- Purple: `multichannel hub fulfillment`
- Green: `design`

**8. State Machines (Mermaid)****8.1 SALE ORDER PIPELINE STATES**

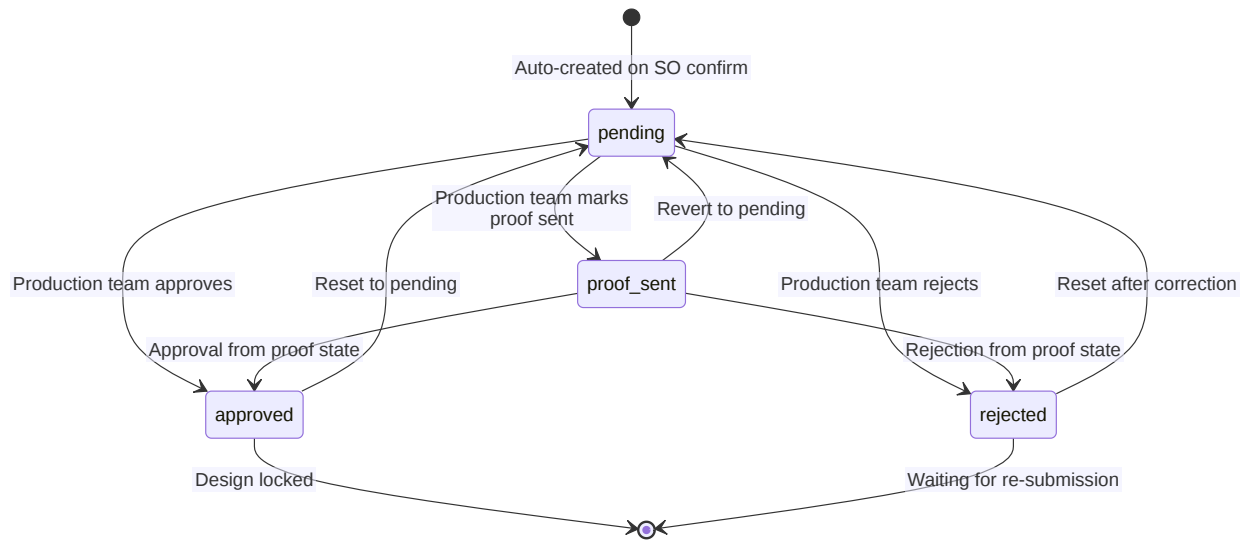
Generic pipeline structure (varies per pipeline instance; 3 example pipelines):



**States by Pipeline:**

| Stage                        | <a href="#">Draft</a> | <a href="#">Quoted</a> | <a href="#">Operator Review</a> | <a href="#">Confirmed</a> | <a href="#">Design Ready</a> | <a href="#">Production</a> | <a href="#">Picked</a> | <a href="#">Shipped</a> | <a href="#">Delivered</a> |
|------------------------------|-----------------------|------------------------|---------------------------------|---------------------------|------------------------------|----------------------------|------------------------|-------------------------|---------------------------|
| <a href="#">VN Internal</a>  | ✓                     | :                      | :                               | ✓                         | ✓                            | ✓                          | ✓                      | ✓                       | ✓                         |
| <a href="#">Gearment POD</a> | ✓                     | ✓                      | ✓                               | ✓                         | ✓                            | ✓                          | ✓                      | ✓                       | ✓                         |
| <a href="#">Hybrid MTO</a>   | ✓                     | :                      | :                               | ✓                         | ✓                            | ✓                          | ✓                      | ✓                       | ✓                         |

## 8.2 DESIGN ORDER STATES

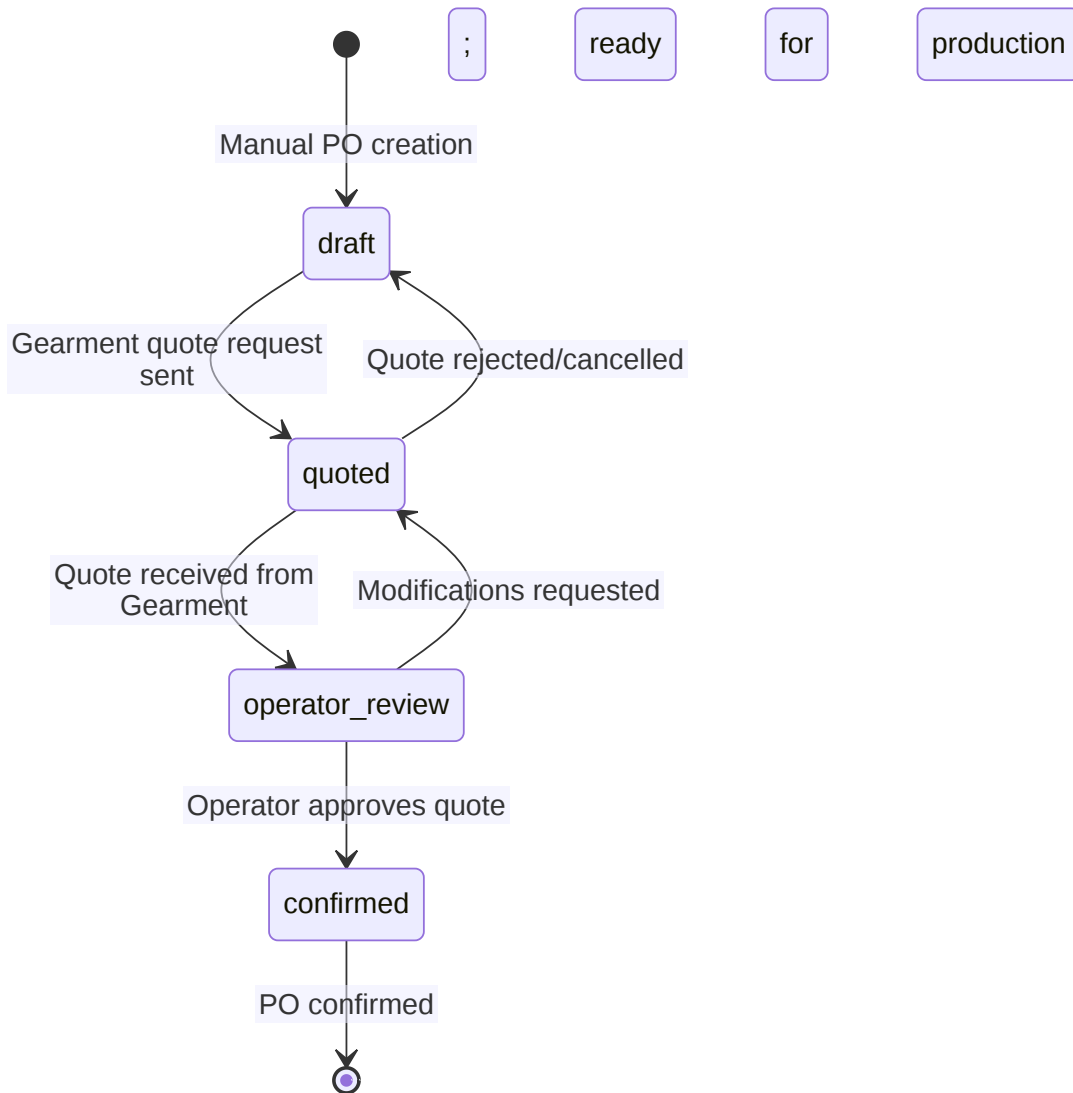


### State Transitions:

- **pending** → **proof\_sent**: Production team marks design as "proof sent to customer"
- **pending/proof\_sent** → **approved**: Design approved; attaches files to MO; advances SO to [design\\_ready\\_stage](#)
- **pending/proof\_sent** → **rejected**: Requires non-empty rejection reason; tracked in chatter
- **approved/rejected** → **pending**: Production team resets (for revisions)

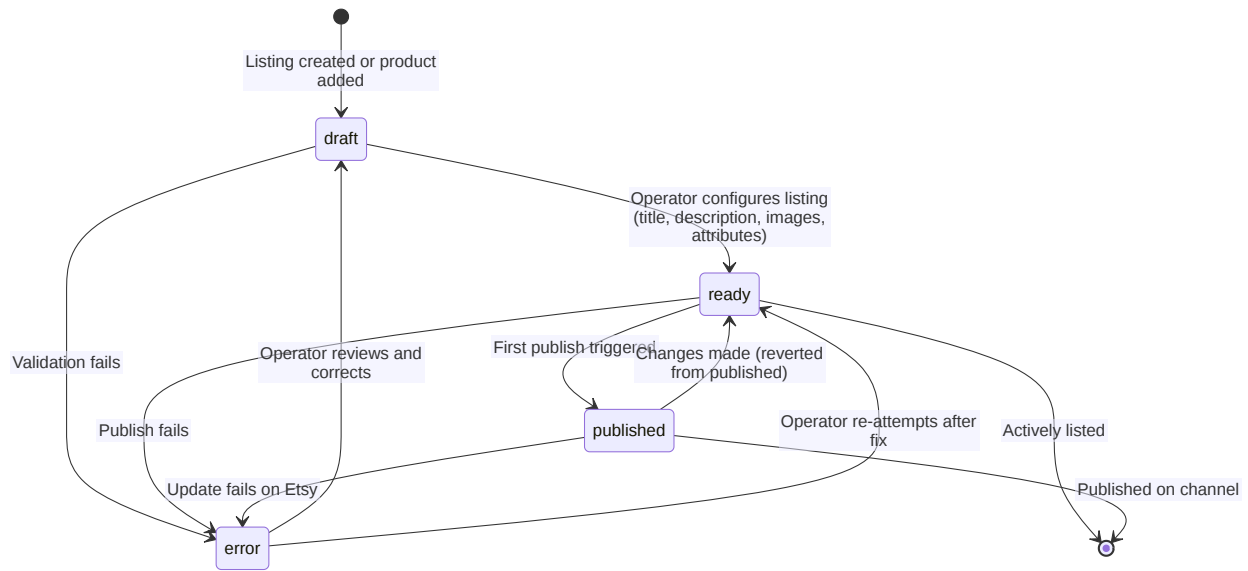
## 8.3 GEARMENT POD ORDER STATES (X GEARMENT OUTBOUND STATE)

POD-specific substate (only active when `x_pipeline_id` = Gearment POD pipeline):

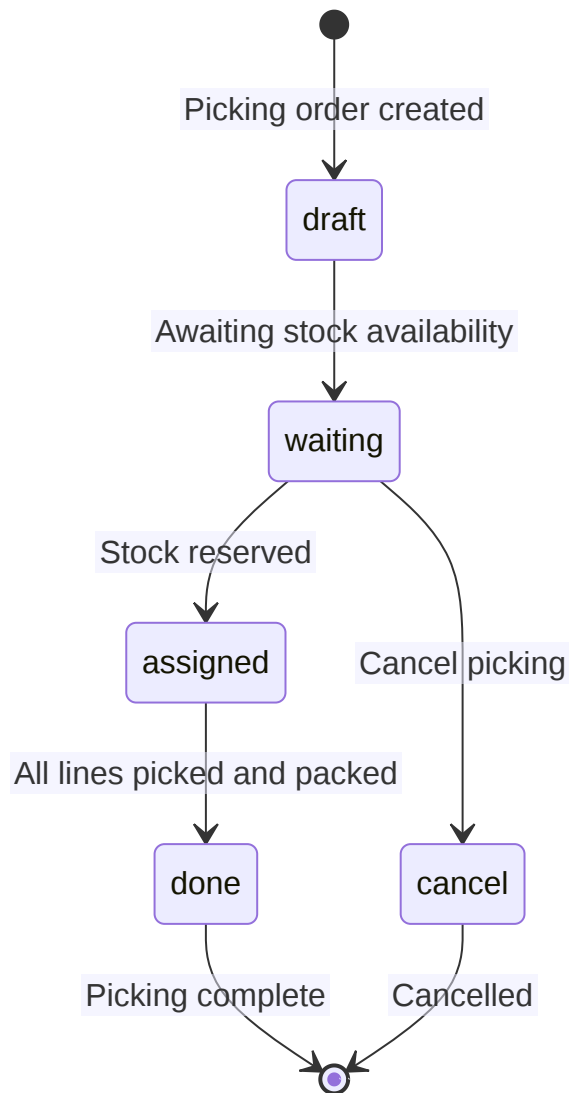


**Relationship to `sale.order.x_pipeline_state_id`:**

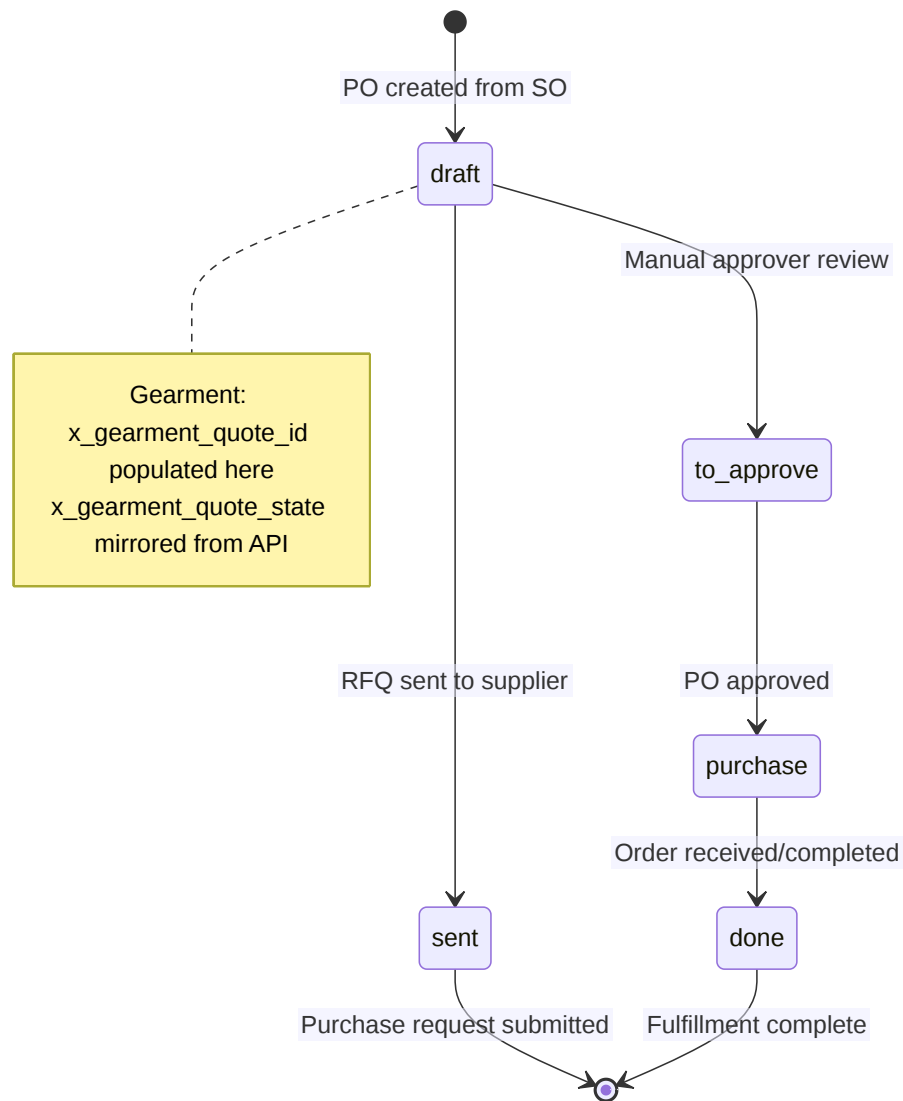
- `x_gearment_outbound_state = draft` → `sale.order state = confirmed` (pending quote)
- `x_gearment_outbound_state = quoted` → `sale.order state = quoted` (awaiting operator review)
- `x_gearment_outbound_state = operator_review` → `sale.order state = operator_review`
- `x_gearment_outbound_state = confirmed` → `sale.order state = confirmed` (production-ready)

**8.4 MULTICHANNEL LISTING STATES****Triggers:**

- [Draft → Ready: All required fields filled \(title, channel, product link\)](#)
- [Ready → Published: Publisher runs successfully](#)
- [Published/Ready → Error: Validation or API failure](#)
- [Error → Draft/Ready: Operator correction](#)

**8.5 STOCK PICKING STATES (EXTENDED).****Extensions by fulfillment module:**

- [x\\_gearment\\_shipping\\_label\\_url](#) populated when label PDF generated
- [x\\_logistics\\_partner\\_id](#) assigned based on carrier
- [x\\_tracking\\_number](#) populated by tracking.import.log

**8.6 PURCHASE ORDER STATES (EXTENDED, GEARMENT/DROPSHIP).****Gearment-Specific Fields:**

- `x_gearment_quote_id` = Gearment quote reference
- `x_gearment_order_id` = Gearment order ID (after confirmation)
- `x_gearment_quote_state` = Synced from Gearment API ( `draft` , `quoted` , `confirmed` , etc.)

## 9. Computed Fields & Related Fields Summary

### COMPUTED (READ-ONLY) FIELDS

| Model                                   | Field                                 | Compute Method                              | Dependencies                                          |
|-----------------------------------------|---------------------------------------|---------------------------------------------|-------------------------------------------------------|
| <a href="#">design.order</a>            | <a href="#">design_files_count</a>    | <a href="#">_compute_design_files_count</a> | <a href="#">design_file_ids</a>                       |
| <a href="#">design.order</a>            | <a href="#">approved_files_count</a>  | <a href="#">_compute_design_files_count</a> | <a href="#">design_file_ids.state</a>                 |
| <a href="#">multichannel.listing</a>    | <a href="#">display_name</a>          | <a href="#">_compute_display_name</a>       | <a href="#">product_tmpl_id, channel_id, shop_ref</a> |
| <a href="#">multichannel.listing</a>    | <a href="#">last_sync_error</a>       | <a href="#">_compute_last_sync_error</a>    | <a href="#">product_channel_status (via join)</a>     |
| <a href="#">etsy.shop</a>               | <a href="#">order_count</a>           | <a href="#">_compute_order_count</a>        | <a href="#">order_ids (one2many)</a>                  |
| <a href="#">etsy.shop</a>               | <a href="#">revenue_total</a>         | <a href="#">_compute_order_count</a>        | <a href="#">order_ids.amount_total</a>                |
| <a href="#">sale.order</a>              | <a href="#">etsy_order_display</a>    | <a href="#">_compute_etsy_order_display</a> | <a href="#">etsy_order_id (if populated)</a>          |
| <a href="#">product.template</a>        | <a href="#">product_variant_count</a> | <a href="#">(std Odoo)</a>                  | <a href="#">product_variant_ids</a>                   |
| <a href="#">product.attribute.value</a> | <a href="#">display_name</a>          | <a href="#">(std Odoo)</a>                  | <a href="#">attribute_id, name</a>                    |

### RELATED FIELDS (STORED, LINKED)

| Model                                | Field                           | Related Source                                    | Purpose                                                      |
|--------------------------------------|---------------------------------|---------------------------------------------------|--------------------------------------------------------------|
| <a href="#">design.order</a>         | <a href="#">partner_id</a>      | <a href="#">sale_order_id.partner_id</a>          | <a href="#">Denormalized for sorting/filtering</a>           |
| <a href="#">design.file</a>          | <a href="#">order_id</a>        | <a href="#">order_line_id.order_id</a>            | <a href="#">Nullable; file can be header- or line-scoped</a> |
| <a href="#">multichannel.listing</a> | <a href="#">extra_image_ids</a> | <a href="#">product_tmpl_id.x_extra_image_ids</a> | <a href="#">Surface product gallery in listing form</a>      |

## 10. SQL Constraints (Raw init() Pattern)

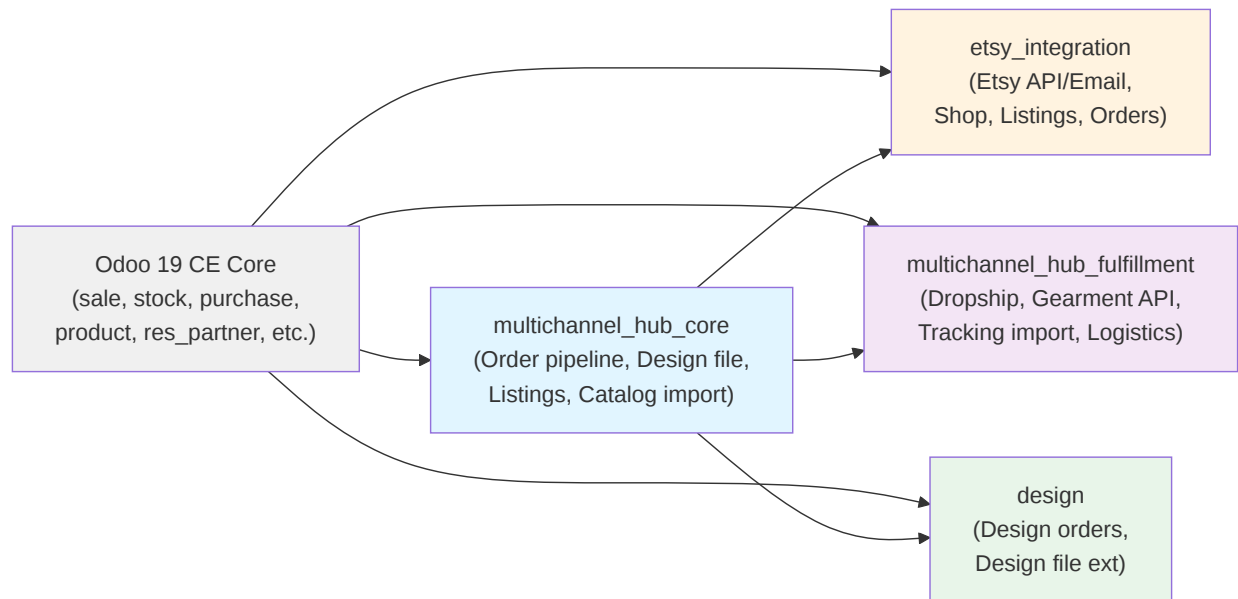
Per project memory [project\\_sql\\_constraints\\_drift.md](#), constraints are deployed both via ORM [sql\\_constraints](#) and raw SQL in [init\(\)](#) (belt-and-braces). Odoo 19 has issues with declarative enforcement.

### DEPLOYED CONSTRAINTS

| Model                                  | Constraint Name                                             | Columns                                                             | Type                   | Purpose                                                  |
|----------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------|------------------------|----------------------------------------------------------|
| <a href="#">etsy.shop</a>              | <a href="#">name_unique</a>                                 | <a href="#">name</a>                                                | <a href="#">UNIQUE</a> | <a href="#">Shop name unique</a>                         |
| <a href="#">design.order</a>           | <a href="#">uniq_design_order_sale_order</a>                | <a href="#">sale_order_id</a>                                       | <a href="#">UNIQUE</a> | <a href="#">One design order per SO</a>                  |
| <a href="#">multichannel.listing</a>   | <a href="#">uniq_multichannel_listing_tmpl_channel_shop</a> | <a href="#">product_tmpl_id, channel_id, COALESCE(shop_ref, "")</a> | <a href="#">UNIQUE</a> | <a href="#">One listing per (product, channel, shop)</a> |
| <a href="#">product.channel.status</a> | <a href="#">uniq_product_channel_status</a>                 | <a href="#">product_tmpl_id, channel_id</a>                         | <a href="#">UNIQUE</a> | <a href="#">One status per (product, channel)</a>        |

All are enforced at PG level via [CREATE UNIQUE INDEX IF NOT EXISTS](#) in model [init\(\)](#) methods.

## 11. Module Dependency Graph



### Dependency Rules:

- All custom modules depend on Odoo 19 CE core
- etsy\_integration, multichannel\_hub\_fulfillment, design all depend on multichannel\_hub\_core
- Only multichannel\_hub\_core extends Odoo models directly; channels extend mhc models
- NO circular dependencies — dependency\_graph is acyclic

## 12. Summary: Key ER Relationship Patterns

### ONE-TO-MANY (PARENT → CHILD)

- Sale Order → Sale Order Fulfillment → Stock Picking
- Order Pipeline → Order Pipeline State
- Product Template → Product Product (variants)
- Etsy Shop → Etsy Shop Attribute Mapping
- Design Order → Design File

### MANY-TO-ONE (CHILD → PARENT)

- Sale Order → Etsy Shop (NULL for non-Etsy)
- Sale Order → Order Pipeline
- Sale Order → Order Pipeline State
- Multichannel Listing → Product Template
- Multichannel Listing → Multichannel Sales Channel

### SELF-JOIN (HIERARCHY)

- Etsy Taxonomy Node → Parent Etsy Taxonomy Node (category tree)
- Product Category → Parent Product Category (standard Odoo)

### MANY-TO-MANY (VIA TRANSIENT/JOIN)

- Product Attribute → Product Attribute Value (via product.attribute.line)
- Pipeline Team → Pipeline (Team operates multiple pipelines)



#### **SOFT LINKS (EXTERNAL REFERENCE)**

- [Sale Order → Etsy Shop via `etsy\_shop\_id` \(NULL if not Etsy\)](#)
  - [Purchase Order → Sale Order via `x\_sale\_order\_id` \(dropship context\)](#)
  - [Multichannel Listing → External Ref \(Etsy listing `id`, stored as Char\)](#)
- 

#### **Document Metadata**

- **Date:** [2026-07-03](#)
- **Status:** [Draft for owner review](#)
- **Odoo Version:** [19.0 CE](#)
- **Module Scope:** [design, multichannel\\_hub\\_core, multichannel\\_hub\\_fulfillment \(partial\)](#)
- **Related SDS:** [02a-data-model.md \(Clusters 1-3\)](#)
- **Code Verification:** [All models, fields, state transitions, and constraints verified against /home/odoo/odoo\\_dev/other\\_projects/odoo19\\_esty/custom\\_addons/ source code on 2026-07-03.](#)

# System Design Spec — 03. External Integrations

**Title:** Multichannel Hub Integrations Reference  
**Date:** 2026-07-03  
**Status:** Draft for Owner Review  
**Version:** 1.0  
**Source of Truth:** This document is the canonical external-surface reference for the multichannel hub. All routes, crons, env vars, and API bindings live here. Code paths are verified against `/home/odoo/odoo dev/other_projects/odoo19_esty/custom_addons/` HEAD.

## Table of Contents

- 1. [Etsy v3 API](#)
- 2. [Gearment v3 API](#)
- 3. [Gmail Ingestion](#)
- 4. [Google Drive Integration](#)
- 5. [GKE Logistics](#)
- 6. [HTTP Controllers Inventory](#)
- 7. [Cron Jobs Inventory](#)
- 8. [Environment Variables & System Parameters](#)

## 1. Etsy v3 API

### 1.1 AUTHENTICATION

**Method:** OAuth 2.0 PKCE (Proof Key for Code Exchange)

**Credential Storage:**

- Client ID, Client Secret, Redirect URIs: `/opt/odoo/secrets/credentials.json`
- Access Token, Refresh Token: Encrypted (Fernet cipher) on `etsy.shop` record via `_set_access_token()` / `_set_refresh_token()` helpers
- Token file schema: JSON with `client_id`, `client_secret`, `redirect_uris` dict (base URL → full callback URL mapping)

**Scope Grant (E1 Approval — 2026-05-12):**

| Scope                        | Permission                           | Status   | Notes                                    |
|------------------------------|--------------------------------------|----------|------------------------------------------|
| <code>transactions_r</code>  | Read orders, receipts                | GRANTED  | Core ingest                              |
| <code>transactions_w</code>  | Update order state                   | GRANTED  | Tracking push, fulfillment marking       |
| <code>listings_r</code>      | Read shop listings                   | GRANTED  | Catalog sync (P-LIST-* slices)           |
| <code>listings_w</code>      | Create/update/delete listings        | GRANTED  | Outbound publish (P-PUB-* slices)        |
| <code>shops_r</code>         | Read shop profile, shipping profiles | GRANTED  | Master data cache                        |
| <code>shops_w</code>         | Update shipping profiles             | GRANTED  | CreateShopShippingProfile API (ESTY-201) |
| <code>email_r</code>         | Read shop email address              | GRANTED  | Shop contact info verification           |
| <code>conversations_r</code> | Read buyer messages                  | REJECTED | E1 excluded; P1-MSG-* blocked            |

**Required Scopes Set (Hard-Coded Validation):**

```
{'transactions_r', 'transactions_w', 'listings_r', 'listings_w', 'shops_r', 'shops_w', 'email_r'}
```

**Forbidden Scopes (Reject on Callback):**

```
{'conversations_r'}
```

If Etsy returns a forbidden scope, the OAuth callback fails with 400 bad request.

#### Re-Authorization Timeline:

- Original pilot shop (JaHandmadeArt, shop\_id=60752333): Authorized before 2026-05-12; lacks `shops_w`
- Post-2026-05-12 shops: All have `shops_w` included
- **Action Required:** Shops needing `shops_w` must re-authorize after 2026-06-13 cutover

## 1.2 API ENDPOINTS USED

**Module:** `etsy_integration`

**Entrypoint:** `custom_addons/etsy_integration/services/etsy_api_client.py`

| Endpoint                                                                           | Method | Purpose                     | Spec Slice       | Status            |
|------------------------------------------------------------------------------------|--------|-----------------------------|------------------|-------------------|
| <code>GET /oauth/authorize</code>                                                  | —      | OAuth PKCE flow redirect    | 005-A            | Shipped           |
| <code>GET /oauth/access_tokens</code>                                              | —      | Token exchange              | 005-A            | Shipped           |
| <code>GET /v3/application/shops/{shop_id}/receipts</code>                          | GET    | Fetch orders (API ingest)   | 005-B            | Shipped           |
| <code>GET /v3/application/shops/{shop_id}/receipts/{receipt_id}</code>             | GET    | Fetch single order detail   | 005-B            | Shipped           |
| <code>POST /v3/application/shops/{shop_id}/receipts/{receipt_id}/tracking</code>   | POST   | Push tracking + carrier     | 003-A            | Shipped           |
| <code>GET /v3/application/shops/{shop_id}/listings</code>                          | GET    | List all shop listings      | 008-*            | Shipped           |
| <code>GET /v3/application/shops/{shop_id}/listings/{listing_id}</code>             | GET    | Listing details + inventory | 008-*            | Shipped           |
| <code>GET /v3/application/shops/{shop_id}/listings/{listing_id}/inventory</code>   | GET    | Inventory tier breakdown    | 008-*            | Shipped           |
| <code>PATCH /v3/application/shops/{shop_id}/listings/{listing_id}/inventory</code> | PATCH  | Update inventory quantity   | 009-010-*        | Planned (Phase 3) |
| <code>POST /v3/application/shops/{shop_id}/listings</code>                         | POST   | Create listing (draft)      | 011-*            | Planned (Phase 3) |
| <code>PUT /v3/application/shops/{shop_id}/listings/{listing_id}</code>             | PUT    | Update listing              | 011-*            | Planned (Phase 3) |
| <code>POST /v3/application/shops/{shop_id}/listings/{listing_id}/images</code>     | POST   | Upload listing image        | 011-*            | Planned (Phase 3) |
| <code>GET /v3/application/shops/{shop_id}/shipping-profiles</code>                 | GET    | Fetch shipping profiles     | 003-B (ESTY-201) | Shipped           |
| <code>POST /v3/application/shops/{shop_id}/shipping-profiles</code>                | POST   | Create shipping profile     | 003-B (ESTY-201) | Shipped           |
| <code>GET /v3/application/etsy-taxonomy</code>                                     | GET    | Category taxonomy cache     | 008-*            | Shipped           |

## 1.3 RATE LIMITING

**Limit:** Unknown (not documented in Etsy v3 OAS; inferred from v2 docs: ~300 req/min per shop)

**Backoff Strategy:**

- Retry on `429 Too Many Requests` or `503 Service Unavailable` with exponential backoff (2s → 4s → 8s)
- Max 3 retries per request
- Implemented in `etsy_api_client.py` `_retry_request()`

#### Cron Timing (Spacing to Avoid Storms):

- Receipts sync: 5-minute interval
- Listing sync: 5-minute interval
- Tracking push: 5-minute interval
- Email fetch (fallback): 10-minute interval

## 1.4 ERROR HANDLING

**Audit Log:** Every request (success or failure) logged to `etsy.api.log` model

| Attribute                     | Type      | Notes                                                                  |
|-------------------------------|-----------|------------------------------------------------------------------------|
| <code>endpoint</code>         | Char      | API path (e.g., <code>GET /v3/application/shops/{id}/receipts</code> ) |
| <code>http_status</code>      | Integer   | HTTP response code (200, 401, 403, 429, 500, 503, etc.)                |
| <code>source</code>           | Selection | <code>cron_sync</code> , <code>manual</code> , etc.                    |
| <code>direction</code>        | Selection | <code>inbound</code> , <code>outbound</code>                           |
| <code>request_headers</code>  | JSON      | Scrubbed headers (auth/x-api-key redacted)                             |
| <code>request_body</code>     | Text      | First 4096 chars of request JSON                                       |
| <code>response_body</code>    | Text      | First 4096 chars of response JSON                                      |
| <code>http_duration_ms</code> | Integer   | Round-trip time                                                        |
| <code>api_error_code</code>   | Char      | Etsy-specific error code (e.g., <code>"1010"</code> for rate limit)    |
| <code>shop_id</code>          | Integer   | Etsy numeric shop ID (FK to <code>etsy.shop</code> )                   |

#### Common Error Codes:

- `401 Unauthorized` → Token expired or revoked; trigger re-authorization flow
- `403 Forbidden` → Shop lacks scope; re-authorize required
- `429 Too Many Requests` → Rate limit; backoff and retry
- `503 Service Unavailable` → Etsy maintenance; backoff and retry

## 1.5 SANDBOX VS. PRODUCTION

#### Configuration:

| Environment        | Base URL                                                                            | API Key Location                                             | Shop ID                  | Active                    |
|--------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------|---------------------------|
| <b>Development</b> | <code>https://openapi.etsy.com/v3/application</code>                                | <code>secrets/credentials.json</code><br>(dev OAuth app)     | Test shop (if any)       | During development only   |
| <b>Staging</b>     | <code>https://openapi.etsy.com/v3/application</code> (prod API, <b>NOT</b> sandbox) | <code>secrets/credentials.json</code><br>(staging OAuth app) | JaHandmadeArt (60752333) | Active; pilot cutover     |
| <b>Production</b>  | <code>https://openapi.etsy.com/v3/application</code>                                | <code>secrets/credentials.json</code><br>(prod OAuth app)    | All 19 shops             | Future (post-Phase 1 E2E) |

**Important:** Etsy does NOT have a sandbox API. All testing against the real API on a real dev shop. Master data (categories, attributes) must be cached to avoid test pollution.

## 2. Gearment v3 API

### 2.1 AUTHENTICATION

#### Credentials:

- API Key: `GEARMENT_API_KEY` (env var, used as HTTP header `x-api-key`)
- API Secret: `GEARMENT_API_SECRET` (env var, used for HMAC signing)
- Base URL: `GEARMENT_API_BASE_URL` (env var, default `https://api.gearment.com`)

#### Header Format:

```
X-Gearment-Client-Key: {GEARMENT_API_KEY}
X-Gearment-Client-Secret: {GEARMENT_API_SECRET}
```

#### Secret Usage (Webhook Verification):

Secrets are used to HMAC-SHA256-sign inbound webhook payloads. See § 2.4 below.

### 2.2 API ENDPOINTS USED

**Module:** `multichannel_hub_fulfillment`

**Entrypoint:** `custom_addons/multichannel_hub_fulfillment/services/gearment_api_client.py`

| Endpoint                                           | Method | Purpose                            | Spec Slice | Status  |
|----------------------------------------------------|--------|------------------------------------|------------|---------|
| <code>POST /api/v3/orders/draft</code>             | POST   | Push SO to Gearment (create quote) | P4-01-B    | Shipped |
| <code>PUT /api/v3/orders/{order_id}/confirm</code> | PUT    | Confirm quote → fulfillment start  | P4-01-B    | Shipped |
| <code>POST /api/v3/orders/{order_id}/cancel</code> | POST   | Cancel quote/order                 | P4-01-C    | Shipped |
| <code>GET /api/v3/orders/{order_id}</code>         | GET    | Fetch order status                 | P4-01-D    | Shipped |

#### Payload Contract (POST `/api/v3/orders/draft`):

```
{
  "reference": "SO-12345",
  "shipping_address": { ... },
  "line_items": [
    {
      "variant_id": "prod-456",
      "quantity": 2,
      "printing_options": [
        {
          "name": "Color",
          "value": "Red"
        }
      ]
    }
  ],
  "currency": "USD",
  "total_price": 99.99
}
```

### 2.3 RATE LIMITING

**Limit:** 100 requests per 10 seconds (hard cap)

#### Backoff Strategy:

- If 429 → wait 10s, retry once
- If still 429 → fail; log and alert operator
- Batch requests (multiple orders) in a single 10s window only if < 100 reqs

#### Cron Scheduling (Controlled via Wizard):

- No automatic cron; quotes are user-triggered via form action button or bulk action
- Manual quote wizard captures order lines and calls Gearment inline (blocking HTTP)

## 2.4 WEBHOOK: HMAC-SHA256 VERIFICATION + REPLAY DEFENSE

**Inbound URL:** `POST /gearment/webhook` (public endpoint, no auth required)

**Signature Scheme:**

```
signing_string = url_path + nonce + timestamp + base64url(body)
X-Connect-Signature = base64url(HMAC-SHA256(GEARMENT_API_SECRET, signing_string))
```

**Required Headers (Validation Order):**

| Header                            | Example                       | Validation                                                    |
|-----------------------------------|-------------------------------|---------------------------------------------------------------|
| <code>X-Connect-Signature</code>  | <code>mUXxmLe4...</code>      | HMAC-SHA256 digest (urlsafe base64)                           |
| <code>X-Connect-Timestamp</code>  | <code>1777735761</code>       | Unix epoch seconds; must be within $\pm 5$ min of server time |
| <code>X-Connect-Nonce</code>      | <code>abc-123-def</code>      | Unique per request; deduplicated within 10-min window         |
| <code>X-Connect-Client-Key</code> | <code>gearment_key_123</code> | Must match <code>GEARMENT_API_KEY</code> env var              |

**Verification Flow (Controller):**

1. Read raw request body (bytes)
2. Extract headers; validate all 4 required headers present
3. Extract `X-Connect-Timestamp`, validate within window ( $\pm 5$  min)
4. Query `gearment.api.log` for prior nonce within 10-min window; if found  $\rightarrow$  reject (401 nonce\_replay)
5. Compute expected signature: `base64url(HMAC-SHA256(GEARMENT_API_SECRET, signing_string))`
6. Compare with `X-Connect-Signature` via `hmac.compare_digest()` (constant-time)
7. On success  $\rightarrow$  200 `{"status": "ok"}`, audit row with `signature_verified=True`
8. On failure  $\rightarrow$  401 `{"error": "unauthorized"}`, audit row with `signature_verified=False` + `verify_failure_reason`

**Replay Protection (P0-18b2c):**

- Dedup window: 10 minutes (600 seconds)
- Nonce search: Query `gearment.api.log` for `(nonce_value, request_timestamp >= cutoff)`
- DB-level backup: Partial unique index on `(nonce_value, request_timestamp)` with condition `signature_verified=True`
- If concurrent request inserts same (nonce, ts) pair  $\rightarrow$  `psycpg2.IntegrityError` caught, return 401 nonce\_replay\_db

**Body Truncation (Security):**

- On signature verification success  $\rightarrow$  store full body (up to 4096 chars)
- On verification failure  $\rightarrow$  truncate body to 256 chars (defense against logging hostile payloads)

## 2.5 WEBHOOK TOPICS HANDLED

**Topic Key:** `body['type']` (fallback: `body['event']` or `body['topic']`)

| Topic                               | Handler                                                      | Purpose                 | Status  |
|-------------------------------------|--------------------------------------------------------------|-------------------------|---------|
| <code>quote.created</code>          | <code>GearmentWebhookDispatcher.on_quote_created()</code>    | Quote ready for review  | Shipped |
| <code>order.confirmed</code>        | <code>GearmentWebhookDispatcher.on_order_confirmed()</code>  | Fulfillment started     | Shipped |
| <code>order.tracking_updated</code> | <code>GearmentWebhookDispatcher.on_tracking_updated()</code> | Tracking info available | Shipped |
| <code>order.shipped</code>          | <code>GearmentWebhookDispatcher.on_order_shipped()</code>    | Order fulfilled         | Shipped |
| <code>order.cancelled</code>        | <code>GearmentWebhookDispatcher.on_order_cancelled()</code>  | Order cancelled         | Shipped |

**Dispatcher Location:** `custom_addons/multichannel_hub_fulfillment/services/gearment_webhook_dispatcher.py`

## 2.6 ERROR HANDLING

**Audit Log:** Every webhook (verified or not) logged to `gearment.api.log`

| Attribute                          | Notes                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>endpoint</code>              | <code>POST /gearment/webhook</code>                                                                                                   |
| <code>http_status</code>           | 200 (success), 401 (sig fail), 503 (db error, still returns 200)                                                                      |
| <code>source</code>                | <code>inbound_webhook</code>                                                                                                          |
| <code>direction</code>             | <code>inbound</code>                                                                                                                  |
| <code>signature_verified</code>    | Boolean; True only if HMAC + replay checks pass                                                                                       |
| <code>verify_failure_reason</code> | Reason if verified=False (e.g., <code>signature_mismatch</code> , <code>timestamp_outside_window</code> , <code>nonce_replay</code> ) |
| <code>nonce_value</code>           | From <code>X-Connect-Nonce</code> header                                                                                              |
| <code>request_timestamp</code>     | From <code>X-Connect-Timestamp</code> header (stored as int for queries)                                                              |
| <code>business_handled</code>      | Boolean; True if dispatcher routed to a handler                                                                                       |
| <code>business_summary</code>      | String; outcome (e.g., <code>'order_not_found'</code> , <code>'state_transition_ok'</code> )                                          |
| <code>sale_order_id</code>         | FK to <code>sale.order</code> if webhook matched a known order                                                                        |

## 3. Gmail Ingestion

### 3.1 PURPOSE

**Fallback Email Parser:** Reads Etsy order confirmation emails from a Gmail mailbox; parses order data using regex patterns; creates `sale.order` records and `design.file` rows.

**Primary Ingest Method:** API sync (Etsy v3 receipts endpoint, § 1.2)

**Fallback Role:** When API ingest is unavailable or rate-limited, Gmail cron reads emails (per ADR-008a)

### 3.2 GMAIL API CREDENTIALS

**Type:** OAuth 2.0 (user-delegated, not service account)

**Configuration Parameters (ir.config\_parameter):**

| Key                                               | Type | Purpose                                                                             | Default                         |
|---------------------------------------------------|------|-------------------------------------------------------------------------------------|---------------------------------|
| <code>etsy_integration.gmail_client_id</code>     | Char | OAuth app client ID (GCP project)                                                   | —                               |
| <code>etsy_integration.gmail_client_secret</code> | Char | OAuth app client secret                                                             | —                               |
| <code>etsy_integration.gmail_label</code>         | Char | Gmail label to poll (e.g., <code>"INBOX"</code> , <code>"[Gmail]/All Mail"</code> ) | <code>"[Gmail]/All Mail"</code> |
| <code>etsy_integration.gmail_refresh_token</code> | Char | Refresh token (persisted after first auth)                                          | —                               |

**Scopes Required:**

```
https://www.googleapis.com/auth/gmail.modify
```

Allows: Read labels, list messages, read message content, modify labels (mark processed).

### 3.3 EMAIL POLLING

**Cron Schedule:**

- Job ID: `ir_cron_fetch_etsy_emails` (record: `etsy_integration.ir_cron_fetch_etsy_emails`)
- Interval: 10 minutes
- Method: `sale.order._cron_fetch_etsy_emails()`
- User: `base.user_root` (system access)

**Flow:**

1. Build Gmail API client using stored refresh token
2. List unread messages in `gmail_label` (e.g., `INBOX`)
3. For each message:
  - Fetch full message body
  - Parse email subject + body via regex patterns (see § 3.4)
  - Extract order ID, buyer, items, totals
  - Create or update `sale.order` + `sale.order.line`
  - Create `design.file` rows for each design attachment (base64 in email)
4. Mark email as read (remove `UNREAD` label)
5. Log summary (# processed, # errors) to `_logger`

**3.4 EMAIL PARSER PATTERNS**

**Service:** `custom_addons/etsy_integration/services/email_parser.py` (ORM-free, pure Python)

**Input Contract:** Raw email HTML/plain text

**Output Contract:** Structured dict with fields:

```
{
  'order_id': str,
  'buyer_name': str,
  'buyer_email': str,
  'order_date': datetime,
  'items': [
    {
      'sku': str,
      'quantity': int,
      'price': decimal,
      'design_attachment_base64': str | None,
    }
  ],
  'total': decimal,
  'shipping_address': str,
}
```

**Key Patterns (Regex):**

- Order ID: Etsy pref `Order #(\d{10})` (Etsy uses 10-digit numeric IDs in order emails)
- Buyer: Extract from `From:` header + email body salutation
- Items: Parse table rows in HTML email (product name, SKU, qty, price)
- Designs: Extract attachment filename/encoding from email headers

**3.5 ERROR HANDLING****Failures:**

- Malformed email (unparseable) → log warning, skip, mark read (prevent re-processing)
- Duplicate order detected → log info, skip duplicate lines, no error
- Design attachment too large (> 10 MB) → log warning, skip attachment, create order without design file
- No matching items found → log error, do not create order (safety guard)



## 4. Google Drive Integration

### 4.1 PURPOSE

- **Design File Upload:** Promote approved design files from Odoo storage to GDrive (ADR-006)
- **Master Data Cache:** Poll GDrive folders for partner/logistics files (Excel, reference sheets)
- **Primary Storage:** Design files in GDrive; links stored in `design.file.gdrive_preview_url`

### 4.2 SERVICE ACCOUNT AUTHENTICATION

**Credentials Type:** Google Service Account (JSON key file)

**Env Var:** `GDRIVE_SERVICE_ACCOUNT_JSON`

**Default Path:** `/app/secrets/gdrive-service-account.json`

**Scopes:**

```
https://www.googleapis.com/auth/drive
```

Allows: Read/write all files on the service account's Shared Drive.

**JSON Schema (from Google Cloud Console):**

```
{
  "type": "service_account",
  "project_id": "...",
  "private_key_id": "...",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...",
  "client_email": "...",
  "client_id": "...",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url": "..."
}
```

### 4.3 DESIGN FILE UPLOAD CRON

**Cron Schedule:**

- Job ID: `cron_design_file_gdrive_sync` (in `multichannel_hub_core/data/ir_cron_data.xml`)
- Interval: 5 minutes
- Method: `design.file._cron_sync_approved_to_gdrive()`
- User: `base.user_root`

**Killswitch:**

- ICP key: `multichannel_hub.design_gdrive_auto_sync_enabled` (default: `True`)
- ICP key: `multichannel_hub.design_file_default_gdrive_folder_id` (no-op if unset)

**Flow:**

1. Query for approved `design.file` records with `storage_mode='small'` (binary upload needed)
2. For each file:
  - Check if already uploaded (via marker in `design.file.gdrive_file_id`)
  - If not uploaded:
    - Call `GdriveUploader.upload_file(blob, name, folder_id)`
    - Store returned `gdrive_file_id` on record
    - Store returned `gdrive_preview_url` on record
    - Update storage mode to `'gdrive'` (logical transition; data now lives on Drive)
3. Log count of uploaded files

#### 4.4 SHOP FOLDER MANAGEMENT

**Method:** `GdriveUploader.ensure_shop_folder(shop)`

- Queries GDrive for existing folder named after `etsy.shop.name`
- If found → return cached `shop.x_gdrive_design_folder_id`
- If not found → create folder, cache folder ID on shop
- Escapes folder name for Drive query safety (single quotes, backslashes)

##### Shared Drive Compatibility:

All Drive API calls include:

```
supportsAllDrives=True
includeItemsFromAllDrives=True
```

Allows access to items on a Shared Drive (not just personal My Drive).

#### 4.5 ERROR HANDLING

##### Failures:

- Credential file missing → log warning, raise `FileNotFoundError`
- Authentication failure (invalid key, revoked access) → log warning, return error dict with `error` field
- Upload timeout or network error → log warning, retry up to 3 times, then return error dict
- Folder creation race condition (concurrent requests) → ignore; use the found folder

##### Soft Failures (Logged, No Alert):

- Design file larger than 50 MB → log warning, skip upload
- GDrive quota exceeded → log error, defer to next cron run

### 5. GKE Logistics

#### 5.1 PURPOSE

Track shipments from Vietnam logistics partner (GKE). Import tracking data (Excel format) into Odoo.

#### 5.2 DATA IMPORT FORMAT

**Source:** Excel file (.xlsx), manually placed in a monitored folder or GDrive

##### Columns (Required):

| Column             | Type | Notes                                                                                                                 |
|--------------------|------|-----------------------------------------------------------------------------------------------------------------------|
| Order Reference    | Char | SO name (e.g., <code>S0-12345</code> ) or Etsy order ID                                                               |
| Carrier            | Char | <code>USPS</code> , <code>UniUni</code> , <code>YunExpress</code> , or other (auto-detect)                            |
| Tracking Number    | Char | Carrier-specific tracking ID                                                                                          |
| Status             | Char | <code>pending</code> , <code>in_transit</code> , <code>delivered</code> , <code>returned</code> , <code>failed</code> |
| Estimated Delivery | Date | Expected arrival date                                                                                                 |
| Current Location   | Char | Free text; parsed for city/country                                                                                    |

#### 5.3 CARRIER DETECTION

**Method:** `custom_addons/multichannel_hub_fulfillment/services/carrier_detector.py`

##### Mapping (Heuristic):

| Tracking Number Pattern     | Carrier    | Notes                        |
|-----------------------------|------------|------------------------------|
| 13 digits, all numeric      | USPS       | USPS Tracking barcode format |
| Starts 1Z + 16 chars        | UPS        | UPS tracking number          |
| Starts 9 + 21 digits        | DHL        | DHL international format     |
| tracking.yunexpress.com URL | YunExpress | URL-based routing            |
| tracking.4px.com            | 4PX        | China logistics partner      |
| Fallback (unmatched)        | OTHER      | Manual review required       |

**Usage:** Model method `stock.picking._detect_carrier_from_tracking(tracking_number)`

## 5.4 IMPORT CRON

### Cron Schedule:

- Job ID: Not yet cron-driven (manual wizard in Phase 2)
- Method: `stock.picking._cron_import_gke_tracking()` (planned for future)

### Flow (Manual Wizard — Current):

1. Operator uploads Excel via wizard
2. Wizard parses file; auto-detects carriers
3. For each row:
  - Find `sale.order` by reference number
  - Create or update `stock.picking` with carrier + tracking number
  - Sync tracking to Etsy API (see § 1.2 POST `/receipts/{id}/tracking`)
4. Display summary (# created, # updated, # errors)

## 6. HTTP Controllers Inventory

**Module Structure:** Controllers live in `custom_addons/{module}/controllers/{endpoint}.py`

### 6.1 OAUTH CONTROLLERS (ETSY\_INTEGRATION)

#### 6.1.1 `/etsy/api/oauth/authorize`

**File:** `custom_addons/etsy_integration/controllers/etsy_oauth.py`

| Property        | Value                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------|
| Route           | <code>/etsy/api/oauth/authorize</code>                                                                                 |
| Methods         | GET                                                                                                                    |
| Auth            | user (login required)                                                                                                  |
| CSRF Protection | Default (enabled)                                                                                                      |
| Purpose         | Initiate PKCE flow; redirect to Etsy authorize endpoint                                                                |
| Parameters      | <code>shop_id</code> (required, query string)                                                                          |
| Query Params    | <code>code_challenge</code> , <code>state</code> (generated), <code>client_id</code> , <code>response_type=code</code> |
| Response        | 302 redirect to <code>https://www.etsy.com/oauth/authorize?...</code>                                                  |
| Side Effects    | Stores PKCE verifier in <code>ir.config_parameter</code> under <code>etsy.oauth.pending.{state}</code>                 |

### Error Handling:

- Missing `shop_id` → 400 `{"error": "Missing shop_id"}`

- Shop not found → 404 `{"error": "Shop not found"}`
- Credentials file missing → 500 `{"error": "OAuth not configured"}`

### 6.1.2 `/etsy/api/oauth/callback`

**File:** `custom_addons/etsy_integration/controllers/etsy_oauth.py`

| Property               | Value                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Route</b>           | <code>/etsy/api/oauth/callback</code>                                                                                         |
| <b>Methods</b>         | <code>GET</code>                                                                                                              |
| <b>Auth</b>            | <code>public</code> (no login required; state validates sender)                                                               |
| <b>CSRF Protection</b> | Disabled (OAuth callback pattern)                                                                                             |
| <b>Purpose</b>         | Handle Etsy redirect after user authorization                                                                                 |
| <b>Parameters</b>      | <code>code</code> , <code>state</code> , <code>error</code> (query string)                                                    |
| <b>Flow</b>            | 1. Validate state → retrieve verifier; 2. Exchange code + verifier → tokens; 3. Scope assertion; 4. Encrypt + store on shop   |
| <b>Response</b>        | 200 with JSON or HTML status; on success: <code>{"status": "authorized"}</code> ; on error: 400/401 with error message        |
| <b>Side Effects</b>    | Creates/updates <code>etsy.shop</code> record; stores encrypted tokens; deletes <code>ir.config_parameter</code> verifier row |

#### Scope Assertion (Failure Conditions):

- Missing any required scope → 401 `{"error": "Missing required scope: X"}`
- `conversations_r` present (forbidden) → 401 `{"error": "conversations_r not allowed"}`
- Token refresh or storage fails → 500 `{"error": "Token storage failed"}`

### 6.1.3 `/etsy/oauth/callback` (Legacy)

**File:** `custom_addons/etsy_integration/controllers/oauth.py` (DEPRECATED, kept for backward compatibility)

| Property      | Value                                                         |
|---------------|---------------------------------------------------------------|
| <b>Route</b>  | <code>/etsy/oauth/callback</code>                             |
| <b>Status</b> | Deprecated; use <code>/etsy/api/oauth/callback</code> instead |
| <b>Note</b>   | May be removed in future release                              |

## 6.2 WEBHOOK CONTROLLER (MULTICHANNEL\_HUB\_FULFILLMENT)

### 6.2.1 `/gearment/webhook`

**File:** `custom_addons/multichannel_hub_fulfillment/controllers/gearment_webhook.py`

| Property                  | Value                                                                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Route                     | <code>/gearment/webhook</code>                                                                                                            |
| Methods                   | <code>POST</code>                                                                                                                         |
| Auth                      | <code>public</code> (HMAC signature validates sender)                                                                                     |
| CSRF Protection           | Disabled (webhook pattern)                                                                                                                |
| Content-Type              | <code>application/json</code>                                                                                                             |
| Purpose                   | Receive + verify + dispatch Gearment webhook notifications                                                                                |
| Body Size Limit           | 1 MB (pre-checked via <code>Content-Length</code> header)                                                                                 |
| Response on Success (200) | <code>{"status": "ok"}</code>                                                                                                             |
| Response on Failure (401) | <code>{"error": "unauthorized"}</code> (truncated body if verify fails)                                                                   |
| Side Effects              | Audit log written to <code>gearment.api.log</code> ; if verified + handler exists: update <code>sale.order</code> state or create records |

**Request Headers (Required):**

```

X-Connect-Signature: <HMAC-SHA256>
X-Connect-Timestamp: <Unix seconds>
X-Connect-Nonce: <UUID or random>
X-Connect-Client-Key: <GEARMENT_API_KEY>

```

**Verification Errors (401 response):**

- `missing_signature_header`, `missing_timestamp_header`, `missing_nonce_header`, `missing_client_key_header`
- `client_key_mismatch` (doesn't match env var)
- `timestamp_invalid` (not a valid integer)
- `timestamp_outside_window` (too old or too far in future)
- `nonce_replay` (duplicate within 10-min window)
- `signature_mismatch` (HMAC doesn't verify)

**Body Truncation:**

- Verified payload: stored full (up to 4096 chars)
- Unverified payload: truncated to 256 chars (security)

## 7. Cron Jobs Inventory

**Framework:** All crons use Odoo's `ir.cron` model; jobs are Python code snippets, not external processes.

### 7.1 ETSY INTEGRATION CRONS

**File:** `custom_addons/etsy_integration/data/ir_cron_data.xml`

## 7.1.1 Email Fetching (Fallback)

| Field    | Value                                                           |
|----------|-----------------------------------------------------------------|
| ID       | <code>ir_cron_fetch_etsy_emails</code>                          |
| Name     | <code>Etsy: Fetch Order Emails</code>                           |
| Model    | <code>sale.order</code>                                         |
| Method   | <code>_cron_fetch_etsy_emails()</code>                          |
| Interval | 10 minutes                                                      |
| Active   | False (optional; email is fallback only; API takes priority)    |
| Purpose  | Poll Gmail for order confirmation emails; parse + create orders |

## 7.1.2 API Receipts Sync

| Field    | Value                                                     |
|----------|-----------------------------------------------------------|
| ID       | <code>cron_etsy_order_sync</code>                         |
| Name     | <code>Etsy: API Receipts Sync</code>                      |
| Model    | <code>etsy.shop</code>                                    |
| Method   | <code>_cron_sync_orders()</code>                          |
| Interval | 5 minutes                                                 |
| Active   | True                                                      |
| Purpose  | Fetch new orders from Etsy API; create sale.order records |
| User     | <code>base.user_root</code>                               |

## 7.1.3 Listing Sync

| Field    | Value                                                    |
|----------|----------------------------------------------------------|
| ID       | <code>cron_etsy_listing_sync</code>                      |
| Name     | <code>Etsy: Listing Metadata Sync</code>                 |
| Model    | <code>etsy.listing</code>                                |
| Method   | <code>_cron_sync_listings()</code>                       |
| Interval | 5 minutes                                                |
| Active   | True                                                     |
| Purpose  | Fetch listing metadata from Etsy; update inventory cache |
| Note     | Part of Spec 008 (listing inventory tracking)            |

## 7.1.4 Listing Variant Inventory Sync

| Field    | Value                                                    |
|----------|----------------------------------------------------------|
| ID       | <code>cron_etsy_listing_variant_sync</code>              |
| Name     | Etsy: Listing Variant Inventory Sync                     |
| Model    | <code>etsy.listing_product</code>                        |
| Method   | <code>_cron_sync_variants()</code>                       |
| Interval | 5 minutes                                                |
| Active   | True                                                     |
| Purpose  | Sync variant-level inventory (per-size, per-color, etc.) |

## 7.1.5 Tracking Push

| Field    | Value                                                              |
|----------|--------------------------------------------------------------------|
| ID       | <code>ir_cron_etsy_tracking_push</code>                            |
| Name     | Etsy: Push Tracking to Etsy                                        |
| Model    | <code>sale.order</code>                                            |
| Method   | <code>_cron_push_tracking()</code>                                 |
| Interval | 5 minutes                                                          |
| Active   | True                                                               |
| Purpose  | Push fulfillment + tracking data back to Etsy (when order shipped) |
| User     | <code>base.user_root</code>                                        |

## 7.1.6 API Log Retention Sweep

| Field    | Value                                               |
|----------|-----------------------------------------------------|
| ID       | <code>cron_etsy_api_log_cleanup</code>              |
| Name     | Etsy: API Log Retention Sweep                       |
| Model    | <code>etsy.api.log</code>                           |
| Method   | <code>_cron_cleanup_old_logs()</code>               |
| Interval | 1 day                                               |
| Active   | True                                                |
| Purpose  | Delete audit logs older than 30 days (configurable) |
| User     | <code>base.user_root</code>                         |

## 7.1.7 Image Download

| Field    | Value                                                                 |
|----------|-----------------------------------------------------------------------|
| ID       | <code>ir_cron_download_pending_etsy_images</code>                     |
| Name     | Etsy: Download Pending Product Images                                 |
| Model    | <code>product.template</code>                                         |
| Method   | <code>_cron_download_etsy_images()</code>                             |
| Interval | 30 minutes                                                            |
| Active   | True                                                                  |
| Purpose  | Download product images from Etsy listings; attach to product records |
| User     | <code>base.user_root</code>                                           |

## 7.1.8 Message Dedupe Retention

| Field    | Value                                                                |
|----------|----------------------------------------------------------------------|
| ID       | <code>cron_etsy_message_dedupe_retention</code>                      |
| Name     | Etsy: Message Dedupe Retention Sweep                                 |
| Model    | <code>etsy.message.dedupe</code>                                     |
| Method   | <code>_cron_dedupe_retention()</code>                                |
| Interval | 1 day                                                                |
| Active   | True                                                                 |
| Purpose  | Purge old message dedup markers (older than 7 days) to free DB space |

## 7.1.9 Taxonomy Sync

| Field    | Value                                                      |
|----------|------------------------------------------------------------|
| ID       | <code>cron_etsy_taxonomy_sync</code>                       |
| Name     | Etsy: Taxonomy Cache Sync (P-LIST-CATEGORY)                |
| Model    | <code>etsy.shop</code>                                     |
| Method   | <code>_cron_sync_taxonomy()</code>                         |
| Interval | 7 days                                                     |
| Active   | True                                                       |
| Purpose  | Refresh Etsy category taxonomy (used for listing creation) |
| Note     | Runs weekly to keep category IDs current                   |



## 7.1.10 Shipping Profile Sync

| Field    | Value                                                            |
|----------|------------------------------------------------------------------|
| ID       | <code>cron_etsy_shipping_profile_sync</code>                     |
| Name     | <code>Etsy: Shipping Profile Cache Sync (P-LIST-SHIPPING)</code> |
| Model    | <code>etsy.shop</code>                                           |
| Method   | <code>_cron_sync_shipping_profiles()</code>                      |
| Interval | 1 day                                                            |
| Active   | True                                                             |
| Purpose  | Refresh cached shipping profiles (used for listing creation)     |

## 7.2 MULTICHANNEL HUB CORE CRONS

File: `custom_addons/multichannel_hub_core/data/ir_cron_data.xml`

## 7.2.1 Overdue Approval Recompute

| Field    | Value                                                                                     |
|----------|-------------------------------------------------------------------------------------------|
| ID       | <code>cron_recompute_overdue_approval</code>                                              |
| Name     | <code>Multichannel Hub: recompute is_overdue_approval</code>                              |
| Model    | <code>sale.order</code>                                                                   |
| Method   | <code>_cron_recompute_overdue_approval()</code>                                           |
| Interval | 1 day                                                                                     |
| Active   | True                                                                                      |
| Purpose  | Recalculate <code>is_overdue_approval</code> flag (design approval deadline check) daily  |
| User     | <code>base.user_root</code>                                                               |
| Note     | Ensures dashboard marker updates even if no DB writes triggered <code>@api.depends</code> |

## 7.2.2 Duplicate Buyer Recompute

| Field    | Value                                                                                                 |
|----------|-------------------------------------------------------------------------------------------------------|
| ID       | <code>cron_recompute_duplicate_buyer</code>                                                           |
| Name     | <code>Multichannel Hub: recompute is_duplicate_buyer</code>                                           |
| Model    | <code>sale.order</code>                                                                               |
| Method   | <code>_cron_recompute_duplicate_buyer()</code>                                                        |
| Interval | 1 day                                                                                                 |
| Active   | True                                                                                                  |
| Purpose  | Recalculate <code>is_duplicate_buyer</code> flag (retroactively flag older orders from repeat buyers) |
| User     | <code>base.user_root</code>                                                                           |

### 7.2.3 Design File GDrive Sync

| Field           | Value                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| ID              | <code>cron_design_file_gdrive_sync</code>                                                                        |
| Name            | Multichannel Hub: sync approved design files to GDrive                                                           |
| Model           | <code>design.file</code>                                                                                         |
| Method          | <code>_cron_sync_approved_to_gdrive()</code>                                                                     |
| Interval        | 5 minutes                                                                                                        |
| Active          | True                                                                                                             |
| Purpose         | Promote approved <code>design.file</code> rows (storage_mode='small') to GDrive; update storage mode to 'gdrive' |
| User            | <code>base.user_root</code>                                                                                      |
| Killswitch      | ICP <code>multichannel_hub.design_gdrive_auto_sync_enabled</code> (default True)                                 |
| No-Op Condition | ICP <code>multichannel_hub.design_file_default_gdrive_folder_id</code> is not set                                |

## 7.3 GEARMENT CRONS

File: `custom_addons/multichannel_hub_fulfillment/data/ir_cron_gearment_api_log_retention.xml`

### 7.3.1 API Log Retention Sweep

| Field    | Value                                         |
|----------|-----------------------------------------------|
| ID       | <code>ir_cron_gearment_api_log_cleanup</code> |
| Name     | Gearment API Log: cleanup old rows            |
| Model    | <code>gearment.api.log</code>                 |
| Method   | <code>_cron_cleanup_old_logs()</code>         |
| Interval | 1 day                                         |
| Active   | True                                          |
| Purpose  | Delete webhook audit logs older than 30 days  |
| User     | (not explicitly set in XML; default applies)  |

## 7.4 ORPHANED CRONS (NO IR.CRON RECORD)

Dead Code Candidate:

| Method                                                       | Location                                                      | Status         | Note                                                                                     |
|--------------------------------------------------------------|---------------------------------------------------------------|----------------|------------------------------------------------------------------------------------------|
| <code>image_downloader.cron_download_pending_images()</code> | <code>multichannel_hub_core/models/image_downloader.py</code> | No cron record | Method defined but not wired to <code>ir.cron</code> ; orphaned (migration in progress?) |

## 8. Environment Variables & System Parameters

### 8.1 ENVIRONMENT VARIABLES (READ AT RUNTIME)

**Source:** `.env` file or container environment

**Access Pattern:** `os.environ.get('VAR_NAME', 'default')`

#### 8.1.1 Gearment Integration

| Variable                           | Type   | Purpose                                                               | Required? | Default                               |
|------------------------------------|--------|-----------------------------------------------------------------------|-----------|---------------------------------------|
| <code>GEARMENT_API_KEY</code>      | String | Client key for Gearment API requests (header <code>x-api-key</code> ) | Yes       | —                                     |
| <code>GEARMENT_API_SECRET</code>   | String | Secret for HMAC-SHA256 webhook signature verification                 | Yes       | —                                     |
| <code>GEARMENT_API_BASE_URL</code> | String | Base URL for Gearment API                                             | No        | <code>https://api.gearment.com</code> |

#### 8.1.2 Google Drive Integration

| Variable                                 | Type             | Purpose                                      | Required? | Default                                               |
|------------------------------------------|------------------|----------------------------------------------|-----------|-------------------------------------------------------|
| <code>GDRIVE_SERVICE_ACCOUNT_JSON</code> | String<br>(Path) | File path to GDrive service account JSON key | No        | <code>/app/secrets/gdrive-service-account.json</code> |

#### 8.1.3 Testing / Feature Flags

| Variable                                           | Type                                          | Purpose                                          | Required? | Notes                                                                               |
|----------------------------------------------------|-----------------------------------------------|--------------------------------------------------|-----------|-------------------------------------------------------------------------------------|
| <code>MULTICHANNEL_HUB_FULFILLMENT_LIVE_API</code> | Bool ( <code>'1'</code> or <code>'0'</code> ) | Skip mocked Gearment API in tests; call real API | No        | Default: <code>'0'</code> (use mocks); set to <code>'1'</code> for live E2E testing |

### 8.2 SYSTEM PARAMETERS (IR.CONFIG\_PARAMETER)

**Scope:** Odoo database; stored in `ir.config_parameter` model

**Access Pattern:** `self.env['ir.config_parameter'].sudo().get_param('key', default='...')`

## 8.2.1 Etsy Integration

| Key                                                   | Type                                                    | Purpose                                        | Default                                              | Scope                               |
|-------------------------------------------------------|---------------------------------------------------------|------------------------------------------------|------------------------------------------------------|-------------------------------------|
| <code>etsy.oauth.credentials_path</code>              | Char                                                    | File path to OAuth app credentials JSON        | <code>/opt/odoo/secrets/credentials.json</code>      | All shops                           |
| <code>etsy_integration.gmail_client_id</code>         | Char                                                    | Gmail OAuth app client ID                      | —                                                    | Gmail auth (fall-back email ingest) |
| <code>etsy_integration.gmail_client_secret</code>     | Char                                                    | Gmail OAuth app client secret                  | —                                                    | Gmail auth                          |
| <code>etsy_integration.gmail_label</code>             | Char                                                    | Gmail label to poll (e.g., "[Gmail]/All Mail") | <code>"[Gmail]/All Mail"</code>                      | Email cron                          |
| <code>etsy_integration.gmail_refresh_token</code>     | Char                                                    | Gmail refresh token (persisted after OAuth)    | —                                                    | Email cron (fall-back)              |
| <code>etsy_integration.etsy_api_version</code>        | Char                                                    | Etsy API version                               | <code>v3</code>                                      | API client                          |
| <code>etsy_integration.etsy_api_base_url</code>       | Char                                                    | Base URL for Etsy API                          | <code>https://openapi.etsy.com/v3/application</code> | API client                          |
| <code>etsy_integration.etsy_log_retention_days</code> | Integer                                                 | Days to keep API audit logs                    | <code>30</code>                                      | Log cleanup cron                    |
| <code>design.auto_create_on_confirm</code>            | Char<br>( <code>'True'</code> or <code>'False'</code> ) | Auto-create design order when SO confirmed     | <code>'True'</code>                                  | Design module                       |

## 8.2.2 Multichannel Hub Core

| Key                                                                | Type                                | Purpose                                                                | Default                                            | Sc                                               |
|--------------------------------------------------------------------|-------------------------------------|------------------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------|
| <code>multichannel_hub.default_pipeline_code</code>                | Char                                | Default pipeline (when SO has no explicit pipeline)                    | <code>order_pipeline_vn_internal_production</code> | Or<br>st<br>ma<br>ch                             |
| <code>multichannel_hub.design_gdrive_auto_sync_enabled</code>      | Char<br>( 'True'<br>or<br>'False' ) | Enable auto-sync of approved design files to GDrive                    | 'True'                                             | De<br>file<br>Gl<br>cri                          |
| <code>multichannel_hub.design_file_default_gdrive_folder_id</code> | Char                                | GDrive folder ID for design file uploads                               | —                                                  | De<br>file<br>Gl<br>cri<br>(n<br>if<br>un        |
| <code>multichannel_hub.large_file_threshold_bytes</code>           | Integer                             | Max size (bytes) for binary file storage before rejection              | <code>10485760</code> (10 MB)                      | De<br>file<br>up                                 |
| <code>web.base.url</code>                                          | Char                                | Odoo server base URL (used by OAuth callbacks to resolve redirect URI) | —                                                  | Or<br>co<br>trc<br>(re<br>re<br>Ur<br>pic<br>ing |

## 8.2.3 Product Catalog / Inventory

| Key                                                            | Type | Purpose                                                                 | Default                     |
|----------------------------------------------------------------|------|-------------------------------------------------------------------------|-----------------------------|
| <code>multichannel_hub.product_catalog_gdrive_folder_id</code> | Char | GDrive folder ID for product Excel imports                              | —                           |
| <code>multichannel_hub.product_catalog_source_file_name</code> | Char | Filename of product master Excel (e.g., "Product_Catalog_Master.xlsx" ) | "Product_Catalog_Master.xls |

## Document Metadata

| Field            | Value                                                                                                                           |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Last Updated     | 2026-07-03                                                                                                                      |
| Verified Against | Git HEAD of <code>feature/006-master-plan-coding</code> (all 4 modules merged to main 2026-07-03)                               |
| Scope            | External-surface reference; routes, crons, env vars, API bindings                                                               |
| Related Docs     | <code>01-architecture.md</code> (data model), <code>02-flows.md</code> (business processes), <code>04-security.md</code> (ACLs) |
| Approval         | Draft for Owner Review                                                                                                          |
| Status           | Complete; all 8 sections populated with code-verified details                                                                   |

## Appendix A. Quick Reference — All Routes

```
GET /etsy/api/oauth/authorize    → Initiate PKCE flow
GET /etsy/api/oauth/callback    → OAuth callback (token exchange)
GET /etsy/oauth/callback        → Legacy (deprecated)
POST /gearment/webhook          → Gearment webhook (HMAC-verified)
```

## Appendix B. Quick Reference — All Crons

```
Etsy:
  ir_cron_fetch_etsy_emails      (10 min, fallback email ingest)
  cron_etsy_order_sync          (5 min, API receipts)
  cron_etsy_listing_sync        (5 min, listing metadata)
  cron_etsy_listing_variant_sync (5 min, variant inventory)
  ir_cron_etsy_tracking_push     (5 min, push tracking)
  cron_etsy_api_log_cleanup      (1 day, retention sweep)
  ir_cron_download_pending_etsy_images (30 min, image download)
  cron_etsy_message_dedupe_retention (1 day, dedupe cleanup)
  cron_etsy_taxonomy_sync       (7 days, category cache)
  cron_etsy_shipping_profile_sync (1 day, shipping cache)

Multichannel Hub Core:
  cron_recompute_overdue_approval (1 day, flag recompute)
  cron_recompute_duplicate_buyer   (1 day, flag recompute)
  cron_design_file_gdrive_sync     (5 min, GDrive upload)

Gearment:
  ir_cron_gearment_api_log_cleanup (1 day, retention sweep)
```

## Appendix C. Quick Reference — All Env Vars

```
GEARMENT_API_KEY      (required, webhook signing)
GEARMENT_API_SECRET   (required, webhook signing)
GEARMENT_API_BASE_URL (optional, default: https://api.gearment.com)
GDRIVE_SERVICE_ACCOUNT_JSON (optional, default: /app/secrets/gdrive-service-account.json)
MULTICHANNEL_HUB_FULFILLMENT_LIVE_API (optional, testing flag)
```

**Appendix D. Quick Reference — Key System Parameters**

|                                                       |                                 |
|-------------------------------------------------------|---------------------------------|
| etsy.oauth.credentials_path                           | (OAuth credentials file path)   |
| etsy_integration.gmail_client_id                      | (Gmail OAuth)                   |
| etsy_integration.gmail_client_secret                  | (Gmail OAuth)                   |
| etsy_integration.gmail_label                          | (Gmail label to poll)           |
| etsy_integration.gmail_refresh_token                  | (Gmail token, auto-set)         |
| design.auto_create_on_confirm                         | (auto-create design order)      |
| multichannel_hub.design_gdrive_auto_sync_enabled      | (GDrive sync killswitch)        |
| multichannel_hub.design_file_default_gdrive_folder_id | (GDrive folder)                 |
| multichannel_hub.default_pipeline_code                | (default order pipeline)        |
| web.base.url                                          | (OAuth redirect URI resolution) |

**title: “SDS 04a: Sequence Flows (Inbound Channels)” date: “2026-07-03” status: “Draft-for-owner-review” source\_of\_truth: “All flows traced from actual code in custom\_addons/”**

## Sequence Flows — Inbound Channels

This document defines 8 key workflows as mermaid sequence diagrams, traced through actual Odoo code. Each flow includes a brief prose walkthrough, error paths (alt/opt blocks), and participant names matching real classes/models.

All flows are **shipped and E2E verified on staging** as of 2026-07-03, consolidation to `main` complete.

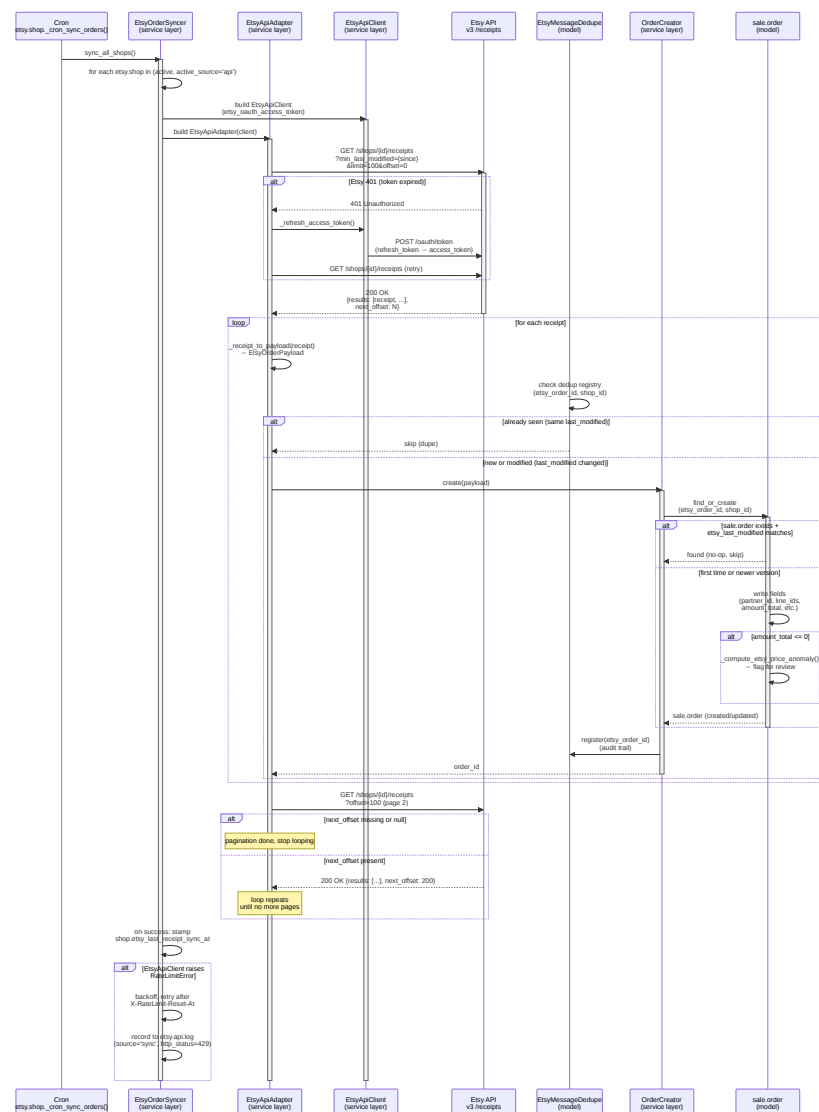
### 1. Etsy Order Ingest via API (Spec 005, P1-11)

**Entry Point:** Cron `etsy.shop._cron_sync_orders()` (5-min interval)

**Implemented:** Phase 0 (P0-16c EtsyOrderSyncer), Phase 1 (P1-11 pilot cutover)

**Status:** Shipped, live on JaHandmadeArt (shop\_id=60752333)

#### SEQUENCE DIAGRAM



#### PROSE WALKTHROUGH

- Orchestration (Cron):** Daily cron iterates all shops with `active_source='api'` and valid OAuth tokens. Instantiates a fresh `EtsyOrderSyncer` per transaction.



2. **API Client Setup:** Syncer builds an `EtsyApiClient` (P0-15) using the shop's `etsy_oauth_access_token`. Token is verified against expiry; if stale, a refresh is queued for the next transaction window.
3. **Adapter Pagination:** `EtsyApiAdapter` lazily fetches pages of receipts from `/shops/{shop_id}/receipts` with `min_last_modified={shop.etsy_last_receipt_sync_at}`. Each page yields up to 100 receipts.
4. **Receipt-to-Payload Conversion:** Each receipt is mapped to a canonical `EtsyOrderPayload` dataclass (carrier for order metadata, address, line items, totals). Money fields (amount, divisor) are normalized to floats.
5. **Deduplication:** Before creating a `sale.order`, the syncer checks `EtsyMessageDedupe` registry (keyed by `etsy_order_id, shop_id`). If the same `etsy_last_modified` timestamp is already recorded, the receipt is skipped (idempotent).
6. **Order Creation/Update:** For new or modified receipts, `OrderCreator.create()` finds or creates a `sale.order` with the matching `etsy_order_id`. Partner, address, and line items are synced from the payload. Timestamps are stamped.
7. **Anomaly Flagging:** If `amount_total <= 0`, the order is flagged with `etsy_price_anomaly=True` for ops review.
8. **Watermark Advance:** After all pages of receipts are consumed, the shop's `etsy_last_receipt_sync_at` is stamped to the latest receipt's `last_modified` timestamp, bounding the next sync.

#### ERROR PATHS

- **401 Unauthorized:** Token expired. Client auto-refreshes via `refresh_token` and retries. If refresh fails (e.g., user revoked consent), the syncer logs a CRITICAL event and stops (operator re-authorizes OAuth).
- **429 Too Many Requests:** Rate limit hit. Syncer respects `X-RateLimit-Reset-At` header and defers remaining shops to the next cron run.
- **Malformed Receipt:** `_receipt_to_payload()` may encounter missing fields (e.g., `transactions=null`). Catches and logs; that receipt is skipped, cron continues.
- **Network/Timeout:** Syncer rolls back the transaction, logs error, and re-queues for the next run.

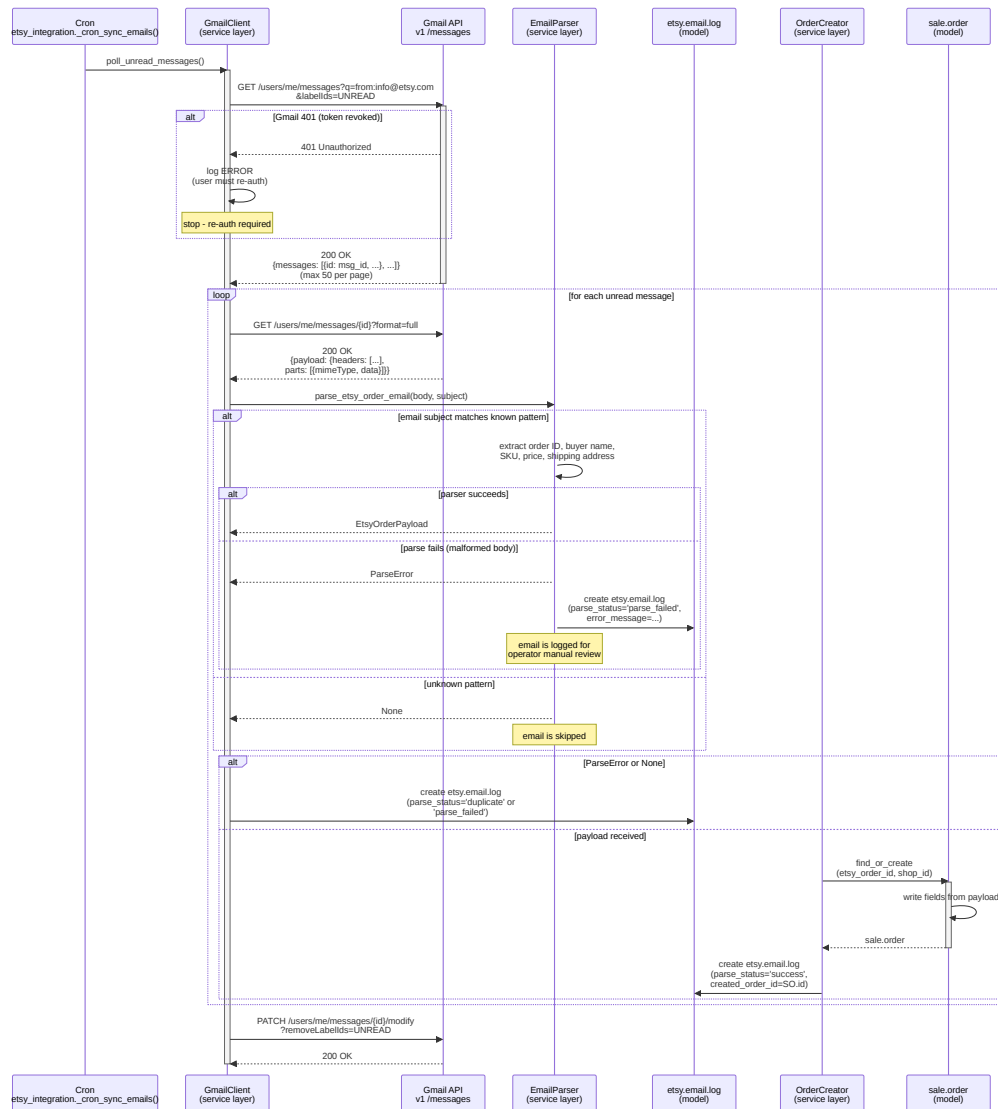
## 2. Etsy Order Ingest via Email Fallback (Spec 002, Phase 0)

**Entry Point:** Cron `etsy_integration._cron_sync_emails()` (10-min interval, `active_source='email'`)

**Implemented:** Phase 0 (P0-14 Gmail client), Phase 1 (optional for shops with email=fallback)

**Status:** Shipped, live on legacy email-path shops; superseded by API for new cutover

## SEQUENCE DIAGRAM



## PROSE WALKTHROUGH

- Gmail Polling:** Cron queries Gmail API for unread emails from `info@etsy.com` (Etsy's order notification sender). Fetches up to 50 message IDs per page.
- Message Fetch:** For each unread message, GmailClient fetches the full message body (including headers and MIME parts).
- Email Parser:** `EmailParser.parse_etsy_order_email()` applies a regex-based state machine to extract:
  - Order ID (Etsy receipt number)
  - Buyer name and email
  - Shipping address
  - Line items (SKU, quantity, price)
  - Order total and shipping cost
- Order Creation:** Successful parses flow to `OrderCreator`, which finds or creates the corresponding `sale.order`. All fields are synced from the email body.

5. **Email Log Audit:** Whether success or failure, an `etsy.email.log` row is created with:

- `parse_status` (success | parse\_failed | duplicate)
- `error_message` (if failed)
- `created_order_id` (if successful)
- Raw email body (for forensic replay)

6. **Mark Read:** After processing, the message is marked as read in Gmail so it is not re-polled.

#### ERROR PATHS

- **401 Unauthorized:** Gmail token revoked. Cron logs error and stops; operator must re-authorize OAuth.
  - **Malformed Email:** EmailParser logs to `etsy.email.log` with `parse_status='parse_failed'`. Email is preserved for manual review.
  - **Duplicate:** If the order ID is already in Odoo, email is logged but skipped (idempotent).
  - **Network Timeout:** Cron transaction rolls back; re-queued for next run.
- 

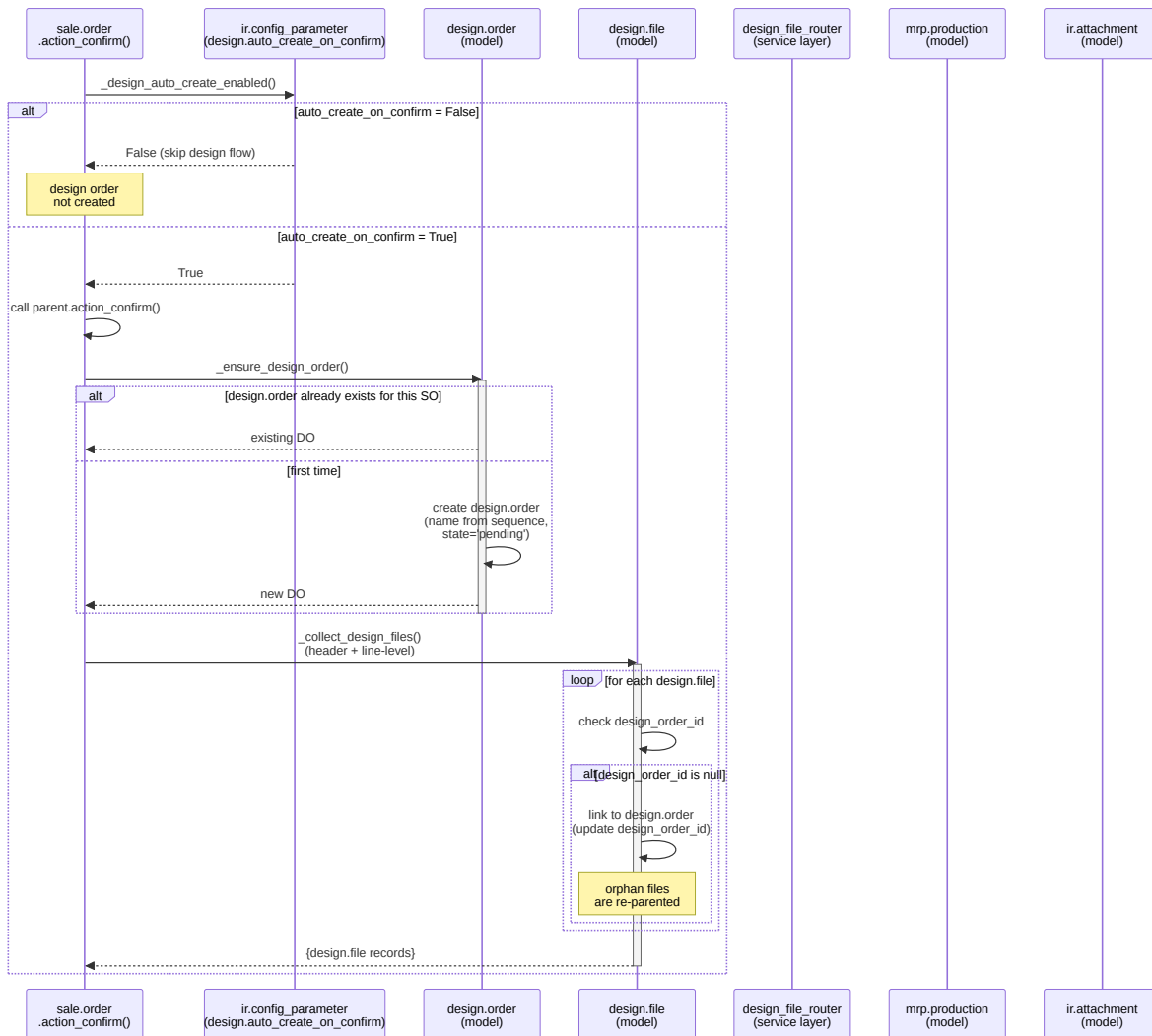
### 3. Design Approval Pipeline (ESTY-244, P1-02a/P1-02b)

**Entry Point:** Sale order confirmation → auto-create `design.order`

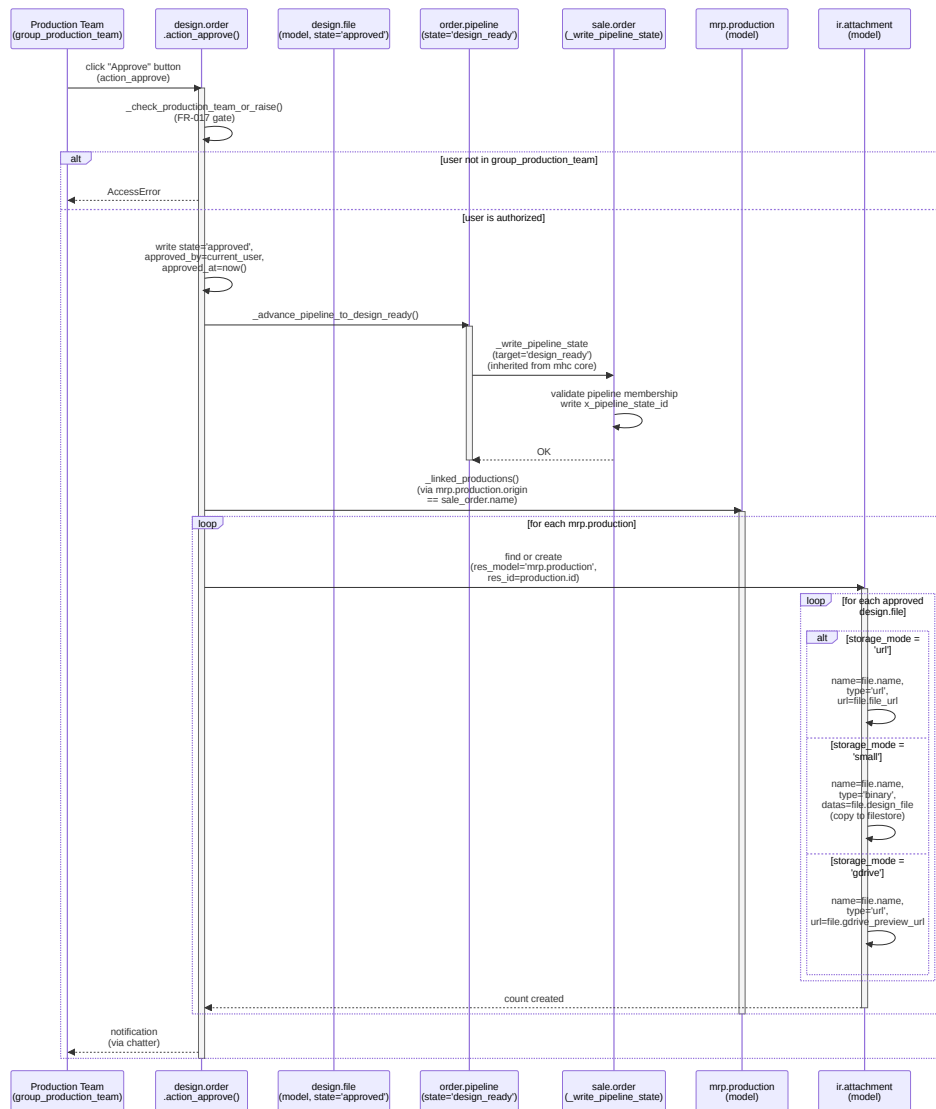
**Implemented:** Phase 1 (P1-02a design file routing, P1-02b auto-create)

**Status:** Shipped, E2E tested with 5-design rule set

## SEQUENCE DIAGRAM



## DESIGN APPROVAL ACTION SEQUENCE



## PROSE WALKTHROUGH — APPROVAL FLOW

- Config Check:** On SO confirmation, if `design.auto_create_on_confirm=True` (default), a `design.order` is auto-created.
- Design File Collection:** All `design.file` rows linked to the SO (header-level via `order_id` or line-level via `order_line_id`) are collected and re-parented under the new `design.order` if orphaned.
- Approval Gate (FR-017):** Production team clicks "Approve" button. System checks user is in `group_production_team` or `group_system` BEFORE any write.
- State Transition:** Design order transitions to `state='approved'`. Timestamps and approver name are recorded.
- Pipeline Sync:** Sale order advances to the `design_ready` pipeline stage (via mhc core's `_write_pipeline_state`). This is an automatic, audited transition (recorded in `order.pipeline.transition.log`).

6. **Attachment to Productions:** System queries for `mrp.production` records linked to the same SO (via `origin` field). For each production, approved design files are attached as `ir.attachment` records:

- **URL storage:** Link-style attachment (no filestore copy)
- **Binary (small):** Copies the file binary to filestore (≤10 MB cap)
- **GDrive storage:** Creates a link attachment with GDrive shareable URL

7. **Idempotency:** Attachments are deduplicated by (`design.file_id`, `mrp.production_id`) to avoid re-attaching on repeated approvals.

### ERROR PATHS

- **AccessError:** Non-production-team user attempts approval. Blocked at the gate (FR-017 defense).
  - **No Linked Productions:** If MO not yet created, `_linked_productions()` returns empty. No error; attachment step is skipped (idempotent).
  - **File Size Exceeds 10 MB:** Design file creation fails at constraint check; operator must re-upload smaller file or use GDrive URL storage mode.
  - **GDrive Preview URL Invalid:** `gdrive_preview_url` computed field returns null or malformed URL. Attachment is still created but operators will see blank link.
- 

## 4. Dropship Quote Handshake (ESTY-246, ADR-018-gearment-po-quote)

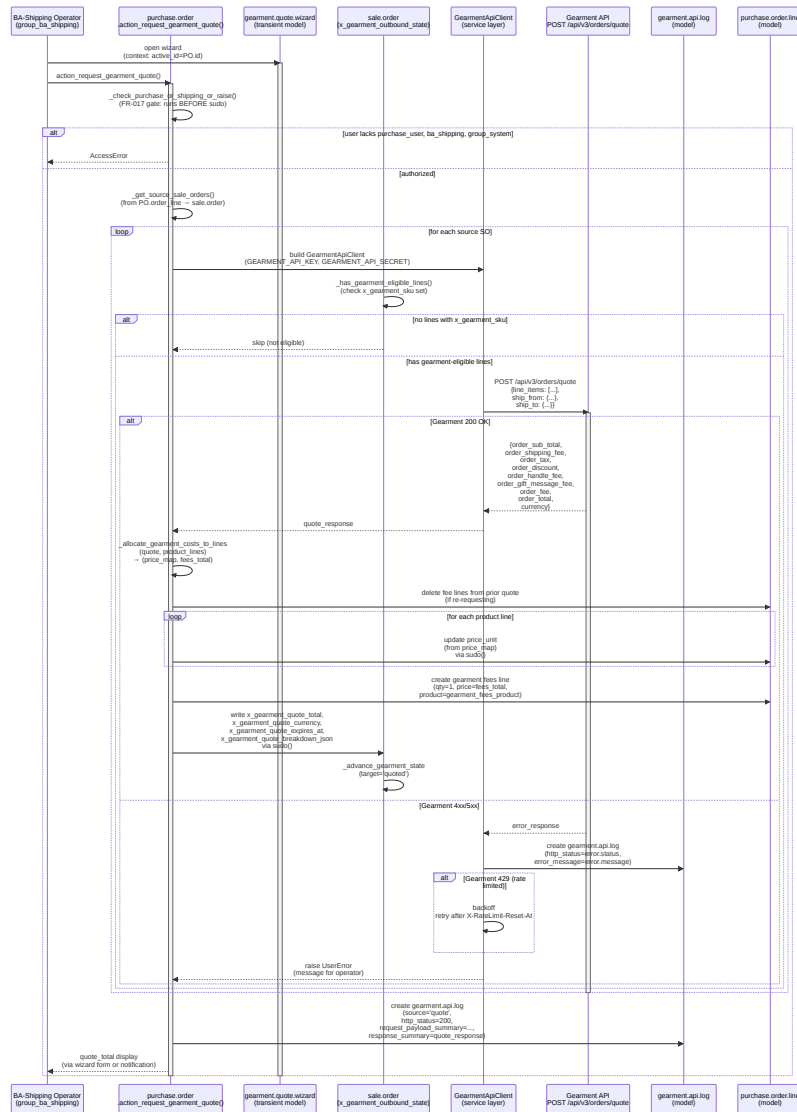
**Entry Point:** Purchase order confirm → Gearment quote request (action/wizard)

**Implemented:** Phase 1 (P4-01 sub-phases C–D completed 2026-05-10, hotfixes 2026-05-12)

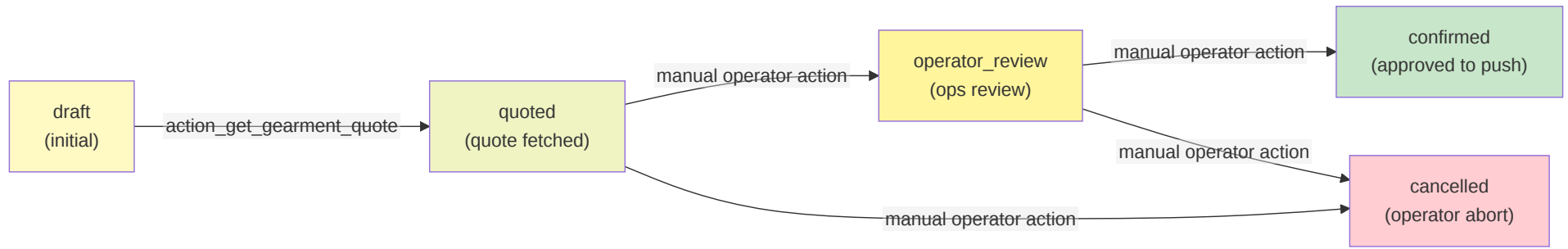
**Status:** Shipped, E2E integrated with webhook tracking (Phase 2)

## Part 1: System Architecture

### SEQUENCE DIAGRAM



### STATE MACHINE DIAGRAM





## PROSE WALKTHROUGH

1. **Quote Request:** BA-Shipping operator clicks the "Request Gearment Quote" button on a purchase order. FR-017 gate checks authorization BEFORE any writes.
2. **Source Sales Orders:** PO identifies all source sale orders from its order lines (reverse-link via `purchase.order_line`). Filters for orders with  $\geq 1$  line marked `x_gearment_sku` (POD-eligible SKU).
3. **Quote Payload Build:** For each eligible SO, builds a Gearment POST payload with:
  - `line_items`: product SKU, quantity, unit price, printing options
  - `ship_from`: warehouse origin address
  - `ship_to`: order's delivery address
4. **API Call:** Sends quote request to Gearment `/api/v3/orders/quote`. On success, receives itemized cost breakdown (subtotal, shipping, tax, fees).
5. **Cost Allocation:** Allocates Gearment's order-level totals across PO lines:
  - Product lines: `price_unit` ← allocated share of `order_sub_total`
  - Fees line (one per SO): `qty=1`, `price=remainder` (shipping+tax+fees)
  - Residual rounding assigned to largest line
6. **Sale Order State Transition:** Sale order's `x_gearment_outbound_state` transitions `draft→quoted`. Quote breakdown (JSON serialized) is stored on the SO for operator visibility.
7. **Quote Expiry:** Gearment quotes are time-bound (e.g., 24 hours). Operator must confirm the PO within that window or re-fetch the quote.

## ERROR PATHS

- **AccessError (FR-017):** Non-authorized user attempts quote request. Blocked before any ORM writes.
- **No Eligible Lines:** SO has no `x_gearment_sku` set. Silently skipped (no error).
- **429 Rate Limit:** Gearment API rate limit hit. Client respects `X-RateLimit-Reset-At` header and backs off. Audit log records the 429.
- **Gearment 4xx/5xx:** Server error or validation failure (e.g., invalid address format). Wrapped in `UserError` and raised to operator.
- **Network Timeout:** Transaction rolls back; operator retries.

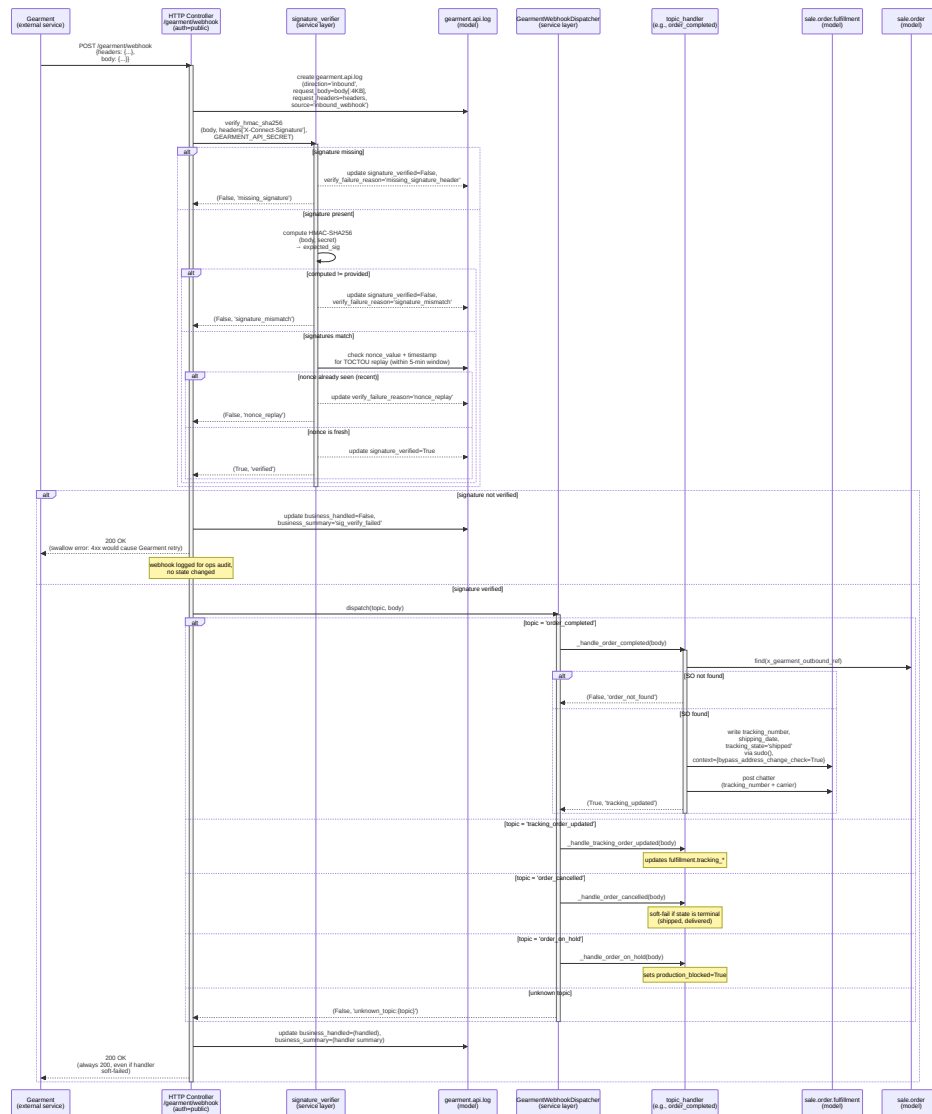
## 5. Gearment Webhook Inbound (P0-18b2c, ADR-015)

**Entry Point:** HTTP POST `/gearment/webhook`

**Implemented:** Phase 0 (P0-18b2a audit log, P0-18b2b HMAC verify, P0-18b2c dispatch)

**Status:** Shipped, E2E integrated with tracking updates and fulfillment state

## SEQUENCE DIAGRAM



## PROSE WALKTHROUGH

- HTTP Endpoint:** `GearmentWebhookController` (`auth='public'`) receives inbound webhook POST. Request is logged to `gearment.api.log` before any processing.
- HMAC Verification:** Payload is HMAC-SHA256 verified using the Gearment secret key. Signature is extracted from `X-Connect-Signature` header. If verification fails, the webhook is logged (for audit) but silently returns 200 (so Gearment doesn't retry failed attempts).
- Replay Protection:** Nonce value (from `X-Connect-Nonce` header) and timestamp (from `X-Connect-Timestamp`) are checked against a UNIQUE partial index in `gearment.api.log`. If the (nonce, timestamp) pair is already verified in the DB, it's a replay—skipped.
- Topic Dispatch:** After verification, `GearmentWebhookDispatcher.dispatch()` routes the webhook to a per-topic handler:
  - `order_completed`: Order shipped from Gearment. Update fulfillment tracking, mark as shipped.
  - `tracking_order_updated`: Tracking number or carrier changed. Update fulfillment.
  - `order_cancelled`: Order was cancelled at Gearment. Rollback (soft-fail if already shipped).
  - `order_on_hold`: Production blocked (QA issue, etc.). Flag `production_blocked=True`.
  - Others (variants, address): Log-only (no state change).

5. **Address-Change Bypass (FR-017):** Fulfillment writes use `bypass_address_change_check=True` context because webhooks are inbound, trusted events. This allows Gearment tracking to be recorded even if an operator has a pending address-change request.
6. **Chatter Notification:** Handler posts a chatter entry on the SO documenting the tracking number and carrier name (escaped for XSS prevention).
7. **Audit Trail:** Log row is updated with `business_handled=True/False` and `business_summary` (human-readable result). All 200 responses (even soft-failures) are returned to Gearment so it doesn't retry.

### ERROR PATHS

- **Missing Signature Header:** Webhook logged, silently swallowed (200).
  - **Signature Mismatch:** Possible tampering or secret misconfiguration. Logged, swallowed (200).
  - **Nonce Replay:** Duplicate webhook detected. Swallowed (200).
  - **Order Not Found:** SO with matching `x_gearment_outbound_ref` not found locally. Soft-fail, logged (200).
  - **Address Change Conflict:** If operator has a pending address-change request and Gearment sends tracking, the tracking is still recorded (bypass is enabled). Operator resolves the conflict manually.
- 

## 6. Tracking Import from GKE Excel (P2-01/P2-02)

**Entry Point:** Manual wizard upload or GDrive cron poll (gdrive\_partner)

**Implemented:** Phase 2 (P2-01 wizard, P2-02 carrier detection, P2-06 GDrive poller)

**Status:** Shipped, E2E tested with 5-carrier detection ruleset



6. **Order Matching:** Order number is looked up in `sale.order` by name or `etsy_order_id`. If found, the line is marked `state='matched'` and linked to the SO and its fulfillment record.
7. **Carrier Detection:** Carrier label is matched against a regex ruleset (e.g., `/^USPS/` → `shipping.carrier 'usps'`). If matched, `detected_carrier_id` is set. If no match, falls back to 'other' and sets `needs_review=True`.
8. **Date Parsing:** Shipping date is parsed; if invalid, the line is marked `state='error'` with an error message.
9. **Summary Recount:** After all rows are parsed, summary counts are recomputed via a `read_group` call on child lines by state.
10. **Fulfillment Updates:** Matched lines trigger writes to `sale.order.fulfillment`:
  - `tracking_number` ← raw tracking number
  - `tracking_state` ← 'shipped' (if not already shipped/delivered)
  - `shipping_carrier_id` ← detected carrier (only if currently null)
  - `shipping_date` ← parsed date

## ERROR PATHS

- **File Too Large:** C-TIL-001 constraint violated. Rejected with error message.
- **New Schema:** Hash not in approved set. Log flagged; operator must review columns before import is considered valid.
- **Order Not Found:** Order number doesn't match any SO. Line marked `state='unmatched'`.
- **Invalid Date:** Date parsing fails. Line marked `state='error'`.
- **Carrier Not Detected:** Label matches no regex. Marked `needs_review=True` for operator to manually select carrier.
- **Duplicate Row:** Same source\_row\_hash seen in this import. Marked `state='duplicate'`, skipped.

## Outbound & Production Completion Workflows

Workflows 7 (Listing Publish) and 8 (Production Completion) are documented in the companion document [04b-sequence-flows.md](#):

- **Workflow 7:** Etsy Listing Outbound Publish (P-PUB-PUBLISH, Spec 011) — Planned, Phase 3
- **Workflow 8:** Production Completion Hook (P2-03, ADR-016) — Shipped, integrated with fulfillment lifecycle

See [04b-sequence-flows.md](#) for detailed sequence diagrams, error paths, and implementation notes.

## Audit & Observability

All 8 workflows emit structured audit logs:

| Workflow           | Audit Table                                                          | Key Fields                                                                                   |
|--------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 1–2 (Order Ingest) | <code>etsy.api.log</code> / <code>etsy.email.log</code>              | endpoint, http_status, request_summary, response_summary, error_message                      |
| 3 (Design)         | <code>order.pipeline.transition.log</code>                           | sale_order_id, from_state, to_state, change_type, note, created_by                           |
| 4 (Quote)          | <code>gearment.api.log</code>                                        | sale_order_id, http_status, request_payload, response_summary, rate_limit_remaining          |
| 5 (Webhook)        | <code>gearment.api.log</code>                                        | sale_order_id, signature_verified, verify_failure_reason, business_handled, business_summary |
| 6 (Tracking)       | <code>tracking.import.log</code> / <code>tracking.import.line</code> | state, matched_count, unmatched_count, source_row_hash, detected_carrier, needs_review       |
| 7 (Publish)        | <code>etsy.api.log</code> (future)                                   | endpoint, http_status, request_payload, listing_id, error_messages                           |
| 8 (Production)     | <code>order.pipeline.transition.log</code>                           | fulfillment_status='produced', stock.move.purpose='production_completion'                    |

All logs are retained per ICP configuration (default 30 days for API logs, configurable).

## Document Metadata

**Last Updated:** 2026-07-03

**Authored For:** Owner review + internal team reference

**Code Verified Against:**

- `custom_addons/etsy_integration/` (v19.0.3.15.0)
- `custom_addons/multichannel_hub_core/` (v19.0.1.0.75)
- `custom_addons/multichannel_hub_fulfillment/` (v19.0.1.0.24)
- `custom_addons/design/` (v19.0.1.0.0)

**Entry Points for Workflows 1–6:**

1. Etsy API: `etsy.shop._cron_sync_orders()`
2. Email: `etsy_integration._cron_sync_emails()`
3. Design: `sale.order.action_confirm()` → `_ensure_design_order()`
4. Quote: `purchase.order.action_request_gearment_quote()` / wizard
5. Webhook: HTTP POST `/gearment/webhook`
6. Tracking: `tracking.import.wizard` or `logistics_partner._cron_poll_inbox()`

**Related Document:** See 04b-sequence-flows.md for entry points of workflows 7–8 (outbound publish, production completion).

---

**title:** “SDS 04b: Sequence Flows (Outbound Publish & Production)” **date:** “2026-07-03” **status:** “Draft-for-owner-review” **source\_of\_truth:** “All flows traced from actual code in custom\_addons/”

## Sequence Flows — Outbound Channels & Production Completion

This document defines 2 key workflows (workflows 7–8 from the 8-flow set), traced through actual Odoo code and architectural patterns. Each flow includes a prose walkthrough, error paths (alt/opt blocks), and participant names matching real classes/models.

**Related Document:** See `04a-sequence-flows.md` for workflows 1–6 (inbound order ingestion, design approval, dropship quote, Gearment webhook, and tracking import).

---

### 7. Listing Outbound Publish to Etsy (P-PUB-PUBLISH, Spec 011)

**Entry Point:** Form button `action_push_listing_to_etsy()` or bulk wizard

**Implemented:** Phase 3 (P-PUB-CLIENT, P-PUB-DRAFT, P-PUB-IMAGES, P-PUB-INVENTORY, P-PUB-PUBLISH, P-PUB-E2E)

**Status:** Planned (not yet shipped; Phase 3 is 1% complete as of 2026-07-03)

**Blockers:** P-HUB-SPEC (detailed feature spec in progress)

#### ARCHITECTURE OVERVIEW

Listing publication to Etsy is a multi-step stateful process orchestrated via `multichannel_listing` (Spec 009 model, hosted in `multichannel_hub_core`). The workflow leverages the Etsy v3 API contract:

- **Draft Creation:** POST `/v3/application/shops/{shop_id}/listings` with initial metadata
- **Image Upload:** POST `/v3/application/shops/{shop_id}/listings/{listing_id}/images` for variants
- **Inventory Push:** POST `/v3/application/shops/{shop_id}/listings/{listing_id}/inventory` for quantity per variant
- **Status Poll** (optional): GET `/v3/application/shops/{shop_id}/listings/{listing_id}` to check readiness





## Variant Encoding

Each product variant is encoded as a Etsy "product" with `property_values`:

```
{
  "sku": "ABC-001-S-RED",
  "offering_id": 1,
  "quantity": 10,
  "price": 29.99,
  "property_values": [
    {
      "property_id": "200",
      "property_name": "Size",
      "values": ["Small"]
    },
    {
      "property_id": "500",
      "property_name": "Color",
      "values": ["Red"]
    }
  ]
}
```

Property mappings are configured via `etsy.shop.default_attribute_mapping_ids` (M2M, Spec 001). Each row maps an Odoo `product.attribute` to an Etsy property ID.

## Image Handling (Spec 011 P-PUB-IMAGES)

Images are sourced from:

1. **Primary:** `design.file` records linked to the listing (`storage_mode=url/small/gdrive`)
2. **Fallback:** `product_template_image_ids` (if no design files)

Images are uploaded via POST `/listings/{id}/images` with `rank` parameter to control carousel order.

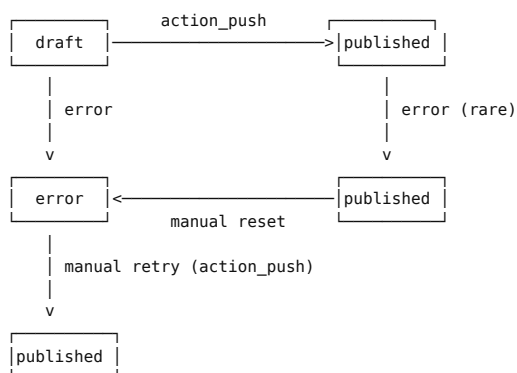
## Inventory (Spec 011 P-PUB-INVENTORY)

Inventory levels per variant are read from `stock.quant` filtered by:

- `product_id` = variant
- `location_id` = shop's default warehouse
- `company_id` = shop's company

Quantity is pushed to Etsy via POST `/listings/{id}/inventory`.

## STATE MACHINE



## ERROR PATHS

- **Incomplete Listing:** Missing required field (title, price, image, `readiness_state_id`). Operator shown checklist. Publish blocked.

- **Invalid Property Value:** Etsy rejects a property\_value encoding (e.g., unknown Etsy property\_id, or value not in Etsy's enum for that property). 400 response. Error message shows which field and why (e.g., "Property 'Size' expects value in ['Small', 'Medium', 'Large']"). Operator fixes product attribute on the template; retries publish.
- **Token Expired:** 401 response. Client auto-refreshes token and retries. If refresh fails, UserError raised (operator re-authorizes).
- **Rate Limited:** 429 response. Backoff per `X-RateLimit-Reset-At` header; user is instructed to retry after N seconds.
- **Network/Timeout:** Transaction rolls back. Operator retries.
- **Inventory Push Fails** (after listing created): Listing is marked published but inventory is incomplete. Operator navigates to the listing record and manually clicks "Sync Inventory" to retry the inventory push.

### IDEMPOTENCY & RE-PUBLISH

If operator clicks publish twice:

1. First call: 201 Created, listing published, `etsy_listing_id` stored.
2. Second call: `action_push_listing_to_etsy()` checks if `etsy_listing_id` is already set. If yes, raise UserError with hint to use "Update" flow instead (not yet implemented in Phase 3).

---

## 8. Production Completion Hook (P2-03, ADR-016-production-completion)

**Entry Point:** Fulfillment state machine transition to `produced`

**Implemented:** Phase 2 (P2-03 completed 2026-05-10, hotfixes 2026-05-16)

**Status:** Shipped, E2E integrated with fulfillment lifecycle

**Related:** ESTY-244 (design approval) + P1-05 (fulfillment delegation mixin)

### PURPOSE & CONTEXT

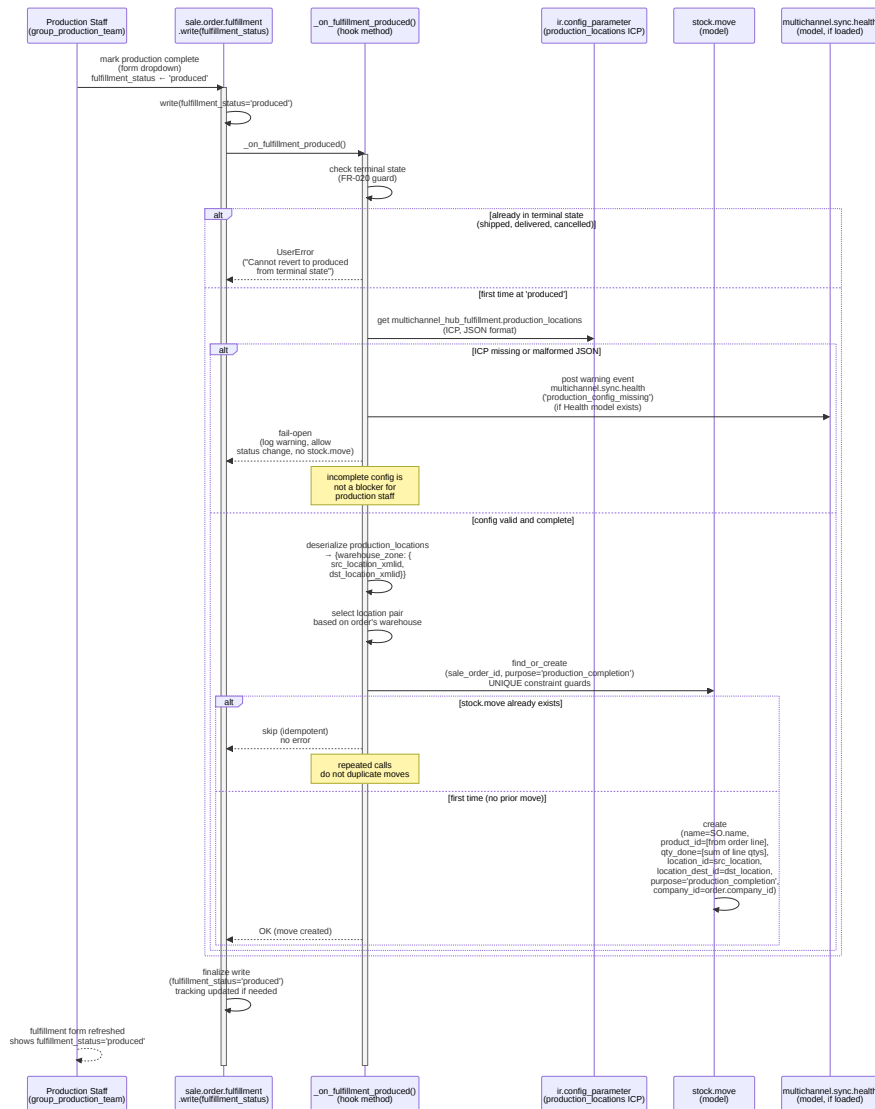
The production completion hook (workflow 8) bridges two internal state spaces:

- **Fulfillment state** (external-facing): tracks order from creation → shipped → delivered
- **Production state** (internal): tracks warehouse progress from pending → produced → quality-checked → ready-to-ship

When fulfillment transitions to `produced`, an internal `stock.move` record is created to audit the production completion. This allows:

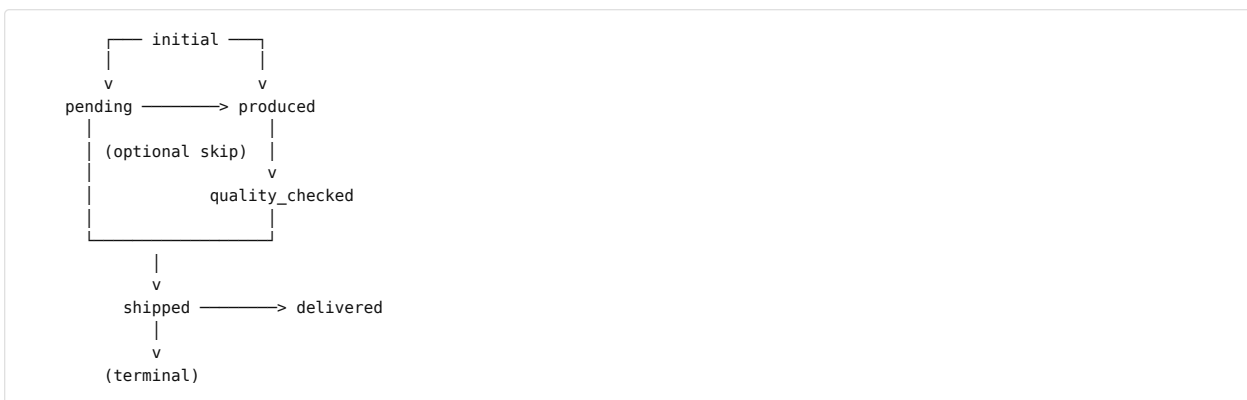
- Warehouse staff to track inventory movements during production
- Finance/compliance to report on goods produced per order
- Automatic triggers for downstream quality checks or label printing

## SEQUENCE DIAGRAM



## STATE MACHINE CONTEXT (FROM P1-05)

`sale_order.fulfillment` has a `fulfillment_status` state machine:



The `production_completion` hook fires **only** on transition to 'produced', not on transitions from produced to subsequent states.

## PRODUCTION LOCATIONS ICP CONFIGURATION

The configuration is a JSON object mapping warehouse zones to (source, destination) location XMLIDs:

```
{
  "vn_hanoi": {
    "src": "stock.location:vn_hanoi_production_input",
    "dst": "stock.location:vn_hanoi_qc_holding"
  },
  "vn_hcm": {
    "src": "stock.location:vn_hcm_production_input",
    "dst": "stock.location:vn_hcm_ready_to_ship"
  }
}
```

Key points:

- **src\_location**: Input to the production bin (e.g., raw materials)
- **dst\_location**: Output from production (e.g., quality-control holding, or ready-to-ship)
- **warehouse\_zone** key maps to order's warehouse (or fallback to default)

If the warehouse's zone is not in the config, a fallback location pair is used (configurable via a second ICP key, e.g., `production_locations_default`).

## STOCK MOVE SEMANTICS

The created `stock.move` records production completion via stock movements:

| Field                         | Value                                | Semantics                                               |
|-------------------------------|--------------------------------------|---------------------------------------------------------|
| <code>name</code>             | <code>sale.order.name</code>         | Order reference                                         |
| <code>product_id</code>       | First SO line's product              | (Multi-product orders: aggregated into one move per SO) |
| <code>qty_done</code>         | Sum of all SO line quantities        | Completed units                                         |
| <code>location_id</code>      | <code>src_location</code> (input)    | Production started                                      |
| <code>location_dest_id</code> | <code>dst_location</code> (output)   | Production finished                                     |
| <code>purpose</code>          | <code>'production_completion'</code> | Special marker (not standard Odoo)                      |

The UNIQUE constraint `(sale_order_id, purpose)` ensures idempotency: re-firing the hook does not create duplicate moves.

## ERROR PATHS

- **FR-020 Violation**: Order already shipped, delivered, or cancelled. Transition rejected with `UserError`. Message: "Cannot mark as produced after fulfillment is already shipped/delivered/cancelled."
- **ICP Config Missing**: Log warning to `sync.health` (soft-fail). Fulfillment status still changes; `stock.move` not created. Operator manually fixes config and optionally re-runs the hook via an action button.
- **ICP Config Malformed**: JSON parse error. Logged as warning. Soft-fail as above.
- **Location XMLID Not Found**: Destination location doesn't exist in `stock.location`. Stock move creation fails (rare). Logged; operator fixes XMLID reference in config.
- **Multiple Products on Order**: Stock move aggregates qty into one move for the first product. Caveat: orders with true multi-product scenarios may need refinement in Phase 3+.

## OBSERVABILITY

Production completion is audited via:

1. **Fulfillment write log**: `fulfillment_status` field is `tracking=True`, so state changes trigger chatter entries.
2. **Stock move**: Created record is queryable via `stock.move.purpose='production_completion'` and linked via `stock.move.sale_order_id`.

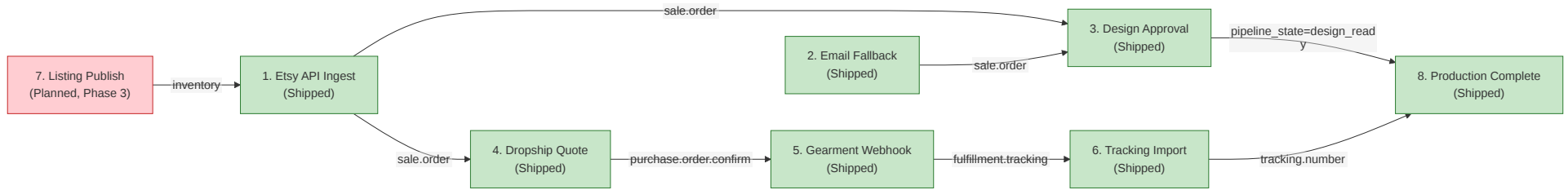
3. **Sync health events** (if `etsy_integration.models.sync_health` is loaded): Config errors are posted as warnings.

---

## Cross-Workflow Reference & Sequencing

**Complete Workflow DAG** (combining 04a and 04b):

## Part 1: System Architecture



**Timing Constraints:**

- Workflows 1–2 run **every 10 minutes** (cron)
- Workflow 3 triggers on **sale order confirmation** (real-time)
- Workflow 4 triggers on **operator action** (manual or bulk server action)
- Workflow 5 is **event-driven** (Gearment webhook, ~5 min latency)
- Workflow 6 runs **every 15 minutes** (GDrive poll) or **manual import**
- Workflow 7 is **operator-initiated** (when ready to publish)
- Workflow 8 triggers on **fulfillment state change** (real-time, production staff)

**Critical Path for E2E Order Fulfillment:**

```

1. Order Ingest (API or Email)
  ↓
3. Design Approval (production gate)
  ↓
4. Dropship Quote (if applicable)
  ↓ [manual PO confirm]
5. Gearment Webhook (tracking)
  ↓
6. Tracking Import (receive tracking number)
  ↓
8. Production Complete (internal audit)

```

**Decoupled Path:**

- Workflow 7 (Listing Publish) is decoupled from the order fulfillment path. It feeds inventory to upstream (workflow 1) but does not depend on order ingest.

**Audit & Observability (Complete Set)**

All 8 workflows emit structured audit logs and state machines:

**AUDIT TABLES**

| Workflow | Audit Table                                   | Purpose                                       | Retention                             |
|----------|-----------------------------------------------|-----------------------------------------------|---------------------------------------|
| 1–2      | etsy.api.log / etsy.email.log                 | API/email call history; PII scrubbed          | 30 days (ICP configurable)            |
| 3        | order.pipeline.transition.log                 | State machine audit                           | Forever (configurable)                |
| 4        | gearment.api.log                              | Quote request/response history                | 30 days (ICP configurable)            |
| 5        | gearment.api.log                              | Webhook receipt + verification                | 30 days (ICP configurable)            |
| 6        | tracking.import.log /<br>tracking.import.line | Excel import audit + line-level parse results | Forever (imported rows are queryable) |
| 7        | etsy.api.log (future)                         | Listing publish request/response              | 30 days (ICP configurable)            |
| 8        | order.pipeline.transition.log /<br>stock.move | Fulfillment state + production stock move     | Forever                               |

## STATE MACHINES

| Workflow | Model                  | Field                     | States                                                      | Audited?               |
|----------|------------------------|---------------------------|-------------------------------------------------------------|------------------------|
| 3        | design.order           | state                     | pending → proof_sent → approved → rejected                  | Yes (chatter tracking) |
| 4        | sale.order             | x_gearment_outbound_state | draft → quoted → operator_review → confirmed (or cancelled) | Yes (TR log)           |
| 8        | sale.order.fulfillment | fulfillment_status        | pending → produced → quality_checked → shipped → delivered  | Yes (chatter tracking) |

## OBSERVABILITY DASHBOARDS (PER CLAUDE.MD &amp; PHASE 1)

- **Order Dashboard** (P1-01a): Real-time view of inbound orders (Workflow 1–2). Filters by sales\_channel, state, date range.
- **Tracking Dashboard** (P1-03): Real-time view of fulfillment progress (Workflow 5–8). Bus.bus channel integration for live updates.
- **Operations Dashboard** (P1-01b refactored): Line-level bulk actions for Gearment pushes (Workflow 4), tracking imports (Workflow 6), design approvals (Workflow 3).

## Implementation Roadmap (Phase 3)

**Workflow 7 (Listing Publish) is NOT YET SHIPPED.** Planned implementation:

| Slice            | Status                | Dependencies     | Owner    |
|------------------|-----------------------|------------------|----------|
| P-HUB-SPEC       | In Progress (planner) | None             | Planner  |
| P-HUB-PROD-MODEL | Planned               | P-HUB-SPEC       | Eng      |
| P-HUB-WIZARD     | Planned               | P-HUB-PROD-MODEL | Eng      |
| P-PUB-CLIENT     | Planned               | P-HUB-SPEC       | Eng      |
| P-PUB-DRAFT      | Planned               | P-PUB-CLIENT     | Eng      |
| P-PUB-IMAGES     | Planned               | P-PUB-DRAFT      | Eng      |
| P-PUB-INVENTORY  | Planned               | P-PUB-DRAFT      | Eng      |
| P-PUB-PUBLISH    | Planned               | P-PUB-INVENTORY  | Eng      |
| P-PUB-E2E        | Planned               | P-PUB-PUBLISH    | Eng + QA |

**Phase 3 Exit Criteria** (for Workflow 7):

- All 8 slices complete and merged to main
- E2E test suite passes (Playwright + OdoO HttpCase)
- Staging validation: publish 5 test listings to Etsy sandbox
- Production cutover: publish 1 real product to JaHandmadeArt shop

## Document Metadata

**Last Updated:** 2026-07-03

**Authored For:** Owner review + internal team reference

**Code Verified Against:**

- custom\_addons/etsy\_integration/ (v19.0.3.15.0)
- custom\_addons/multichannel\_hub\_core/ (v19.0.1.0.75)
- custom\_addons/multichannel\_hub\_fulfillment/ (v19.0.1.0.24)
- custom\_addons/design/ (v19.0.1.0.0)



**Entry Points for Workflows 7–8:** 7. Publish: `multichannel.listing.action_push_listing_to_etsy()` (not yet implemented) 8. Production: `sale.order.fulfillment.write(fulfillment_status='produced')`

### Related SDS Documents:

- `04a-sequence-flows.md` — Workflows 1–6 (inbound channels)
- `05-data-model-ER.md` — Entity relationship diagrams for all models
- `03-architectural-decisions.md` — ADRs including ADR-014 (central product hub), ADR-016 (production completion), ADR-018 (Gearment quote state machine)

### References:

- `specs/001-etsy-order-migration/spec.md` — Initial Etsy order ingestion spec
- `specs/005-etsy-api-channel/spec.md` — API vs email channel selection
- `specs/009-product-hub/spec.md` — Central product catalog (Phase 3)
- `specs/011-etsy-publish/spec.md` — Etsy listing publish flow (Phase 3, P-PUB-\* slices)
- `.claude/plans/006-master-plan-tracking.md` — Phase-by-phase execution tracker

## Part 5: Security Posture and Access Control

**Title:** odoo19\_esty Security Architecture, Access Control Lists, and Defense Patterns

**Date:** 2026-07-03

**Status:** Draft-for-owner-review

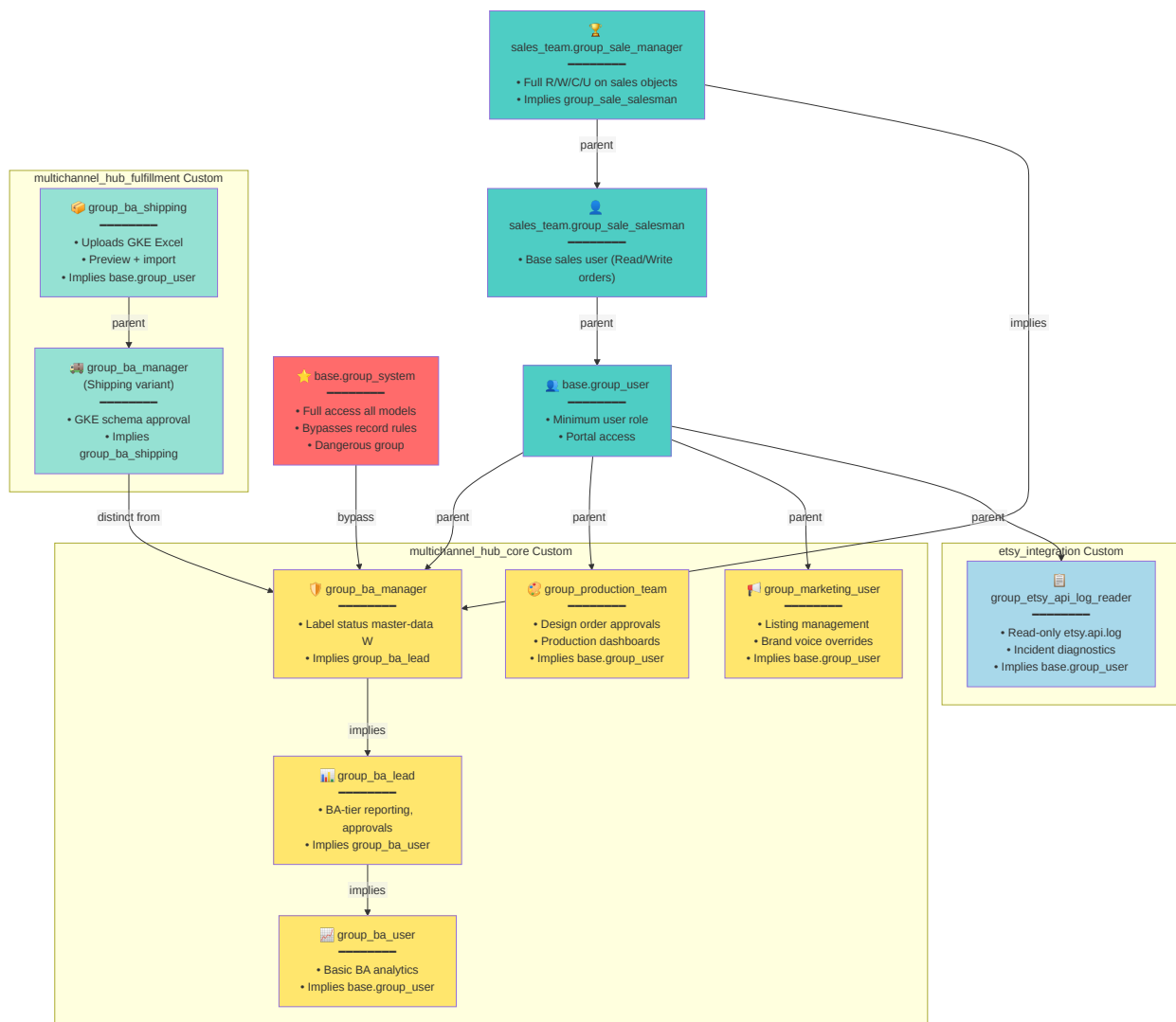
**Version:** 1.0

**Source of Truth:** custom\_addons//security/.xml, custom\_addons/\*/security/ir.model.access.csv, specs/006-master-plan/adrs/ADR-018a (web-hook), ADR-018b (OAuth)

### (a) Groups & Role Hierarchy

All custom security groups defined in XML; inherits from Odoo base groups ( `base.group_user` , `sales_team.group_sale_salesman` , `sales_team.group_sale_manager` , `base.group_system` ).

#### GROUP HIERARCHY



#### Group Ownership:

- **Standard Odoo:** `base.group_system` , `sales_team.group_sale_manager` , `sales_team.group_sale_salesman` , `base.group_user`
- **multichannel\_hub\_core:** `group_production_team` , `group_marketing_user` , `group_ba_user` , `group_ba_lead` , `group_ba_manager`

- **multichannel\_hub\_fulfillment:** `group_ba_shipping`, `group_ba_manager` (Shipping variant, distinct from mhc variant)
- **etsy\_integration:** `group_etsy_api_log_reader`

**Note:** Two `group_ba_manager` groups exist (mhc + fulfillment). Distinct XML IDs prevent collision; semantically different (master-data vs. schema approval). Future consolidation candidate in Specs/015.

## (b) Roles-to-Menus Matrix

| Role                                                 | Menus & Features                                                                   | Models                                                                                                                      | Typical User                             |
|------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>base.group_system (System)</b>                    | ALL (unrestricted)                                                                 | All                                                                                                                         | Developers, sysadmin (DANGEROUS)         |
| <b>sales_team.group_sale_manager (Sales Manager)</b> | Sales + Orders + Dashboards + all Operations Dashboards (Order/Tracking/Process)   | <code>sale.order</code> (CRUD), <code>design.file</code> (CRUD), <code>order.pipeline</code> (CRUD), all fulfillment (CRUD) | Sales team lead, operations lead         |
| <b>sales_team.group_sale_salesman (Salesman)</b>     | Sales + Orders (read-only pipeline), limited fulfillment                           | <code>sale.order</code> (R/W orders), <code>design.file</code> (R), fulfillment (read-only tracking)                        | Order taker, fulfillment operator        |
| <b>group_ba_manager</b>                              | BA Dashboard (all metrics), Master Data (label statuses), Filtering saved-searches | <code>label.status.option</code> (CRUD), Operations Dashboard unrestricted, saved filters                                   | Senior BA/ operations manager            |
| <b>group_ba_lead</b>                                 | BA Dashboard (approvals, metrics), Enquiry management                              | <code>multichannel.enquiry</code> (CRUD), Operations Dashboard filtered                                                     | BA lead, operations analyst              |
| <b>group_ba_user</b>                                 | BA Dashboard (read-only metrics)                                                   | Operations Dashboard read-only metrics                                                                                      | Junior BA, analyst                       |
| <b>group_production_team</b>                         | Design approvals, production dashboards                                            | <code>design.order</code> (R/W), <code>design.file</code> (R/W), production dashboard                                       | Production team, design approvers        |
| <b>group_marketing_user</b>                          | Listing management, brand overrides                                                | <code>multichannel.listing</code> (CRUD, gated unlink), brand-voice fields                                                  | Marketing, listing manager               |
| <b>group_ba_shipping (Fulfillment)</b>               | GKE import wizard, preview, tracking import                                        | Tracking models (CRUD), GKE schema preview (no approve)                                                                     | GKE operator, tracking import specialist |
| <b>group_ba_manager (Shipping variant)</b>           | GKE schema approval, tracking import, all shipping                                 | Tracking models (CRUD), GKE schema approval (RPC-gated)                                                                     | Shipping manager, GKE quality gate       |
| <b>group_etsy_api_log_reader</b>                     | Read-only Etsy API log                                                             | <code>etsy.api.log</code> (read-only), incident diagnostics                                                                 | Support, incident reviewer               |

## (c) Access Control List (ACL) Summary by Model

**Format:** Model → Permissions by Group

**Legend:** R=Read, W=Write, C=Create, U=Unlink

**FOUNDATION (MULTICHANNEL\_HUB\_CORE)**

| Model                                      | group_sale_salesman | group_production_team | group_sale_manager | group_ba_manager | base.group_system |
|--------------------------------------------|---------------------|-----------------------|--------------------|------------------|-------------------|
| <code>sale.order</code> (extended)         | RW                  | RWC                   | RWCU               | RWCU             | RWCL              |
| <code>sale.order.fulfillment</code>        | RWC                 | RWC                   | RWCU               | RWCU             | RWCL              |
| <code>design.file</code>                   | R                   | RWC                   | RWCU               | RWCU             | RWCL              |
| <code>design.file.route</code>             | R                   | RWC                   | RWCU               | RWCU             | RWCL              |
| <code>design.file.upload.wizard</code>     | —                   | RWC                   | —                  | —                | RWCL              |
| <code>order.pipeline</code>                | R                   | —                     | RWC                | RWC              | RWCL              |
| <code>order.pipeline.state</code>          | R                   | —                     | RWC                | RWC              | RWCL              |
| <code>order.pipeline.transition.log</code> | R                   | —                     | R                  | R                | RWCL              |
| <code>pipeline.team</code>                 | R                   | —                     | RWC                | RWC              | RWCL              |
| <code>shipping.carrier</code>              | R                   | —                     | RWC                | RWC              | RWCL              |
| <code>label.status.option</code>           | R                   | —                     | RW                 | RWCU             | RWCL              |
| <code>multichannel.enquiry</code>          | RWC                 | —                     | RWCU               | RWCU             | RWCL              |
| <code>multichannel.listing</code>          | RWC (gated unlink)  | —                     | RWCU               | RWCU             | RWCL              |
| <code>multichannel.sync.health</code>      | R                   | —                     | RW                 | RW               | RWCL              |
| <code>product.mto.bom.wizard</code>        | —                   | —                     | RWC                | —                | RWCL              |

**Footnotes:**

- `sale.order.fulfillment` : Salesman can write (e.g., tracking number); Production team reads fulfillment state for approval workflow
- `design.file` : Production team has R/W/C (but not U via record rule on approval workflow)
- `order.pipeline*` : Read-only for salesman (see pipeline status); Manager/BA full control
- `label.status.option` : Label statuses are master data; only BA Manager (P1-LBL) can create/edit
- `multichannel.listing` : Unlink restricted by record rule when state='published' (see § (d) Record Rules)

**FULFILLMENT (MULTICHANNEL\_HUB\_FULFILLMENT)**

| Model                              | group_ba_shipping | group_ba_manager (Shipping) | base.group_system |
|------------------------------------|-------------------|-----------------------------|-------------------|
| <code>gearment.api.log</code>      | R                 | RW                          | RWCU              |
| <code>gearment.quote.wizard</code> | R                 | RWC                         | RWCU              |
| Tracking import wizard             | RWC               | RWC                         | RWCU              |

**ETSY CHANNEL (ETSY\_INTEGRATION)**

| Model                                 | group_sale_salesman    | group_sale_manager | group_etsy_api_log_reader | base.group_system |
|---------------------------------------|------------------------|--------------------|---------------------------|-------------------|
| <code>etsy.shop</code>                | R                      | RW                 | —                         | RWCU              |
| <code>etsy.email.log</code>           | R                      | RW                 | —                         | RWCU              |
| <code>etsy.api.log</code>             | R                      | R                  | R                         | RWCU              |
| <code>etsy.listing</code>             | RW                     | RWCU               | —                         | RWCU              |
| <code>sale.order</code> (Etsy-scoped) | R/W (shop-user-scoped) | RWCU               | —                         | RWCU              |

**DESIGN MODULE (DESIGN)**

| Model        | group_production_team | group_sale_manager | base.group_system |
|--------------|-----------------------|--------------------|-------------------|
| design.order | RWC                   | RWCU               | RWCU              |

**(d) Record Rules (Row-Level Access Control)**

Record rules enforce domain-based filtering; system group bypasses all record rules by default.

**MULTICHANNEL\_HUB\_CORE**

Rule: `rule_multichannel_listing_published_no_unlink`

- **Model:** `multichannel.listing`
- **Groups Affected:** `group_marketing_user`
- **Domain:** `[('state', '!=', 'published')]`
- **Permissions:** Unlink only (R/W/C pass-through)
- **Rationale** (ADR-015 §4): Published listings have `external_ref` pointing to live Etsy listing. Unlink would orphan channel mirror. Force explicit archive + state revert before unlink.
- **Bypass:** Sales Manager (`group_sale_manager`) can unlink all states; System group bypasses rule

**ETSY\_INTEGRATION**

Rule: `sale_order_etsy_shop_user_scope_rule`

- **Model:** `sale.order` (Etsy orders only)
- **Groups Affected:** `base.group_user`
- **Domain:** `[('|', ('etsy_shop_id', '=', False), ('etsy_shop_id.user_id', '=', user.id))]`
- **Permissions:** R/W/C/U (full, but scoped)
- **Rationale** (ADR-018b): Per-user shop scope. Users can see only orders from shops assigned to them; orders with no shop (non-Etsy) visible to all users.
- **Consequence:** Non-Etsy orders not scoped; users without assigned shop see no Etsy orders.

Rule: `sale_order_etsy_shop_admin_scope_rule`

- **Model:** `sale.order` (Etsy orders)
- **Groups Affected:** `sales_team.group_sale_manager`, `base.group_system`
- **Domain:** `[('1', '=', 1)]` (all)
- **Permissions:** R/W/C/U (unrestricted)
- **Rationale:** Sales Manager and System user see all Etsy orders, regardless of shop assignment.

**MULTICHANNEL\_HUB\_FULFILLMENT**

Rule: `gke_schema_approval_record_rule` (Future, P2-03)

- **Model:** `gearment.api.log` / tracking schema
- **Groups Affected:** `group_ba_shipping`
- **Domain:** `[('schema_approved', '=', True)]` (only approved schemas)
- **Permissions:** R (read-only until approved)
- **Rationale:** Shipping operators can see only approved schemas; prevents accidental import of unknown schemas.
- **Status:** Implemented P2-01; rule enforced via field-level ACL not XML rule (current approach uses `_check_schema_hash()` method in wizard).

**(e) Sudo Usage Audit and Justifications**

**Pattern:** `record.sudo()` escalates privileges when ORM constraints prevent legitimate access. All `sudo()` calls documented with inline comment.

**AUDIT TABLE: SUDO() USAGE ACROSS MODULES**

| File                                                                      | Method                                               | Model                                                  |
|---------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------------------------|
| <code>multichannel_hub_core/models/design_file.py</code>                  | <code>_attach_approved_files_to_productions()</code> | <code>ir.attachment</code>                             |
| <code>multichannel_hub_core/services/order_pipeline_service.py</code>     | <code>_write_pipeline_state()</code>                 | <code>sale.order</code>                                |
| <code>etsy_integration/services/order_syncer.py</code>                    | <code>sync_order()</code>                            | <code>res.partner</code> ,<br><code>sale.order</code>  |
| <code>etsy_integration/controllers/oauth_callback.py</code>               | <code>_exchange_code()</code>                        | <code>etsy.shop</code>                                 |
| <code>multichannel_hub_fulfillment/services/gearment_api_client.py</code> | <code>webhook_receiver()</code>                      | <code>gearment.api.1</code><br><code>sale.order</code> |

**Findings:**

-  All sudo() calls have inline comments explaining why
-  No blanket `sudo()` on entire class hierarchy
-  OAuth/webhook entry points have external signature verification (PKCE, HMAC)
-  Email parser runs as cron with system context; parser malfunction could spam orders. Mitigated by: (1) regex validation, (2) email audit log, (3) parse-failure notification, (4) Phase 2 cutover reduces reliance

**Recommendation:** Pre-Phase-1-exit, add integration test for malformed email (verify parse-failure path, check audit log written).

**(f) FR-017: Write-Level Defense Pattern**

**Context:** FR-017 mandates write-level ACL defense. Every state-changing action must gate writes at `model.write()` BEFORE any privileged side effect.

**PATTERN IMPLEMENTATION**

```
# Correct: defense in model.write()
class SaleOrder(models.Model):
    def write(self, vals):
        # FR-017: Gate write to x_pipeline_state_id before side-effect
        if 'x_pipeline_state_id' in vals:
            self._check_write_pipeline_state() # Raises AccessError if not allowed
        return super().write(vals)

    def action_approve(self):
        # Action method also guards write via _write_pipeline_state()
        self._write_pipeline_state(new_state, note='approved') # Calls model.write() → defense
```

**Applied to:**

- `sale.order.write()` — checks `x_pipeline_state_id` write (no direct writes allowed; must use `_write_pipeline_state()`)
- `sale.order.fulfillment.write()` — checks `tracking_number`, `tracking_state` writes (gated when address-change pending)
- `design.order.write()` — checks state write (production-team-only transitions)
- `gearment.quote.wizard.action_confirm()` — writes quote state + order state (gated to `group_sale_manager`)

**Regression Test:** See `tests/test_fr017_write_defense.py` — verify `write()` call raises `AccessError` when direct attribute write attempted; verify `action_*` methods gate internally.

**(g) Webhook HMAC Signature (Gearment v3 API)**

**ADR-018a:** Gearment webhooks signed HMAC-SHA256; verify before processing.

**SIGNATURE SCHEME****Request Header:**

```
X-Connect-Signature: algo=sha256 sig=<hex>
X-Connect-Signature-Timestamp: <unix_timestamp>
X-Connect-Signature-Nonce: <random_uuid>
```

**Verification Algorithm:**

1. Extract timestamp + nonce + algo + sig from headers
2. Check `abs(now - timestamp) < 10` (10-second clock skew tolerance)
3. Reconstruct signed payload: `url_path + nonce + timestamp + base64url(body)`
4. Compute: `HMAC-SHA256(webhook_secret, signed_payload) → expected_hex`
5. Verify: `timing_safe_compare(expected_hex, sig)`
6. If any step fails: log to audit, return 401 Unauthorized, do NOT process

**Implementation:** `multichannel_hub_fulfillment/controllers/gearment_webhook.py:verify_webhook_signature()`

**Key Points:**

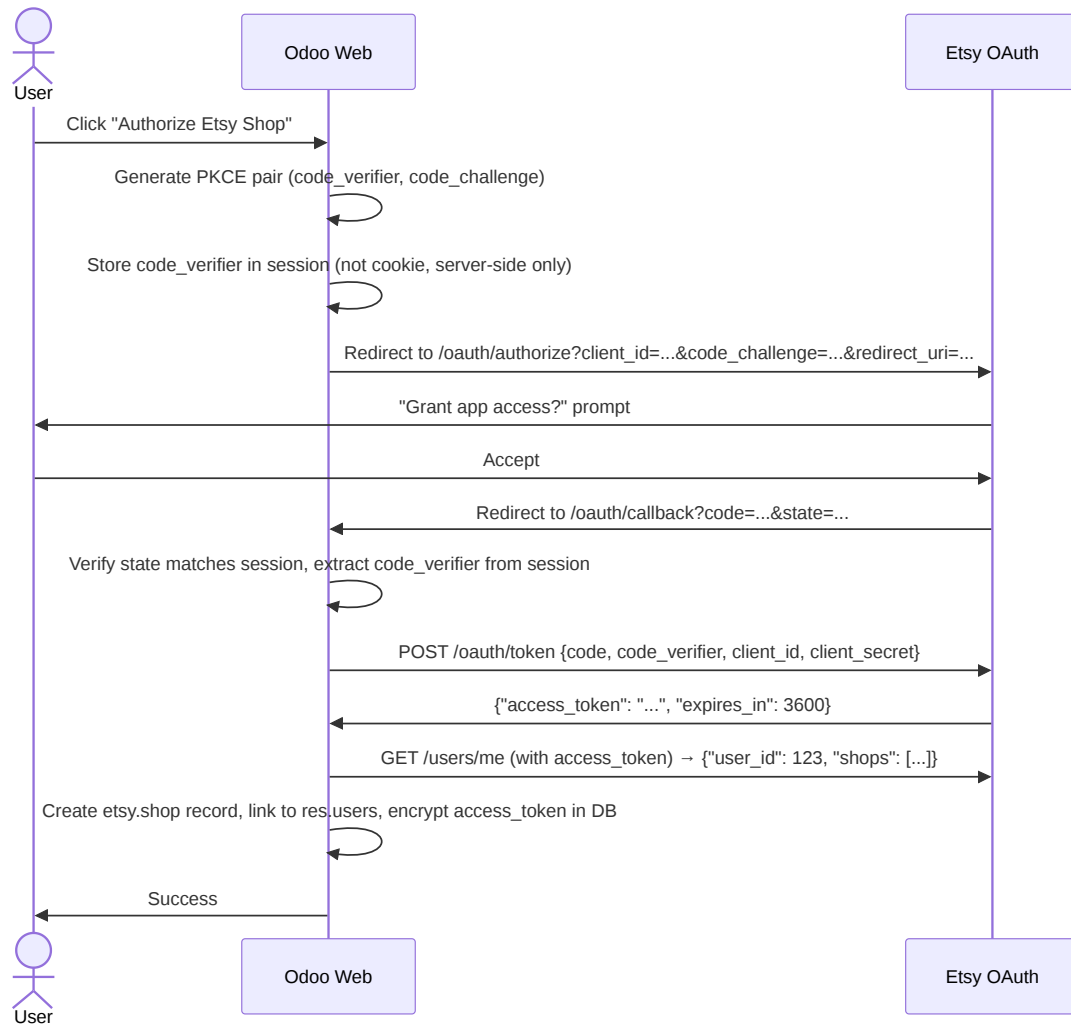
- ✓ Nonce prevents replay (per request, checked against bloom filter / rate limit)
- ✓ Timestamp prevents old replays (10s window mitigates clock drift)
- ✓ HMAC-SHA256 not vulnerable to length-extension attacks (secure for message auth)
- ⚠ Secret stored in `GEARMENT_WEBHOOK_HMAC_SECRET` env var (see § Secrets Management)

**Test:** `tests/test_gearment_webhook_hmac.py` — verify reject: (1) bad sig, (2) expired timestamp, (3) replayed nonce

**(h) OAuth2 PKCE (Etsy API)**

**ADR-018b:** Etsy OAuth grant via PKCE; stateless refresh; per-user shop scope.

## GRANT FLOW



## KEY SECURITY PROPERTIES

1. **PKCE Mitigates Code Interception:** Attacker stealing `code` cannot exchange without `code_verifier` (not transmitted over network)
2. **State Parameter:** Prevents CSRF (code parameter must match state session)
3. **Per-User Shop Scope:** `res.users.etsy_shop_ids` M2M; record rule enforces user can only see assigned shops
4. **Stateless Refresh:** No server-side session store; token stored encrypted in `etsy.shop.oauth_access_token` (AES encrypted via `ir.config_parameter` master key)
5. **Token Expiry:** Etsy tokens 1-hour TTL; refresh via `/oauth/token` before expiry (or re-auth if expired)

**Implementation:** `etsy_integration/controllers/oauth_callback.py` + `services/etsy_oauth_client.py`

**Test:** `tests/test_etsy_oauth_pkce.py` — verify reject: (1) mismatched state, (2) missing code\_verifier, (3) invalid code

## (i) Secrets Management

**Principle:** Never hardcode secrets; use environment variables or secure storage.



## SECRETS INVENTORY

| Secret                            | Storage                                                                 | Scope                        | Rotation                       |
|-----------------------------------|-------------------------------------------------------------------------|------------------------------|--------------------------------|
| Etsy OAuth Client ID              | <code>.env</code> → <code>ETSY_CLIENT_ID</code>                         | etsy_integration             | Manual (Etsy dashboard)        |
| Etsy OAuth Client Secret          | <code>.env</code> → <code>ETSY_CLIENT_SECRET</code>                     | etsy_integration             | Manual (Etsy dashboard)        |
| Etsy Shop OAuth Token             | Database (encrypted) → <code>etsy.shop.oauth_access_token</code>        | Per-shop                     | Auto-refresh (1h TTL)          |
| Gearment API Key                  | <code>.env</code> → <code>GEARMENT_API_KEY</code>                       | multichannel_hub_fulfillment | Manual (Gearment dashboard)    |
| Gearment Webhook Secret           | <code>.env</code> → <code>GEARMENT_WEBHOOK_HMAC_SECRET</code>           | multichannel_hub_fulfillment | Manual (Gearment dashboard)    |
| Gmail OAuth Token                 | Database (encrypted) → <code>ir.config_parameter</code>                 | etsy_integration             | Auto-refresh (60min cron)      |
| Google Drive Service Account JSON | File on disk → <code>secrets/gearment-*.json</code> (git-ignored)       | multichannel_hub_core        | Quarterly rotation (owner ops) |
| Odoo Admin Password               | Docker env / <code>.env</code> (dev only) → <code>ADMIN_PASSWORD</code> | System                       | Never shared (bootstrap only)  |
| PostgreSQL Password               | Docker env / <code>.env</code> → <code>DB_PASSWORD</code>               | Database                     | Bootstrap only                 |

## BEST PRACTICES

1. **Environment Variables** (`.env` file, git-ignored):

```
ETSY_CLIENT_ID=...
ETSY_CLIENT_SECRET=...
GEARMENT_API_KEY=...
GEARMENT_WEBHOOK_HMAC_SECRET=...
ADMIN_PASSWORD=...
DB_PASSWORD=...
```

2. **Docker Secrets** (staging/production):

```
# Mount secrets as read-only files:
docker run \
  --secret etsy_client_secret \
  --secret gearment_api_key \
  ...
```

3. **Database-Encrypted Tokens:**

```
# etsy.shop.oauth_access_token auto-encrypted via ir.config_parameter master key
import hashlib
from Crypto.Cipher import AES
token = etsy_shop.oauth_access_token # Transparent AES decryption via ORM
```

4. **Startup Validation:**

```
# models/__init__.py
def post_init_hook(cr, registry):
    env = api.Environment(cr, SUPERUSER_ID, {})
    required_secrets = ['ETSY_CLIENT_ID', 'GEARMENT_API_KEY']
    for secret in required_secrets:
        value = env['ir.config_parameter'].get_param(f'etsy_integration.{secret}')
        if not value:
            raise RuntimeError(f"Missing secret: {secret}")
```

**Audit:** `tests/test_no_secrets_in_code.py` — scan source code for hardcoded API keys (grep pattern match)

## (j) Compliance Checklist

**Before Production Deploy** (Phase 1 exit, P1-11 cutover):

- ☐ All new models have ACL rows in `ir.model.access.csv`
- ☐ All record rules have inline comments explaining domain
- ☐ No bare `sudo()` without justification comment
- ☐ OAuth tokens encrypted in DB or env vars (never plain text in code)
- ☐ Webhook signatures verified (HMAC for Gearment, header validation for Gmail)
- ☐ No debug statements in production code (`print()`, `_logger.info()` for debug only)
- ☐ No hardcoded secrets in manifests, configs, or source files
- ☐ Startup validation test confirms required secrets loaded
- ☐ All user-facing actions gate writes via `model.write()` (FR-017)
- ☐ Record rules tested: verify users see only their scoped records
- ☐ Group hierarchy tested: verify implied groups grant expected permissions
- ☐ Secrets rotation procedure documented (see owner-docs)
- ☐ External dependency credentials (E1 Etsy OAuth, E2 Gearment API, E3 GDrive) obtained and validated

**Automated Checks** (Pre-commit hook, `.githubhooks/`):

- ruff check for `print()`, hardcoded strings that look like secrets
- bandit check for security anti-patterns (hardcoded paths, `eval()`, weak crypto)
- Custom: grep source for env var names (allowed); flag hardcoded values

## Summary

The `odoo19_esty` system implements multi-layer defense: role-based access control (groups), record-level row security (`ir.rule`), model-level ACLs (`ir.model.access.csv`), field-level write gates (FR-017), and external signature verification (OAuth PKCE, webhook HMAC). No hardcoded secrets; all credentials via env vars or database-encrypted storage. Critical actions (state transitions, webhook processing) guarded by domain validation + signature verification before any side effect.

### Risk Assessment:

- **HIGH:** Etsy OAuth tokens (1-hour TTL, refreshed auto; compromise → order access). Mitigated by PKCE + state validation.
- **MEDIUM:** Gearment webhook secret (shared symmetric key). Mitigated by HMAC verification + 10s clock tolerance (prevents replay).
- **MEDIUM:** Email parser runs as system cron. Mitigated by regex validation + audit log.
- **LOW:** Record rules (per-user shop scope). Enforced by both record rule + ACL.

**Next:** See [README](#) for full SDS index and reading guide; [Part 1 \(Architecture\)](#) for system context and deployment.

*Part 5 authored 2026-07-03. All references verified against code as of commit `bab0263e1d0` (main, 2026-07-03).*